408 详解 · 2022 年选择题 · 从零版

只含选择题第 1–40 题，不含综合应用题，不含 2021 年真题内容。

每题固定 6 节：入手点 → 题干专有名词从零解释 → 考查知识点 → 正确选项为什么对 → 每个错误选项为什么错＋举例 → 笔记本一句话。

说明：凡题干问"不正确 / 不可能 / 不一定 / 不会影响"的题（第 3、8、17、21、22、23、29、30、32 题），第 4 节说明正确选项（即那个错述 / 不可能项）为什么成立，第 5 节说明其余选项为什么是对的（配例子），这样 6 节结构全卷统一。

默认读者是零基础跨考学生：不默认任何 408 前置知识，每题自成体系，不依赖翻王道书、不依赖其他题的解析；题干与选项里的每个术语都讲清，推理和计算的每一步都写出，不用"显然""易得""同理"这类跳步词；错误选项的例子选最直观的小数字、小例子、生活类比。

本版每题含完整题干与选项，可脱离真题册独立使用。

升级版说明：每题在原有 6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；原有题干、选项、答案与 6 节内容未作改动。

第 1 题 · 双重循环的时间复杂度（正确答案：B）

题干与选项：

下列程序段的时间复杂度是（）。

```
for (i = 1; i <= n; i *= 2)
    for (j = 0; j < i; j++)
        sum++;
```

- A. $O(\log_2 n)$
- B. $O(n)$
- C. $O(n \log_2 n)$
- D. $O(n^2)$

入手点： 别数循环层数，数 sum++ 总共执行几次，排成数列求和。

题干专有名词从零解释：

- 时间复杂度：衡量"执行次数随 n 增长的量级"，只看量级、不看常数。
- 基本操作：这里是 sum++，复杂度数的是它总共执行多少次。
- 外层循环 `i*=2`：i 每轮翻倍，依次取 1、2、4、8……。
- 内层循环：执行次数随 i 变化，不是每轮都跑 n 次。
- O(n)：执行次数与 n 同量级，即不超过 n 的常数倍。

考查知识点： 循环次数求和、等比数列、O 记号。

正确选项为什么对： i 取 1、2、4、……、2^k（其中 2^k ⩽ n，k 是使 2^k 不超过 n 的最大整数），内层执行 i 次，总次数 S = 1 + 2 + 4 + …… + 2^k。求和用"乘 2 相减"：2S = 2 + 4 + …… + 2^(k+1)，两式相减、中间项全消，得 S = 2^(k+1) − 1。又因为 2^(k+1) = 2 × 2^k ⩽ 2n，所以 S ⩽ 2n − 1 < 2n。总量被 2n 摁住，是 n 的常数倍 → O(n)。举例：n = 100 时，1+2+4+8+16+32+64 = 127，小于 2n = 200，就是 O(n) 的量级。

错误选项为什么错＋举例：

- A（$O(\log_2 n)$）：只数了外层圈数（$\log_2 n + 1$ 轮），漏了内层干的活。举例：n = 1024 时外层 11 轮，但 sum++ 执行了 2047 次，远不是 11 的量级。
- C（$O(n \log_2 n)$）：等于假设内层每轮都跑满 n 次。举例：n = 8 时实际是 1+2+4+8 = 15；按 n log n 算是 8×3 = 24 且量级不对——内层前几轮只有 1、2、4 次，根本没跑满 n。
- D（$O(n^2)$）：等于假设内外层都跑 n 次。举例：n = 100 时 n^2 = 10000，实际才 127，差了两个量级。

笔记本一句话： 第 1 题＋翻倍循环先排数列再求和，总量约等于最后一项的 2 倍，不是逐层相乘。

【图像】



双重循环每轮执行次数排成的数列与总和

【定位】 数据结构 · 第 1 章绪论 · 时间复杂度与循环次数求和

【易混对比】

- 只数外层跑了几圈，和数里面一共干了多少活，是两回事：外层圈数是 $log_2 n$ 的量级，里面一共干的活是 n 的量级，答案看的是后者。
- 每一轮都跑满 n 次，和每一轮跑的次数跟着外层变量一起变，也是两回事：前者会得到 $n \log_2 n$，后者排成数列求和才得到 n。
- 数列 $1 + 2 + 4 + \cdots$ 是越往后越大、后一项是前一项的 2 倍，这种数列叫等比数列；它的总和主要由最后一项决定，不是把层数相乘得到。

答案看的是里面一共干了多少活，不是外层跑了几圈。

【坑位】

看到两层循环，先别急着把两层的量级相乘，一定要先把每轮的次数排出来。

最容易踩的坑，是看到两层循环就直接把两层的量级相乘。外层变量 i 每次翻倍，内层跑的次数不是固定的 n，而是跟着 i 走的 $1, 2, 4, 8, \cdots$。如果把内层当成每轮都跑 n 次，就会错选 $n \log_2 n$；如果把两层都当成跑 n 次，就会错选 n^2。正确的做法是先把每轮的次数排出来，再求和。

【套路】

- 第一步，把外层变量依次取的值写出来：$1, 2, 4, 8, \cdots$。
- 第二步，把内层在每一轮执行的次数排成一列，和外层变量一一对应。
- 第三步，对翻倍的数列乘 2 相减求和，看总和被哪个量级摁住。
- 第四步，只保留量级，写出 O 的结果；常数倍不用管。

总量跟着最后一项走，而不是跟着层数乘起来走。

【类比】

像每天往存钱罐里存钱，第一天存 1 元，第二天存 2 元，第三天存 4 元，每天存的钱都是前一天的 2 倍。问一共存了多少钱，不能只数存了几天，也不能假设每天都存了最后一天那么多，要把每天的钱加起来。加到最后会发现，总数还不到最后一天存钱数的 2 倍，这就是 $O(n)$ 的感觉：总量跟着最后一项走，而不是跟着天数乘起来走。

【口诀】 翻倍循环先排队，末项翻倍定量级。

第 2 题 · 栈的入栈 / 出栈序列（正确答案：D）

题干与选项：

设 in 和 out 分别表示栈的入栈序列和出栈序列，栈 ST 初始为空。下列叙述中正确的是（ ）。

- A. 由入栈序列 in 不能判断出栈序列 out 是否可能
- B. 由出栈序列 out 不能判断入栈序列 in 是否可能
- C. 入栈序列 in 与出栈序列 out 一定不同
- D. 入栈序列 in 与出栈序列 out 可能互为倒序

入手点： 栈只能从顶进出，合法性靠模拟验证；先看两个极端（边入边出、全入再出）。

题干专有名词从零解释：

- 栈：后进先出（LIFO）的结构，只能从一端（栈顶）进出。
- 入栈序列 in：元素进栈的顺序。
- 出栈序列 out：元素出栈的顺序。
- 栈顶：唯一能进出的口子，任一时刻只能弹出"最后入栈且还没出栈"的那个元素。
- 合法序列：按规则实际能走出来的出入顺序。

考查知识点： 栈的 LIFO 特性与出栈序列合法性判断。

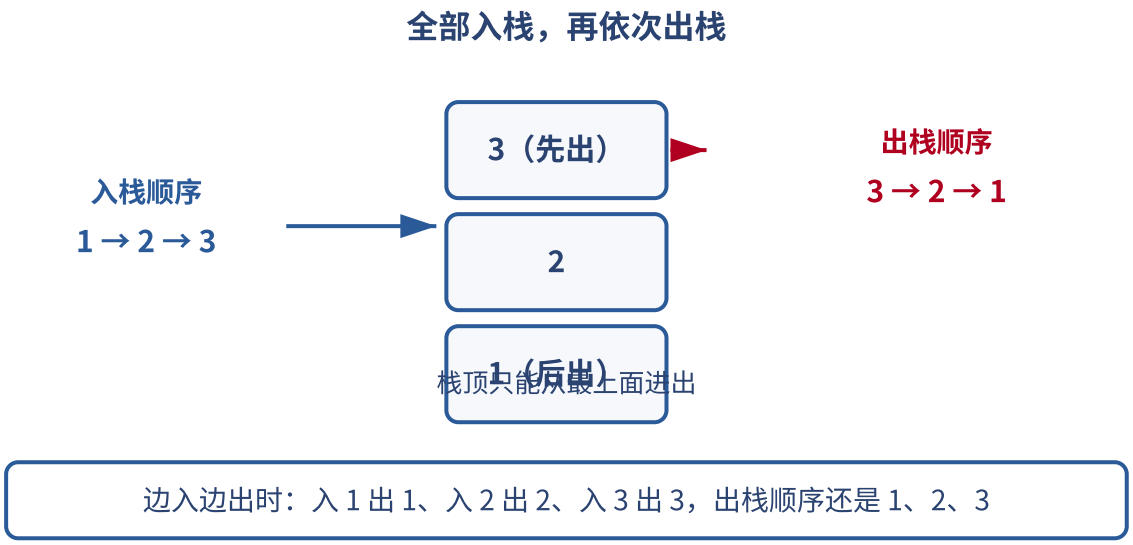
正确选项为什么对： 全部入栈再依次出栈，out 恰好是 in 的倒序。举例：in = 1、2、3，全入栈后栈内从底到顶是 1、2、3，依次弹出 3、2、1，out = 3、2、1，正好是倒序——所以"可能互为倒序"成立。

错误选项为什么错＋举例：

- A（给定 in 就不能判断 out 是否可能）：错，模拟就能判。举例：in = 1、2、3，out = 3、1、2：模拟入 1、入 2、入 3、弹 3 ✓，此时栈顶是 2 不是 1，判死刑——out 不合法。能判断。
- B（由 out 反推 / 验证 in 也不可能）：错，同样可以模拟验证。举例：out = 2、1、3，in = 1、2、3：入 1、入 2、弹 2、弹 1、入 3、弹 3，全对上，合法。
- C（in 与 out 一定不同）：错，边入边出就相同。举例：in = 1、2、3：入 1 弹 1、入 2 弹 2、入 3 弹 3，out = 1、2、3，和 in 一模一样。

笔记本一句话： 第 2 题＋栈序列合法性靠模拟判，倒序和全同都是合法极端，别把"可能"当"必然"。

【图像】



全部入栈后再依次出栈得到倒序的栈示意图

【定位】 数据结构 · 第 3 章栈和队列 · 栈的出栈序列合法性

【易混对比】

- “可能互为倒序”和“一定互为倒序”是两回事：前者是说存在一种走法能走出倒序，后者是说所有走法都是倒序；本题问的是前者，一个例子就能成立。
- “全部入栈再依次出栈”和“边入栈边出栈”是两种走法：前者得到倒序，后者可以得到和入栈顺序一样的出栈顺序，两种都是合法的。
- 入栈顺序和出栈顺序本身也是两回事：入栈顺序是元素进来的先后，出栈顺序是元素出去的先后，两者可以相同，也可以相反，还可以是其他合法的顺序。

可能只要一个走得通的例子就能成立，一定才需要所有走法都成立。

【坑位】

看到“可能”二字，先别把它读成“一定”，先问自己能不能造出一个例子。

最容易踩的坑，是把“可能”读成“一定”。看到“互为倒序”就觉得不对，理由是边入边出时出栈顺序明明可以和入栈顺序一样。但题目说的是“可能”，只要有一种走法能得到倒序就够了，全部入栈再依次出栈就是这种走法。做带“可能”二字的判断题时，先问自己：能不能造出一个例子？造得出来，这个说法就成立。

【套路】

- 第一步，拿最小的例子动手模拟，比如入栈顺序是 $1, 2, 3$。
- 第二步，按题目给出的出栈顺序一步一步走：要出栈的元素如果正好在栈顶，就弹出；如果不在栈顶，就继续入栈，直到它来到栈顶。
- 第三步，如果走到某一步，要出的元素既不在栈顶、入栈序列又已经用完了，这个出栈顺序就是不合法的。
- 第四步，要证明“可能”，只要给出一种走得通的走法；要证明“一定”，才需要把所有走法都考虑到。

【类比】

像摞盘子：盘子只能从最上面一个一个往上摞，也只能从最上面一个一个往下拿。把 $1, 2, 3$ 号盘子按顺序摞上去，再从最上面往下拿，拿到的顺序就是 $3, 2, 1$，正好倒过来。如果摞一个拿一个，拿到的顺序就还是 $1, 2, 3$。两种拿法都合规矩，倒序只是其中一种拿法，不是唯一拿法。

【口诀】 栈顶进出要模拟，全入全出是倒序。

第 3 题 · 中序遍历中相邻结点的关系（正确答案：B，仅 III）

题干与选项：

设 p、q 是二叉树中的两个结点，在中序遍历序列中 p、q 相邻且 p 在 q 之前。下列关于 p 与 q 关系的叙述中，不可能的是（）。

I. q 是 p 的双亲

II. q 是 p 的右孩子

III. q 是 p 的右兄弟

IV. q 是 p 的双亲的双亲

- A. 仅 I
- B. 仅 III
- C. 仅 II、III
- D. II、IV

入手点： 中序相邻只看遍历顺序不看辈分，逐项造小树验证，不能凭感觉。

题干专有名词从零解释：

- 二叉树：每个结点最多 2 个孩子（左孩子、右孩子）的树。
- 中序遍历：访问顺序 = 左子树 → 根 → 右子树。
- 双亲：父结点；右孩子：排在右边的孩子；右兄弟：同一双亲的、排在右边的孩子。
- 双亲的双亲：祖父。
- 中序后任：在中序序列里紧跟在某结点后面的结点。规律：x 有右子树时，后任是右子树中最左的结点；x 无右子树时，往上找，直到某个祖先是"从左孩子上去的"，那个祖先就是后任。

考查知识点： 中序遍历的后任规律。

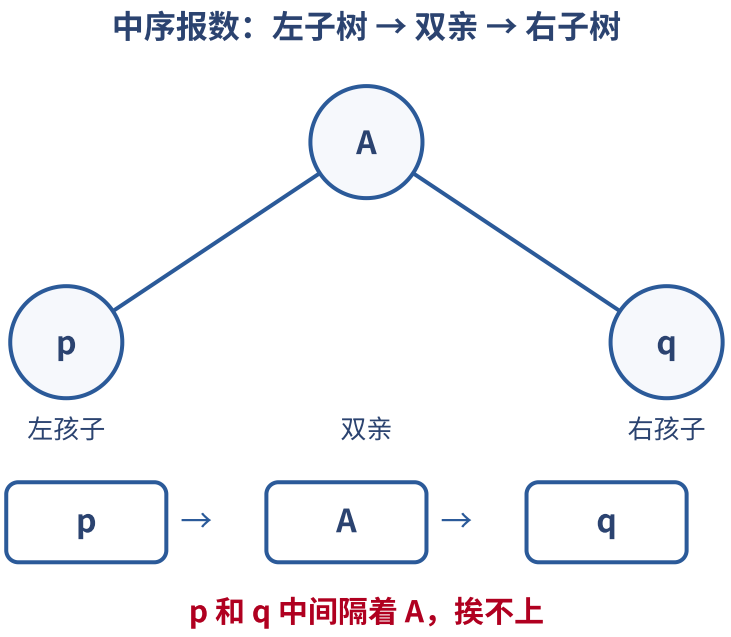
正确选项为什么对（本题找的是不可能项）： III（q 是 p 的右兄弟）不可能：中序顺序是"p 所在左子树 → 双亲 → 右子树"，双亲永远夹在两兄弟之间。举例：根 A，左孩子 p，右孩子 q，中序是 p、A、q——p 的后任是 A 不是 q，中间隔着 A，两人永远挨不上。

错误选项为什么错＋举例（其余三项都可能，配构造）：

- A（仅 I）：I 可能，选它等于说只有 I 不可能，错。举例：根 q，左孩子 p 且 p 无右孩子，中序是 p、q，p 的后任就是 q。
- C（仅 II、III）：II 可能，所以 C 错——虽然 III 确实不可能，但 C 把 II 也算进了不可能，错一半也是错。举例：p 有右孩子 q 且 q 无左孩子，中序先 p，再进右子树第一个就是 q，紧挨着，II 是可能的。
- D（II、IV 的组合）：II、IV 都可能。举例（IV）：q 的左孩子是 m，m 的右孩子是 p，中序是 m、p、q——p 的后任是 q，而 q 正是 p 的双亲 m 的双亲。

笔记本一句话： 第 3 题＋中序相邻看遍历顺序不看辈分：双亲、右孩子、祖父都能紧挨，唯独右兄弟中间隔着双亲。

【图像】



中序遍历中右兄弟之间隔着双亲的示意图

【定位】 数据结构 · 第 5 章树与二叉树 · 二叉树的中序遍历与后任

【易混对比】

- 右孩子和右兄弟是两回事：右孩子是某个结点的、排在右边的孩子；右兄弟是和某个结点共用同一个双亲、排在右边的孩子。一个结点可以是另一个结点的右孩子，同时不是它的右兄弟。
- 双亲和双亲的双亲也是两回事：双亲是往上数一代，双亲的双亲是往上数两代，也就是祖父；中序相邻的两个结点，辈分上可以差一代，也可以差两代。
- “在树里挨着”和“在中序序列里挨着”更是两回事：树里挨着说的是父子、兄弟这种直接关系，中序序列里挨着说的只是报数时前后脚，辈分远近不决定报数的前后。

中序里挨着不看辈分远近，只看报数顺序。

【坑位】

别凭辈分猜报数顺序，两兄弟在树里挨着，报数时中间还隔着双亲。

最容易踩的坑，是凭辈分猜报数顺序，觉得两兄弟在树里挨着，报数时也该挨着。但中序报数的顺序是：先报左边的子树，再报双亲，最后报右边的子树。两兄弟之间永远夹着双亲，左边的兄弟报完，下一个报的是双亲，不是右边的兄弟。所以右兄弟不可能成为中序里的下一个结点，而双亲、右孩子、祖父都有可能成为下一个。

【套路】

- 第一步，先确定下一个结点的两条路：一个结点如果有右子树，就找右子树里最左边的结点。
- 第二步，如果这个结点没有右子树，就往上找从左孩子走上来的祖先，这个祖先就是下一个结点。
- 第三步，对每个候选关系画一棵最小的树，树上只有两三个结点就够了，照着中序顺序报一遍，看两个结点是不是真的前后脚。
- 第四步，凡是问“不可能”的题，只要有一个候选关系在任何小树里都走不通，它就是答案。

【类比】

像一家人按“左边的孩子先报、家长再报、右边的孩子后报”的规矩报数。左边的孩子报完，下一个报的必然是家长，不可能是右边的孩子，因为家长的报数位置就夹在两个孩子中间。报数的前后脚看的是报数规矩，不是看谁和谁在家里关系更近。

【口诀】 中序相邻看顺序，兄弟中间隔双亲。

第 4 题 · 三叉树的最小高度（正确答案：C，6）

题干与选项：

一棵三叉树共有 244 个结点（规定叶结点的高度为 1），则该树的高度至少是（）。

- A. 8
- B. 7
- C. 6
- D. 5

入手点： 问"至少"先算满装容量，找第一个装得下的高度。

题干专有名词从零解释：

- 三叉树：每个结点最多 3 个孩子的树。
- 高度：本题口径是叶结点高度为 1，即高度数的是层数。
- 满装：每层都排满的状态，此时同高度的树结点最多。
- 结点数：树里结点的总数，本题 244。

考查知识点： m 叉树高度与结点数的关系。

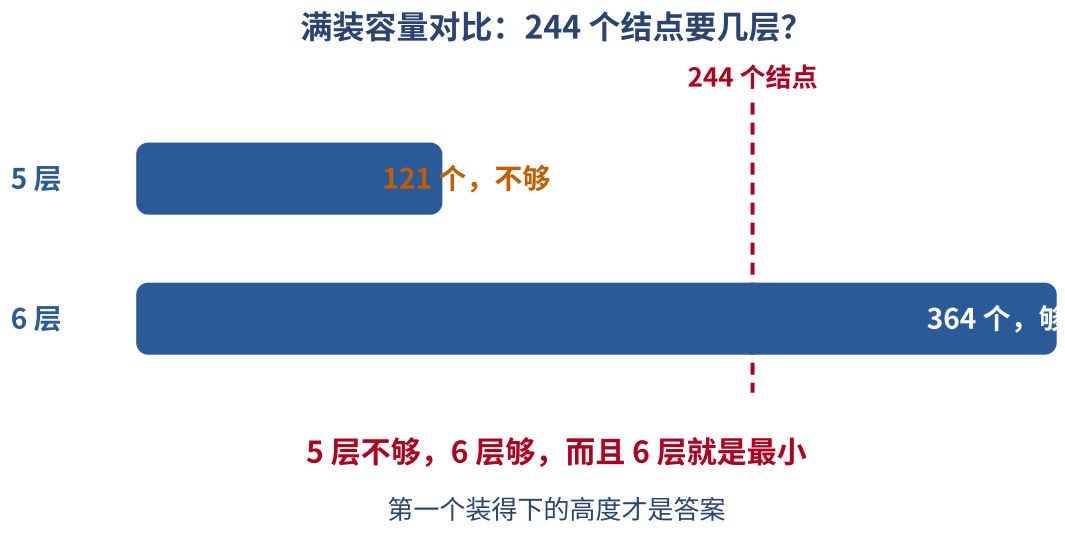
正确选项为什么对： 高度 h 的三叉树满装时每层分别是 1、3、9……个，总数 S = 1 + 3 + …… + 3^(h−1)。求和用"乘 3 相减"：3S = 3 + 9 + …… + 3^h，两式相减得 2S = 3^h − 1，所以 S = (3^h − 1)/2。代入：h = 5 → (243 − 1)/2 = 121 < 244，装不下；h = 6 → (729 − 1)/2 = 364 ⩾ 244，装得下。所以至少 6。

错误选项为什么错＋举例：

- A（8）：容量早就够了，"至少"取最小。举例：h = 8 满装 (6561 − 1)/2 = 3280，远超 244，不是"至少"。
- B（7）：h = 7 时满装 (2187 − 1)/2 = 1093 ⩾ 244，也装得下，但 6 就够了，7 不是最小。
- D（5）：满装才 121 个。举例：5 层三叉树哪怕每层塞满也只有 121 个位置，244 个结点根本塞不进去。

笔记本一句话： 第 4 题＋"n 个结点 m 叉树至少多高"先算满装 (m^h − 1)/(m − 1)，找第一个装得下的 h。

【图像】



三叉树 5 层满装 121 个不够、6 层满装 364 个够的容量对比

【定位】 数据结构 · 第 5 章树与二叉树 · 三叉树的高度与结点数

【易混对比】

- “装得下”和“至少”是两回事：7 层、8 层都装得下，但题目问的是至少，也就是第一个装得下的高度，不能挑大的。
- 每层最多能放几个，和所有层加起来一共能放几个，也是两回事：第 5 层一层最多放 81 个，但 5 层加起来满装也只有 121 个，离 244 个还差得远。
- 高度的数法也要先看清：本题把只有自己、没有孩子的结点高度记为 1，也就是高度数的是层数；如果有的口径把这种结点记为 0，算出来的高度会差 1，做题前先认准题干的口径。

装得下还不够，要取第一个装得下的高度，那才是至少。

【坑位】

别把 3 的 5 次方当成 5 层能装的总数，满装总数还要把各层加起来算。

最容易踩的坑，是看到 243 这个数就以为 5 层够了。243 只是 3 的 5 次方，不是 5 层满装的总数。满装总数是把第 1 层到第 5 层的个数加起来：$1+3+9+27+81=121$，只有 121 个位置，244 个结点根本塞不进去。另一个坑是算出 6 层够了还往上选 7 层，忘了“至少”要取最小的。

【套路】

- 第一步，先算满装容量：每层排满时，第 1 层 1 个，第 2 层 3 个，第 3 层 9 个，依此类推，把各层加起来。
- 第二步，从低往高逐层试，找到第一个容量不小于结点总数的高度。
- 第三步，这个高度就是答案，不要再往上加。

【类比】

像停车场每层车位翻 3 倍：第 1 层 1 个车位，第 2 层 3 个，第 3 层 9 个，依此类推。要停 244 辆车，不能只看某一层有多少车位，要把各层车位加起来看够不够；5 层加起来只有 121 个车位，肯定不够，6 层加起来有 364 个车位，才够停，而且 6 层就是最少要建的层数。

【口诀】 满装容量先算清，装下为止取最小。

第 5 题 · 哈夫曼编码树 vs 定长编码树（正确答案：D）

题干与选项：

设字符集 S 中有 n（n > 2）个字符，用二叉树表示哈夫曼编码集得到 T_1，用二叉树表示定长编码集得到 T_2。下列叙述中正确的是（ ）。

- A. T_1 与 T_2 的结点数相同
- B. T_1 的高度大于 T_2 的高度
- C. 频次不同的两个字符在 T_1 中一定处于不同层
- D. T_2 中所有叶子结点都在同一层

入手点： 两棵树的"长相"完全不同：定长看个数、哈夫曼看频次。

题干专有名词从零解释：

- 字符集 S：要编码的字符集合，n > 2 个字符。
- 编码树：用二叉树表示编码，叶子是字符，从根往下左走 0、右走 1，叶子离根的距离就是编码长度。
- 哈夫曼编码：按频次建树，出现越多的字符离根越近、编码越短。加权路径长度 = 每个字符的编码长度 × 它的出现频次，全部加起来；哈夫曼建树让这个和最小，所以叫"最优"编码。
- 定长编码：每个字符编码长度相同，和频次无关。长度为什么是 $\lceil \log_2 n \rceil$ 位：b 位二进制只有 2^b 种组合，要区分 n 个字符必须 2^b ⩾ n，b 至少 $\log_2 n$；位数是整数，不够就向上取整，记作 $\lceil \log_2 n \rceil$。举例：n = 5 时 $\log_2 5 \approx 2.32$，2 位只有 4 种组合不够，向上取整得 3 位。
- 结点：叶子 = 字符；内部结点 = 建树时合并产生的结点。
- 层 / 深度：叶子离根的距离，等于编码长度。

考查知识点： 哈夫曼树与定长编码树的结构差异。

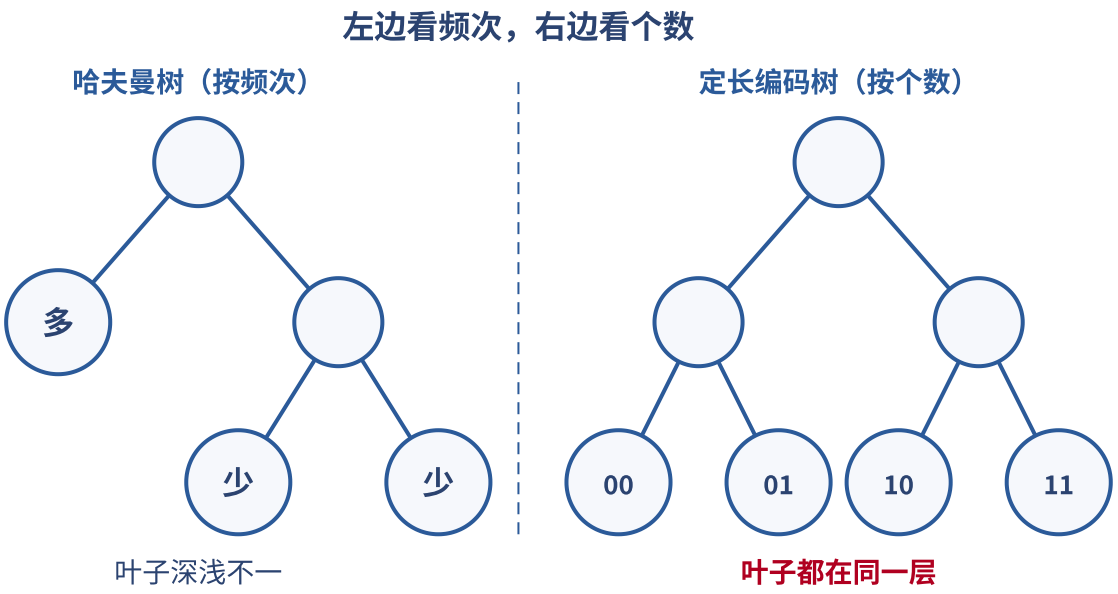
正确选项为什么对： 定长编码人人等长，叶子必然在同一层。举例：n = 4 个字符，定长 2 位：00、01、10、11，四个叶子全在第 2 层。

错误选项为什么错＋举例：

- A（两树结点数相同）：哈夫曼 n 个叶子合并 n − 1 次——为什么是 n − 1 次：开始 n 个叶子结点，每次合并把两个结点合成一个（两个变一个，结点总数净减 1），合到只剩 1 个根要合 n − 1 次，每次产生 1 个内部结点，所以内部结点 n − 1 个，总数 n + (n − 1) = 2n − 1；定长树结点数取决于补齐规模。举例：n = 5 时哈夫曼共 9 个结点；定长 3 位要 8 个叶子位置，满树结点数是 15——反正不一定等于 9。
- B（哈夫曼树高度一定大于定长树）：高度可大可小，"大于"说死了。举例：4 个字符频次均匀（各 25%）时哈夫曼树就是满树，高 2，和定长一样；频次悬殊（如 70%、10%、10%、10%）时哈夫曼树才会被拉高。
- C（频次不同的字符在哈夫曼树中一定不同层）：错，可以同层。举例：字符频次 5、4、3、2，建树过程可能让频次 3 和 2 的字符落在同一深度（编码一样长）——哈夫曼只保证总加权路径最短，不保证频次不同就分层。

笔记本一句话： 第 5 题＋哈夫曼看频次（结点恒 2n−1、叶子可不同层），定长看个数（叶子必同层、与频次无关）。

【图像】



哈夫曼树按频次长短不一与定长编码树叶子同层的结构对比

【定位】 数据结构 · 第 5 章树与二叉树 · 哈夫曼树与定长编码树

【易混对比】

- 按出现多少次来排，和按一共有几个字符来排，是两回事：前者决定哈夫曼树的长相，后者决定定长编码树的长相。
- 哈夫曼树里叶子离根的距离可以不一样：出现多的字符离根近、编码短，出现少的字符离根远、编码长；定长编码树里所有叶子离根的距离都一样，和出现多少次没有关系。
- 两棵树的结点总数也不一定相同：哈夫曼树有 n 个叶子时，内部结点是 $n-1$ 个，总数是 $2n-1$ 个；定长编码树的结点数要看补齐到多少个叶子位置，不一定等于这个数。

定长叶子必在同一层，哈夫曼叶子可以不在同一层。

【坑位】

频次不同不等于层一定不同，哈夫曼只保证总代价最小，不保证频次不同就分层。

最容易踩的坑，是觉得出现次数不一样，编码长度就一定不一样。哈夫曼建树只保证一件事：把每个字符的编码长度乘以它的出现次数、全部加起来的总和最小。两个出现次数不同的字符，完全可以落在同一层、编码一样长，只要这样拼出来的总和仍然最小。

【套路】

- 第一步，先问这棵树看频次还是看个数：看频次就往哈夫曼想，看个数就往定长想。
- 第二步，定长编码先算位数：几位二进制够区分所有字符，就取几位，不够就往上取整。
- 第三步，哈夫曼树先数叶子：有几个字符就有几个叶子，再数内部结点，总数是叶子数加上内部结点数。
- 第四步，凡是带“一定”二字的说法，先找一个反例：能造出反例，这个说法就不成立。

【类比】

像整理快递柜：常用的包裹放在门口随手可取的格子，不常用的包裹放在里屋深处的格子，格子深浅按使用多少次来定，这就是哈夫曼的思路。定长编码则像给每个包裹发一个同样大小的格子，不管常用还是不常用，格子大小一律相同，格子排在同一排。

【口诀】 哈夫曼看频次，定长只看个数齐。

第 6 题 · 无向图连通与边数的关系（正确答案：D）

题干与选项：

对无向图 G = (V, E)，下列叙述中正确的是（ ）。

- A. 若 |V| > |E|，则 G 一定是连通图
- B. 若 |V| < |E|，则 G 一定是连通图
- C. 若 |V| = |E| − 1，则 G 一定是不连通图
- D. 若 |V| > |E| + 1，则 G 一定是不连通图

入手点： 连通只认一条硬线：边数不够一定不连通，边多只能说可能。

题干专有名词从零解释：

- 无向图：边没有方向的图，记作 G = (V, E)。
- |V|：顶点数；|E|：边数。
- 连通：任意两个顶点之间都有路可达。
- 树：连通且无环的图，边数恰好是 |V| − 1，一条边都不能再少。
- 连通分量：连通的一块；不连通的图由多个分量组成。

考查知识点： 连通图的边数下界。

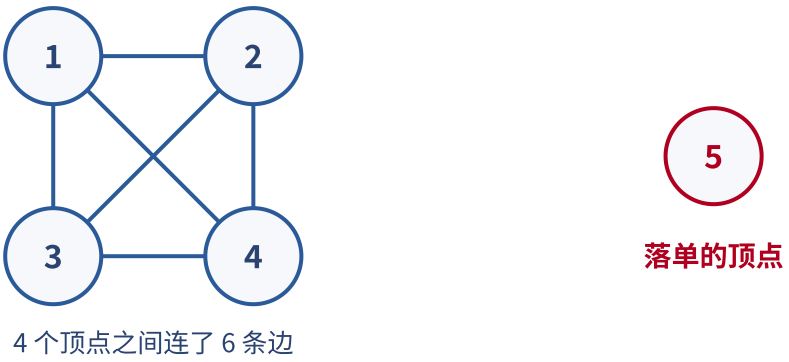
正确选项为什么对： D 说 |V| > |E| + 1，等价于 |E| < |V| − 1。为什么至少 |V| − 1 条边才可能连通：把顶点一个一个连进已连通的部分，每加一条边最多让一个新顶点连入；从 1 个顶点起，要连进剩下 |V| − 1 个顶点，至少 |V| − 1 条边。边数不到这个数，一定不连通。举例：5 个顶点只有 3 条边，哪怕 3 条边全不重复，也最多串起 4 个顶点，至少剩 1 个孤立点。

错误选项为什么错＋举例：

- A（只凭 |V| > |E| 就断定连通与否）：两种结果都有，断定不了。举例：4 个顶点 3 条边可以是树（连通，如 1–2–3–4 链）；4 个顶点 2 条边一定不连通。同样满足 |V| > |E|，结论却相反。
- B（|V| < |E| 边多就一定连通）：错。举例：5 个顶点 6 条边：把 4 个顶点连成带环的团（6 条边全堆在里面），第 5 个顶点孤立——边再多照样不连通。
- C（|E| = |V| + 1 就一定不连通）：错，两种都能构造。举例：5 个顶点 6 条边，可以在树（4 条边）上再加 2 条边成环，照样连通；也可以 4 个顶点连成 K4（6 条边全堆在里面）＋1 个孤立点，6 条边、5 个顶点，|E| = |V| + 1，照样不连通。

笔记本一句话： 第 6 题＋边数 < 顶点数 − 1 必不连通；边再多也只能说可能连通，反例是"有环分量＋孤立点"。

【图像】



5 个顶点 6 条边仍不连通的反例：4 个顶点抱团加 1 个孤立点

【定位】 数据结构 · 第 6 章图 · 连通图的边数下界

【易混对比】

- 边数不够，和边数够了，是两回事：边数不够时一定不连通，边数够了时只是有可能连通，还要看边是不是都堆在了一处。
- 可以把边数下限想成一道门槛：没跨过门槛就一定不成，跨过门槛也不一定成；前一种“没达标就一定不成、达标了还不一定成”的关系，叫必要条件；那种“一满足就一定成”的关系，才叫充分条件；本题里边数够只是必要条件，不是充分条件。
- “连通”本身也要认清：任意两个顶点之间都有路可达，才叫连通；只要有一个顶点落单，整张图就不连通，不管其他顶点之间连得多密。

边数不到下限一定不连通，边数超过下限只能说可能连通。

【坑位】

边再多也别直接断定连通，先看有没有被落下的孤立点。

最容易踩的坑，是觉得边数多就一定连通。边可以全部堆在几个顶点之间，堆出很多环，却把剩下的顶点晾在一边。判断连通时不能只数边的总数，还要看边有没有把所有顶点都串起来；一个孤立点就能推翻“一定连通”的结论。

【套路】

- 第一步，先算下限：把顶点数减 1，得到连通至少需要的边数。
- 第二步，把题目给的边数和下限比一比：边数比下限少，直接判定一定不连通。
- 第三步，边数不少于下限时，再造反例验证：试着把边堆在一处、留一个顶点落单，造得出来就说明不能断定连通。
- 第四步，凡是带“一定”的说法，只要一个反例就能推翻，不用把所有情况都列出来。

【类比】

像给一排灯泡拉线：要把 n 个灯泡都串起来，最少要拉 $n-1$ 根线，少一根就一定有灯泡没接上。线拉得再多，如果都绕在前几个灯泡之间、最后一个灯泡一根线都没接上，整排灯还是不通。数线够不够只是第一步，还要看

线有没有接到每一个灯泡上。

【口诀】 边不够数必断线，边够数未必连成片。

第 7 题 · AOE 网中时间余量最大的活动（正确答案：B，活动 g）

题干与选项：

下图所示的 AOE 网中有 10 个活动 a–j（顶点表示事件，边表示活动，边权表示活动耗时；顶点 1 为源点，顶点 6 为汇点）。各活动与边的对应关系如下：a：1→2，权 2；b：1→3，权 5；c：2→3，权 1；d：2→4，权 3；e：3→4，权 3；f：3→5，权 4；g：3→6，权 1；h：4→5，权 1；i：4→6，权 4；j：5→6，权 1。其中时间余量最大的活动是（ ）。

（图示说明：原题 AOE 图为图片，无法直接贴图；上表已完整列出图的全部顶点、边与权，已与原卷在线预览核对一致。）

- A. 活动 c
- B. 活动 g
- C. 活动 h
- D. 活动 j

入手点： 余量题只有一条路：正推 ve 取 max、倒推 vl 取 min、余量 = vl(终) − ve(始) − 权。

题干专有名词从零解释：

- AOE 网：边表示活动、顶点表示事件的有向无环图。
- 事件（顶点）：某个时刻点，本身不耗时间。
- 活动（边）：耗时间的任务，边权 = 耗时。
- 最早发生时间 ve：事件最早能发生的时刻，从源点正推、取 max（前面的活动都完成才能发生）。
- 最迟发生时间 vl：事件最晚必须发生的时刻，从汇点倒推、取 min（后面的活动都不能被耽误）。
- 时间余量：活动最迟开始时刻 − 最早开始时刻 = vl(终点) − ve(起点) − 边权，拖多久都不影响工期。
- 关键活动：余量为 0 的活动；关键路径：全由关键活动组成的路，决定总工期。

考查知识点： AOE 网 ve / vl 计算与活动余量。

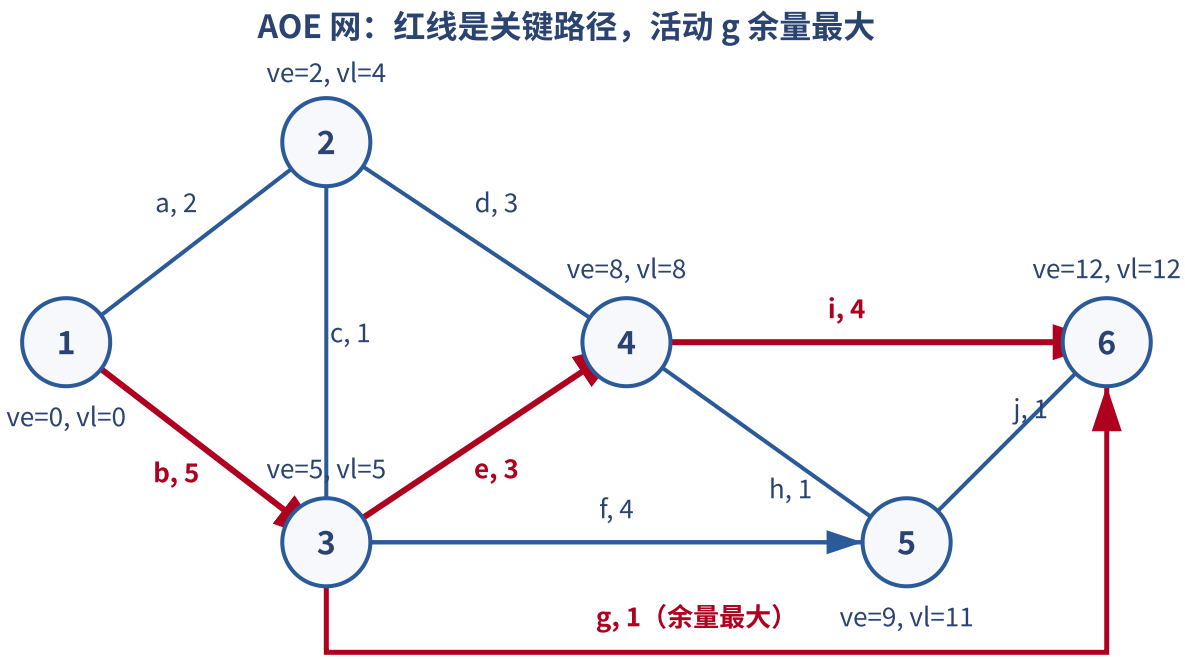
正确选项为什么对： 先把活动和边对应起来（按字母顺序）：a：顶点 1→2，权 2；b：1→3，权 5；c：2→3，权 1；d：2→4，权 3；e：3→4，权 3；f：3→5，权 4；g：3→6，权 1；h：4→5，权 1；i：4→6，权 4；j：5→6，权 1。下面正推、倒推的每一步都按这张表算。正推 ve：ve(1) = 0，ve(2) = 2，ve(3) = max(5, 2+1) = 5，ve(4) = max(2+3, 5+3) = 8，ve(5) = max(5+4, 8+1) = 9，ve(6) = max(8+4, 5+1, 9+1) = 12（工期 12）。倒推 vl：vl(6) = 12，vl(5) = 11，vl(4) = min(12−4, 11−1) = 8，vl(3) = min(8−3, 11−4, 12−1) = 5，vl(2) = min(5−1, 8−3) = 4，vl(1) = min(4−2, 5−5) = 0。g 是从顶点 3 到顶点 6、权为 1 的活动，余量 = 12 − 5 − 1 = 6。全场余量：a 2、b 0、c 2、d 3、e 0、f 2、g 6、h 2、i 0、j 2，g 最大；关键路径是 1→3→4→6（b、e、i）。

错误选项为什么错＋举例：

- A（c 余量最大）：c 的权是 1，余量 = vl(3) − ve(2) − 1 = 5 − 2 − 1 = 2，小于 g 的 6。举例：c 最多拖 2 天，再多就拖工期；g 能拖 6 天。
- C（h 余量最大）：h 的余量 = vl(5) − ve(4) − 1 = 11 − 8 − 1 = 2，小于 g 的 6。举例：h 最多拖 2 天，再多就拖工期；g 能拖 6 天。
- D（j 余量最大）：j 的余量 = vl(6) − ve(5) − 1 = 12 − 9 − 1 = 2，同样不及 g。

笔记本一句话： 第 7 题＋AOE 余量 = vl(终) − ve(始) − 权；正推取 max、倒推取 min，余量 0 的就是关键活动。

【图像】



AOE 网各事件的最早与最迟发生时间及活动 g 的余量

【定位】 数据结构 · 第 6 章图 · AOE 网与活动时间余量

【易混对比】

- 事件最早能发生的时刻，和事件最晚必须发生的时刻，是两回事：前者从起点往后推、每一步取最大的那个值，后者从终点往前倒推、每一步取最小的那个值。
- 活动的最早开始时刻，和活动本身的耗时，也是两回事：最早开始时刻是起点事件最早能发生的时刻，余量还要再减去活动耗时，不能只看开始时刻。
- 余量为零的活动，和余量最大的活动，位置正好相反：余量为零的活动拖不得，拖一天工期就拖一天；余量最大的活动最能拖，拖到余量用完才影响工期。

余量最大的活动不一定是耗时最短的活动，要算完所有活动再比。

【坑位】

耗时短的活动不一定余量大，余量还要看终点最晚能拖到哪天。

正推和倒推的取大取小一旦颠倒，余量全盘算错，做完一定要回头核对一次方向。

最容易踩的坑，是只盯着活动本身耗时短，就以为它的余量大。活动 g 本身只耗时 1，但它的终点是最晚才必须发生的终点事件，所以能拖的时间反而最长。余量看的是整条路的松紧，不是单个活动的长短；只比耗时不比余量，就会把真正能拖的活动漏掉。

【套路】

- 第一步，从起点往后正推，每一步取大：一个事件前面有几个活动，就取其中最大的完成时刻。
- 第二步，从终点往前倒推，每一步取小：一个事件后面有几个活动，就取其中最小的倒推时刻。
- 第三步，用终点最晚时刻减起点最早时刻，再减活动耗时，得到每个活动的余量，公式写成 $余量 = vl(终点) - ve(起点) - 权$。
- 第四步，把所有活动的余量比一比，最大的就是答案；余量为零的活动连起来就是关键路径。

【类比】

像装修赶工期：每道工序最早能开工的日子，要看前面的工序是不是都干完了，取最晚完工的那一道；最晚必须开工的日子，要看后面的工序还能拖几天，取最紧的那一道。两者的差再减去这道工序自己要花的天数，就是这道工序能拖的天数。有的工序本身只花一天，但因为后面的工序不着急，反而能拖最久。

【口诀】 正推最大倒推最小，余量迟早耗时扣。

第 8 题 · 5 阶 B 树删除关键字后的根（正确答案：D，60、90、110、350 不可能）

题干与选项：

一棵 5 阶 B 树，其根结点的关键字序列为 [60, 90, 260, 350]，五个孩子结点的关键字序列依次为 [30, 50]、[70, 80, 85]、[100, 110]、[280, 300]、[400, 500]。删除关键字 260 并经过必要的调整后得到新树 T_1。下列选项中，不可能是 T_1 的根结点关键字序列的是（ ）。

- A. 60、90、280
- B. 60、90、350
- C. 60、85、110、350
- D. 60、90、110、350

入手点： 删内部关键字分两步：先找前驱 / 后继顶上，再处理叶子亏空（借或并）；根要么被换一个关键字，要么少一个关键字。

题干专有名词从零解释：

- 5 阶 B 树：每个结点最多 4 个关键字、最多 5 个孩子；除根外，每个结点至少 $\lceil 5/2 \rceil - 1 = 2$ 个关键字（5 ÷ 2 = 2.5，孩子数是整数，向上取整得 3 个孩子；关键字数 = 孩子数 − 1，所以最少 2 个）。
- 根：顶层结点；内部结点：非叶子结点（本题 260 就在根里）。
- 前驱：左子树里的最大值；后继：右子树里的最小值。
- 亏空：结点关键字少于下限（本题少于 2 个），必须调整。
- 借：某个兄弟有 ⩾ 3 个关键字时，父结点关键字下沉补亏，兄弟的最大（或最小）关键字上顶到父结点——根会被换掉一个关键字。
- 合并：兄弟都只有 2 个、借不出时，把亏空结点＋一个兄弟＋中间的父关键字合成一个结点——根会少一个关键字。

考查知识点： B 树删除内部关键字的调整规则。

正确选项为什么对（本题找的是不可能项）： 根里出现 110 只能是走了前驱顶替；但前驱一走，叶子 [100, 110] 删 110 后剩 [100] 必亏空，后续无论借（左兄弟 [70, 80, 85] 有 3 个，借得出：父关键字 90 下沉补成 [90, 100]，兄弟最大值 85 上顶，根变 [60, 85, 110, 350]）还是合并（根都会丢一个关键字），根都不可能保持"60、90 原样＋多一个 110、350 不动"。所以 D 是白嫖的调整，不可能。

错误选项为什么错＋举例（其余三项都可能，配走向）：

- A（60、90、280 可能）：走后继 280 顶替，根先变 [60, 90, 280, 350]；叶子 [280, 300] 删 280 剩 [300] 亏空，向右与 [400, 500] 合并、父关键字 350 下沉，根丢 350 → [60, 90, 280]。
- B（60、90、350 可能）：同样走后继，叶子亏空后向左与 [100, 110] 合并、父关键字 280 下沉，根丢 280 → [60, 90, 350]。
- C（60、85、110、350 可能）：走前驱 110 顶替，叶子亏空后向左兄弟借：90 下沉、85 上顶 → [60, 85, 110, 350]。

笔记本一句话： 第 8 题＋B 树删内部关键字：前驱 / 后继顶上，叶子亏空必借或必并——根要么被换一个关键字，要么少一个关键字，没有白嫖。

【图像】



B 树删内部关键字：前驱顶替后叶子亏空、向兄弟借关键字的连锁调整

【定位】 数据结构 · 第 7 章查找 · B 树的删除调整

【易混对比】

- 向兄弟借一个关键字，和把两个结点合并成一个，是两回事：兄弟手头宽裕时才能借，兄弟手头也紧时只能合并；借完根里换一个字，合并完根里少一个字。
- 前驱和后继也是两回事：前驱是左边子树里最大的关键字，后继是右边子树里最小的关键字；删内部关键字时用哪一个顶上，走法不同，后面的调整也不同。
- 根里换一个字，和根里少一个字，更是两回事：换字是根的关键字被兄弟顶上来的字替换，少字是根的关键字跟着合并沉下去了；两种结果都可能，但不可能出现“根原样不动还多一个字”的结果。

根要么被换一个关键字，要么少一个关键字，没有原样不动还多一个字的走法。

【坑位】

只看根里换了哪个字还不够，叶子亏空后的借或并会反过来再改根。

最容易踩的坑，是以为前驱顶上就完事了。以前驱顶替为例，叶子删掉前驱后只剩一个关键字，少于下限 2 个，必须接着处理：要么向手头宽裕的兄弟借，要么和兄弟合并。借的时候根里被换掉一个字，合并的时候根里少一个字；不管走哪条路，根都不可能保持原样再多出一个字。只看第一步、不看连锁调整，就会错判成“白嫖”的结果。

【套路】

- 第一步，先找前驱或后继顶上：删的是内部关键字，就用左边最大的或右边最小的关键字把位置补上。
- 第二步，再处理叶子亏空：叶子关键字少于下限时，先看兄弟能不能借，能借就借，不能借就合并。
- 第三步，借的时候看根里哪个字被换掉，合并的时候看根里哪个字跟着沉下去，顺着这两步把新根写出来。
- 第四步，把每种走法得到的新根都列出来，再和选项一一对照，凡是没走出来的根就是不可能的。

【类比】

像排班表缺人：班组里有人走了，先从相邻班组调一个人顶上，这是第一步；被调走人的班组因此缺人了，就要么再从别的班组借一个人，要么把两个缺人的班组合并成一个班。借人会换掉排班表上的一个名字，合并会让排班表少一个班组；不管怎样，排班表都不可能原样不动还多出一个班组。

【口诀】 删内先顶替，叶子亏了借或并，根换字或少一字。

第 9 题 · 影响散列平均查找长度的因素（正确答案：D，I、II、III 全影响）

题干与选项：

下列因素中，影响散列（哈希）方法平均查找长度的有（ ）。

I. 装填因子

II. 散列函数

III. 冲突解决策略

- A. 仅 I、II
- B. 仅 I、III
- C. 仅 II、III
- D. I、II、III

入手点： ASL 只由一件事决定：冲突多不多、冲突了找起来费不费劲；三个选项各占一个环节。

题干专有名词从零解释：

- 散列 / 哈希：把关键字算成地址的办法。
- 平均查找长度 ASL：平均一次查找要比较（探测）多少次。
- 装填因子 α：表中元素个数 ÷ 表长，衡量表有多满。
- 散列函数：关键字 → 地址的计算规则。
- 冲突：两个不同关键字算到同一地址。
- 冲突解决策略：撞了之后怎么办。开放定址 / 线性探测：撞了就往后顺延找空位，容易"聚集"（扎堆）；链地址法：同义词挂在一条链表上。

考查知识点： 影响散列 ASL 的因素。

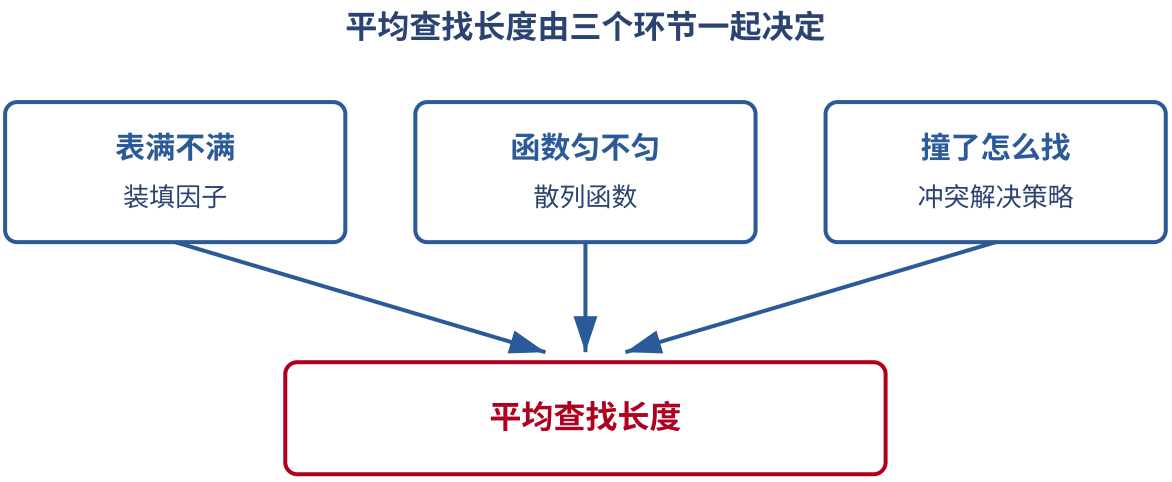
正确选项为什么对： 三个环节全影响：α 越大表越满越容易撞（线性探测成功 ASL ≈ ½(1 + 1/(1−α))，α → 1 时暴涨）；函数越不均匀扎堆越多；策略不同，同样数据下探测代价不同（线性探测有聚集效应，链地址法是链表长度）。

错误选项为什么错＋举例：

- A（只 I、II，漏 III）：策略变了，ASL 就变。举例：同一批数据、同一个散列函数，线性探测和链地址法算出的 ASL 不一样——线性探测里撞了的元素会扎堆，后面来的元素探测次数被前面的"堵车"拉长；链地址法只挂链表，互不干扰。
- B（只 I、III，漏 II）：函数本身就在决定冲突多少。举例：散列函数如果只取关键字的奇偶（比如只看末位），奇数关键字全撞到一起，ASL 直接爆炸；换成均匀分布的函数，冲突立刻大减。
- C（只 II、III，漏 I）：α 是 ASL 公式里的直接变量。举例：表长 100，装 10 个元素（α = 0.1）时几乎不撞；装 95 个（α = 0.95）时几乎每插一个都撞，探测次数成倍涨。

笔记本一句话： 第 9 题＋散列 ASL 三决定：装填因子管"满不满"，散列函数管"匀不匀"，冲突策略管"撞了怎么找"。

【图像】



影响散列平均查找长度的三个环节：表满不满、函数匀不匀、撞了怎么找

【定位】 数据结构 · 第 7 章查找 · 散列查找与平均查找长度

【易混对比】

- 表有多满、函数匀不匀、撞了之后怎么找，是三个不同的环节：表里已经装的元素个数除以表长，这个比值叫装填因子，管的是满不满；把关键字算成地址的规则叫散列函数，管的是匀不匀；撞了之后往后找空位还是挂链表，叫冲突解决策略，管的是撞了怎么找。
- 往后顺延找空位，和把同义词挂在一条链表上，也是两回事：前者容易扎堆，后面来的元素会被前面的堵车拉长查找次数；后者只挂链表，互不干扰，查找次数就是链表长度。
- 表长和装填因子也要分清：表长是表的总容量，装填因子是已装元素个数除以表长；表长不变时装得越多，装填因子越大，越容易撞。

三个因素全都影响，少算任何一个都会选错。

【坑位】

只盯着表满不满会漏掉另外两个环节，三个环节要逐个过。

最容易踩的坑，是觉得平均查找长度只和表满不满有关。函数本身不均匀时，关键字会扎堆撞到一起，查找次数直接变大；冲突解决的办法不同时，同样一批数据算出的查找次数也不一样。三个环节各占一环，漏掉任何一环，都会把本来全影响的题答成只影响其中几项。

【套路】

- 第一步，逐个检查三个环节：先看满不满，再看匀不匀，最后看撞了怎么找。
- 第二步，每个环节都问一句：如果这个环节变了，平均查找次数会不会变？会变就说明它有影响。
- 第三步，把三个环节的结论合起来：只要有一个环节会变，就不能说它没影响；三个环节都会变，答案就是全部影响。

【类比】

像停车场找车位：车位满不满决定了好不好找；车牌号编排得匀不匀决定了车会不会都挤在同一个区；撞了车位之后是往后顺延找空位，还是按区挂牌排队，也决定了要找多久。三个环节任何一个变差，找车位的时间都会变长。

【口诀】 表满函数策略三，少了哪个都不全。

第 10 题 · 二路归并操作的功能（正确答案：A）

题干与选项：

用二路归并排序对数组 M 排序时，"二路归并操作"的功能是（ ）。

- A. 将两个有序表合并为一个新的有序表
- B. 将待排序表划分为大致相等的两部分
- C. 将待排序表划分为 n 个单元素子表
- D. 将待排序表划分为两部分，使一部分元素的值均小于另一部分元素的值

入手点： "归并操作"单指一个动作：把两个有序表合成一个有序表；划分、拆分、快排分区都是别的动作。

题干专有名词从零解释：

- 二路归并排序：把表对半拆、递归排好、再合并的排序；"二路"就是"两路输入"。
- 二路归并操作：特指合并那一步。
- 有序表：元素已按大小排好的表。
- 划分：把表对半切开（归并排序的第 ① 步）。
- 枢轴划分：快排里选一个基准，把表分成"小的一边、大的一边"。

考查知识点： 归并排序各步骤的功能辨析。

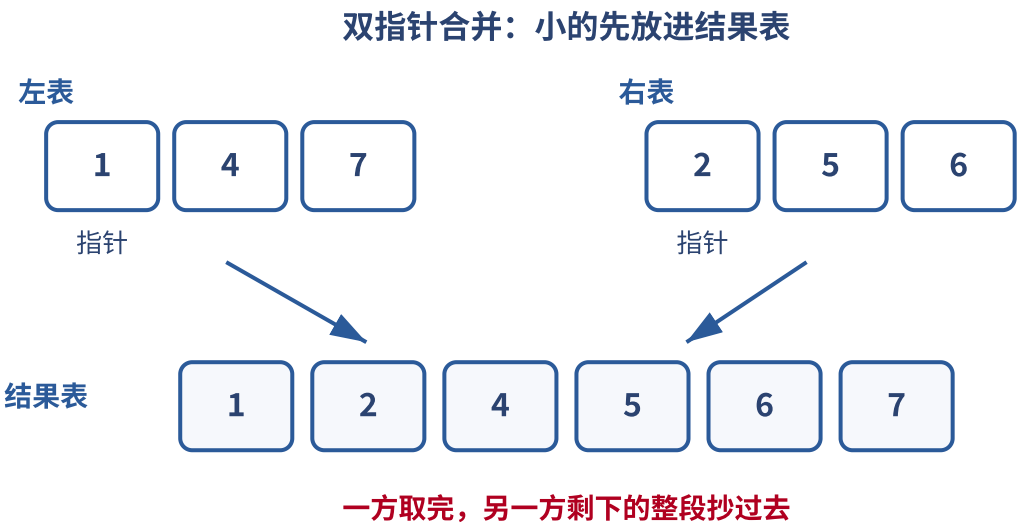
正确选项为什么对： 两个指针分别指着两个有序表的头，谁小先取谁放进结果表，一方取完把另一方剩下的整段抄过去。举例：左表 [1, 4, 7]、右表 [2, 5, 6]：取 1、2、4、5、6、7 → [1, 2, 4, 5, 6, 7]，一趟比较次数至多是两表长度之和减一。

错误选项为什么错＋举例：

- B（划分成大致相等的两部分）：那是第 ① 步划分，不是归并。举例：[7, 2, 5, 1] 先切成 [7, 2] 和 [5, 1]，这一步只切不开合并，叫划分。
- C（划成 n 个单元素部分）：那是递归拆分的终点状态。举例：[7, 2, 5, 1] 一路拆到 [7][2][5][1]，每个单元素天然有序，但这只是拆完的状态，还没开始合并。
- D（一部分值全小于另一部分）：那是快排枢轴划分的效果。举例：快排选 5 作枢轴，一趟后变成 [2, 1]、5、[7, 6]，左边全小于 5、右边全大于 5——归并的两半之间没有任何大小关系，左半最大值完全可以大于右半最小值。

笔记本一句话： 第 10 题＋归并操作只做一件事：两个有序表合成一个有序表；划分是它的前一步，枢轴分区是快排的活。

【图像】



两个有序表用双指针合并为一个有序表的操作示意图

【定位】 数据结构 · 第 8 章排序 · 二路归并排序的归并操作

【易混对比】

- 合并和切分是两回事：合并是把两个已经排好的表合成一个排好的表，切分是把一个表切成两半；归并操作单指合并那一步，不是切分那一步。
- 归并的切分，和快排里选一个基准把表分成小的一边和大的一边，也是两回事：归并切分只管对半切，两半之间没有任何大小关系；快排分区是按大小分，左边全小于基准、右边全大于基准。
- 拆到只剩单元素的状态，和合并完成的状态，更是两回事：单元素天然有序，但那只是拆完还没开始合的状态；归并操作做的是从拆完往回合成的活。

归并操作只做一件事：把两个有序表合成一个新的有序表。

【坑位】

看到“二路”别先想到切分，题目问的是归并操作，只做合并这一件事。

最容易踩的坑，是把归并排序的整个过程都当成归并操作。归并排序确实先切分、再合并，但题目问的“二路归并操作”单指合并那一步。看到“划分为两部分”就选它，等于把前一步切分当成了合并；看到“一部分值全小于另一部分”就选它，等于把快排分区的效果安到了归并头上。

【套路】

- 第一步，双指针比表头：两个指针分别指着两个有序表的开头，每次比较两个表头。
- 第二步，小的先放进结果表，放完后那个指针往后移一位，再比下一对表头。
- 第三步，一方取完就抄另一方剩下的：剩下的那一段已经排好了，不用再比，直接整段抄过去。
- 第四步，数比较次数时记住：一趟比较次数至多是两表长度之和减一，因为最后剩下的那一段不用再比。

【类比】

像把两摞已经按大小排好的扑克牌合成一摞：每次看两摞最上面那张牌，小的那张先拿下来放进新的一摞，拿完再看下一张；其中一摞拿空了，另一摞剩下的牌已经排好了，直接整摞摞上去，不用再一张一张比。

【口诀】 两路有序比大小，小的先行合成串。

第 11 题 · 选直接插入排序而不选快速排序的理由（正确答案：D，I、II、III、IV 全部成立）

题干与选项：

对数据进行排序时，若采用直接插入排序而不采用快速排序，则可能的原因是（ ）。

I. 大部分元素已有序

II. 待排序元素数量很少

III. 要求空间复杂度为 O(1)

IV. 要求排序算法是稳定的

- A. 仅 I、II
- B. 仅 III、IV
- C. 仅 I、II、IV
- D. I、II、III、IV

入手点： 四条逐个对：把两个算法的时间、空间、稳定性底细摆齐再判。

题干专有名词从零解释：

- 直接插入排序：每次把一个元素插进前面已排好的部分。
- 快速排序：选枢轴、分区、递归。
- 稳定性：相等元素的相对次序排完后不变。
- 辅助空间：排序过程中额外用的存储。
- 基本有序：大部分元素已经排好，只剩少量错位。
- 最坏情况：对某种输入退化到最慢（快排选首元素作枢轴时，基本有序的数据会让每次划分极不均衡）。

考查知识点： 排序算法多维对比（时间 / 空间 / 稳定性 / 适用场景）。

正确选项为什么对： I 成立：近有序时直插每插一个几乎不用挪动，接近 O(n)，而快排（选首元素作枢轴）每次划分极不均衡，退化到 $O(n^2)$。II 成立：元素很少时，$O(n^2)$ 和 $O(n \log n)$ 的实际差距小到可忽略，直插实现简单、无递归开销。III 成立：直插只需 1 个临时单元，空间严格 O(1)；快排递归栈平均 $O(\log n)$、最坏 O(n)。IV 成立：直插遇到相等元素放后面，次序不变，稳定；快排远距离交换打乱相等元素次序，不稳定。

错误选项为什么错＋举例：

- A（只 I、II，漏 III、IV）：举例：题目要求空间"必须是 O(1)"时，快排递归栈平均 $O(\log n)$ 就过不了关；要求"相等元素次序不变"时，快排一趟交换就可能把两个相等的 5 换了位置。
- B（只 III、IV，漏 I、II）：举例：1000 个元素基本有序，直插约 1000 步搞定，快排退化到约 100 万次比较——I 是实打实的优势；只有 5 个元素时，直插几步就完，快排还要递归调用、划分子表，纯属大材小用。
- C（I、II、IV，漏 III）：举例：快排最坏情况（已有序、选首元素枢轴）的递归深度 = n，栈空间 O(n)，根本不是 O(1)——"平均 $O(\log n)$"不能当成"就是 O(1)"。

笔记本一句话： 第 11 题＋直插赢快排靠四点：近有序近 O(n)、小数据开销小、空间严 O(1)、稳定；快排软肋是有序退化、要栈空间、不稳定。

【图像】

直接插入排序 vs 快速排序：四维对照

维度	直接插入排序	快速排序
时间	近有序快，小规模开销低	平均快，近有序会退化
空间	**O(1)，无额外空间 ★**	递归栈，额外空间
稳定性	**稳定 ★**	不稳定，远距离交换
适用场景	**近有序 / 小规模 / 要稳定 ★**	大规模、无序数据

直插赢的四点：近有序、小规模、零额外、要稳定

四点全占齐，才选直插；只看平均速度会漏掉后三点

直接插入排序 vs 快速排序四维对照图

【定位】 数据结构 · 第 8 章排序 · 直接插入排序与快速排序的多维对比。

【易混对比】

- 直接插入排序和快速排序：前者像一张一张往已排好的牌里插牌，每次只动邻近的牌；后者像先挑一张基准牌把牌堆分成两堆，再对两堆重复分堆。插牌法步骤简单但牌多时挪动多，分堆法牌多时快，但要递归、要额外记分堆的断点。
- 稳定和不稳定：稳定是指两个数值相等的元素，排完后先后次序不变；不稳定是指相等元素可能被远距离交换打乱次序。本题里稳定性是一条独立要求，不是速度的附带品。
- 平均时间和最坏时间：平均时间是各类输入的大致水平，最坏时间是碰上最不利输入时的水平。快速排序平均很快，但碰上已有序又选首元素作基准时，会退化到最坏水平，不能只拿平均时间下结论。

【坑位】 本题最核心的陷阱，是只用“快速排序平均更快”一句话否定全部四个理由。快速排序平均确实快，但题目问的是“不用快速排序、改用直接插入排序可能的原因”，只要有一条理由成立就够选，而这里四条都成立：近有序时快速排序会退化，数据很少时递归开销反而划不来，空间要求卡在 $O(1)$ 时递归栈过不了关，稳定性要求下远距离交换会打乱相等元素的次序。只盯着平均速度，就会漏掉后三条。

【套路】 碰到“为什么选 A 算法而不选 B 算法”这类题，按四格逐一核对：第一格看输入特点，是不是近有序或规模很小；第二格看时间，是平均还是最坏说了算；第三格看空间，有没有 $O(1)$ 这样的硬上限；第四格看稳定性，有没有相等元素次序不变的要求。四格都对过一遍，再数成立的条数，不要凭第一印象只对一格。

【类比】 先想整理一手扑克牌。直接插入排序就像左手已经拿着一叠排好的牌，右手每摸一张，就插进左手牌里合适的位置，牌基本有序时几乎不用挪动。快速排序就像先挑一张牌作分界，把比它小的放左边、大的放右边，再对左右两堆重复分，牌多时分得快，但牌只有几张时，光是分堆、记断点就比直接插牌还麻烦；而且分堆时远距离换牌，会把两张点数相同的牌的先后次序换掉。

【口诀】 近有序、小规模、零额外、要稳定，直插四点全占齐。

第 12 题 · CPI 与 CPU 执行时间（正确答案：A，2.8，28μs）

题干与选项：

某 CPU 的主频为 1 GHz，共执行 10000 条指令，其中 80% 的指令平均需要 1 个时钟周期，20% 的指令平均需要 10 个时钟周期。该程序的平均 CPI 与 CPU 执行时间分别是（ ）。

- A. 2.8，28 μs
- B. 28，28 μs
- C. 2.8，28 ms
- D. 28，28 ms

入手点： 两步：先加权平均算 CPI，再用"条数 × CPI ÷ 主频"算时间，最后默念一遍单位。

题干专有名词从零解释：

- CPI：平均每条指令花几个时钟周期。
- 主频：每秒多少个时钟周期，1 GHz = 10^9 个 / 秒。
- 时钟周期：一个周期 = 1 / 主频 秒；主频 1 GHz 时一个周期 = 1 ns。
- 加权平均：按占比算平均（几类指令混在一起，各按比例加权）。
- μs（微秒）：1 s = 10^3 ms = 10^6 μs = 10^9 ns，每步差 1000，ms 和 μs 最常翻车。

考查知识点： CPI 加权平均与 CPU 执行时间计算、单位换算。

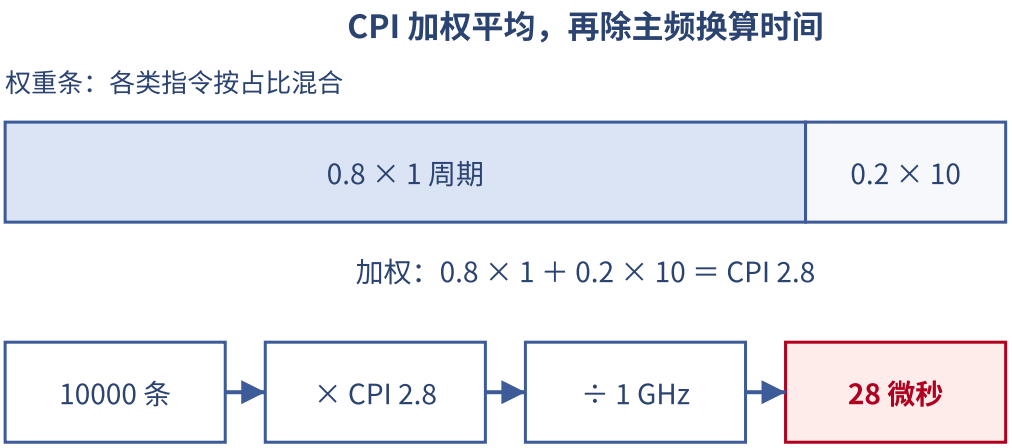
正确选项为什么对： CPI = 0.8 × 1 + 0.2 × 10 = 0.8 + 2.0 = 2.8（加权平均是按比例混合 1 和 10，混合结果不可能比最大的 10 大，也不可能比最小的 1 小，所以必然落在 1 和 10 之间）。总周期数 = 10000 × 2.8 = 28000 个周期；主频 1 GHz → 一个周期 1 ns；时间 = 28000 × 1 ns = 28000 ns = 28 μs（28000 ns ÷ 1000 = 28 μs）。

错误选项为什么错＋举例：

- B（CPI 28，时间 28μs）：CPI 小数点错位。举例：0.8 × 1 + 0.2 × 10 = 2.8，不可能算出 28——加权平均一定夹在最小值 1 和最大值 10 之间，28 超出范围一眼就该怀疑。
- C（CPI 2.8，时间 28ms）：单位差 1000 倍。举例：28000 ns = 28 μs，而 28 ms = 28000000 ns，是前者的 1000 倍——ms 和 μs 之间隔着 ×1000，最后一格最容易翻车。
- D（28，28ms）：两项全错，等于 CPI 小数点错位＋单位错位同时发生。

笔记本一句话： 第 12 题＋CPI 先加权平均（结果必夹在最小最大值之间），时间 = 条数 × CPI ÷ 主频，选答案前默念一遍单位。

【图像】



CPI 加权平均与换算链示意图

【定位】 计算机组成原理 · 第 5 章中央处理器 · 平均 CPI 与 CPU 执行时间的计算。

【易混对比】

- CPI 和主频：CPI 是“平均每条指令花几个时钟周期”，单位是周期数除以条数；主频是“每秒有多少个时钟周期”，单位是赫兹。一个讲每条指令的代价，一个讲时钟跑得有多快，两个量相乘除的方向不要颠倒。
- CPI 和 IPC：IPC 是“每个时钟周期平均完成几条指令”，正好是 CPI 的倒数。题目给的是每类指令的周期数，算的是 CPI，不要把它当成 IPC 去倒着算。
- 毫秒、微秒、纳秒：1 秒 $=10^3$ 毫秒 $=10^6$ 微秒 $=10^9$ 纳秒，相邻两档之间差 1000 倍。本题主频 1 GHz 对应一个周期 1 纳秒，从纳秒到微秒要除以 1000，到毫秒还要再除以 1000。

【坑位】 本题最核心的陷阱，是 CPI 算对了、最后一步单位却滑了一档。28000 纳秒换算时，有人凭感觉写成 28 毫秒，实际 28000 纳秒除以 1000 才等于 28 微秒，再除以 1000 才是 28 毫秒。选答案前把单位链从头默一遍，比重新算一遍 CPI 更能防错。

【套路】 CPI 类题目固定走两步：第一步按占比加权，把各类指令的周期数按比例混合，公式是 $CPI = 0.8 \times 1 + 0.2 \times 10 = 2.8$；第二步算时间，公式是时间 $=$ 指令条数 $\times CPI \div$ 主频，本题是 $10000 \times 2.8 \div 10^9 = 28$ 微秒。算完加一步自查：加权结果一定落在最小周期数和最大周期数之间，超出这个范围就是算错了。

【类比】 先想一条生产线上有两种活：八成是快活，每件花 1 分钟；两成是慢活，每件花 10 分钟。平均每件花的时间不是 1 分钟，也不是 10 分钟，而是按八二比例混合出来的 2.8 分钟。知道平均每件的时间和总件数，再除以生产线每分钟的节拍速度，就是总用时。主频就是节拍速度，CPI 就是平均每件的时间。

【口诀】 先加权、后除频，纳秒微秒差千倍。

第 13 题 · 32 位补码整数范围（正确答案：B，$-2^{31} \sim 2^{31}-1$）

题干与选项：

32 位补码所能表示的整数范围是（ ）。

- A. $-2^{32} \sim 2^{31}-1$
- B. $-2^{31} \sim 2^{31}-1$
- C. $-2^{32} \sim 2^{32}-1$
- D. $-2^{31} \sim 2^{32}-1$

入手点： 补码范围直接套公式，关键是记住负数那边多一个。

题干专有名词从零解释：

- 补码：用最高位作符号位（0 正 1 负）、正数原样、负数取反加 1 的编码。
- 原码：符号位＋数值本身；反码：负数符号位不变、数值位取反。
- n 位：编码总位数，本题 n = 32。
- 整数范围：能表示的最小值到最大值。

考查知识点： n 位补码的表示范围。

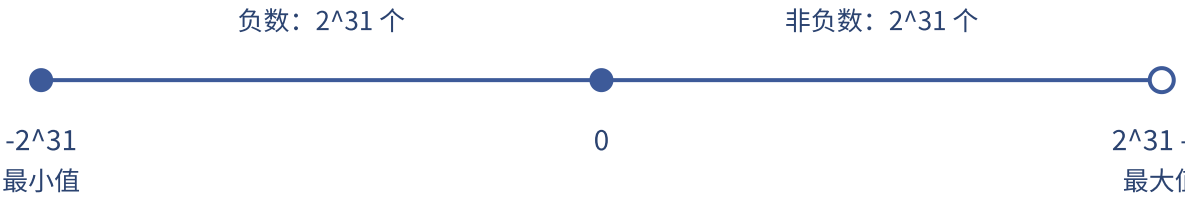
正确选项为什么对： n 位补码范围是 [−2^(n−1), 2^(n−1) − 1]，n = 32 → $[-2^{31}, 2^{31}-1]$。为什么负数多一个：32 位共 2^{32} 个编码，正数占 0 ~ $2^{31}-1$（2^{31} 个），剩下的 2^{31} 个编码全给负数，-2^{31} 没有对应的正数镜像（因为 $+2^{31}$ 需要 33 位）。

错误选项为什么错＋举例：

- A（下限 -2^{32}）：举例：32 位总共只有 2^{32} 个编码，从 -2^{32} 到 $2^{31}-1$ 需要 $2^{32}+2^{31}$ 个编码，根本排不下——下限不可能到 -2^{32}。
- C（上下限指数都写大了一位）：32 位补码是 $[-2^{31}, 2^{31}-1]$，C 写成 $[-2^{32}, 2^{32}-1]$，两边各超了一位。举例：下限 -2^{32} 需要 33 位才能表示，32 位根本排不下；上限 $2^{32}-1$ 是 32 个 1，最高位为 1，在补码里是 −1，不是正数。
- D（上限 $2^{32}-1$）：举例：$2^{32}-1$ 的二进制是 32 个 1，最高位是 1，在补码里这是负数 −1，不是正数——正数最高位必须是 0，最大只能是 0111…1 = $2^{31}-1$。

笔记本一句话： 第 13 题＋n 位补码范围 [−2^(n−1), 2^(n−1) − 1]，负数那边多一个。

【图像】



32 位补码数轴：负数一边比正数多一个数

【定位】 计算机组成原理 · 第 2 章数据的表示和运算 · 补码的表示范围。

【易混对比】

- 原码和补码：原码是“符号位加数值本身”，正数和负数对称，0 有正负两种写法；补码是“正数原样、负数取反加 1”，0 只有一种写法，负数一边因此多出一个数。题目问的是补码，不要拿原码的对称范围去套。
- 补码和无符号数：无符号数 32 位全用来表示数值，范围是 0 到 $2^{32}-1$；补码 32 位要分一位作符号，范围是 -2^{31} 到 $2^{31}-1$。看到上限 $2^{32}-1$，先问一句这是不是无符号数的范围。
- 位数和指数：32 位补码范围里的指数是 31，不是 32。指数 31 来自“总位数减去符号位占的一位”，把 32 直接抄成指数，就会把上下限都写大一位。

【坑位】 本题最核心的陷阱，是默认正负范围对称，觉得最小值既然是 -2^{31}，最大值就该是 2^{31}。补码里正数最大只能写成 $0111\ldots1$，也就是 $2^{31}-1$，因为最高位是 0 才能表示正数；写成 2^{31} 需要第 32 位是 1，那已经是负数的地盘了。对称的直觉在这里不成立，负数一边就是多一个。

【套路】 n 位补码范围固定套公式：最小值 -2^{n-1}，最大值 $2^{n-1}-1$。本题 $n=32$，代入得到 -2^{31} 到 $2^{31}-1$。套完加一步数数自查：正数加 0 一共 2^{31} 个编码，负数一共 2^{31} 个编码，两边加起来正好是 32 位的全部 2^{32} 个编码，数对上了范围才可信。

【类比】 先想一排 2^{32} 个带编号的储物格，编号从 0 开始往两边分：一半格子分给非负数，从 0 分到 $2^{31}-1$；另一半格子分给负数，从 -1 一直分到 -2^{31}。非负数那边因为 0 占了一个格子，正数只能分到 $2^{31}-1$；负数那边没有 0 占位，就能多分到 -2^{31} 这一格。

【口诀】 补码负多一，正顶要减一，指数是位减一。

第 14 题 · IEEE 754 单精度表示（正确答案：A，BEE0 0000H）

题干与选项：

将 −0.4375 表示为 IEEE 754 单精度浮点数，其十六进制表示是（ ）。

- A. BEE0 0000H
- B. BF60 0000H
- C. BF70 0000H
- D. C0E0 0000H

入手点： 浮点拼装三步：化二进制规格化 → 阶码 +127 → 尾数去掉领头的 1 补齐 23 位。

题干专有名词从零解释：

- IEEE 754 单精度：32 位浮点标准 = 1 位符号＋8 位阶码＋23 位尾数。
- 符号位：0 正 1 负。
- 阶码：真指数＋127 偏置后的值（偏置是为了让阶码不用带符号位）。
- 尾数：规格化后小数点后面的部分，整数位的 1 被"藏起来"不存。
- 规格化：把二进制小数写成 1.xxx × 2^指数 的形式。
- 十六进制表示：每 4 位二进制对应 1 位十六进制。

考查知识点： IEEE 754 单精度编码过程。

正确选项为什么对： −0.4375：小数转二进制用"乘 2 取整"：0.4375×2 = 0.875 取 0；0.875×2 = 1.75 取 1；0.75×2 = 1.5 取 1；0.5×2 = 1.0 取 1，得 0.0111_2（验算：0.25 + 0.125 + 0.0625 = 0.4375）；规格化：0.0111_2 = 1.11 × 2^−2（小数点右移 2 位）。符号位 = 1（负数）；阶码 = −2 + 127 = 125；125 换二进制：125 = 64 + 32 + 16 + 8 + 4 + 1 = 01111101_2；尾数 = "11" 后面补 0 到 23 位 = 11000000000000000000000。拼起来：1 01111101 11000000000000000000000，按 4 位分组：1011 1110 1110 0000 0000 0000 0000 0000 = BEE0 0000H。

错误选项为什么错＋举例：

- B（BF60 0000H，对应 −0.875）：规格化时小数点少移一位，写成 1.11 × 2^−1 = 0.875，指数错一位。举例：0.4375 的二进制是 0.0111，小数点要移 2 位才到 1.11，移 1 位得到的是 0.111 = 0.875，那是另一个数。
- C（BF70 0000H，对应 −0.9375）：阶码和尾数都错位拼出来的。举例：−0.9375 = −1.111 × 2^−1，尾数"111"和阶码 126 的组合——既不是 −0.4375 的规格化结果，也不是它的尾数。
- D（C0E0 0000H，对应 −7）：指数方向弄反。举例：C0E0 的阶码段是 10000001 = 129，真指数 = 129 − 127 = +2，1.11 × 2^2 = 7——小数点往右移成了往左移的反方向。

笔记本一句话： 第 14 题＋浮点拼装三步：规格化 → 阶码 +127 → 尾数去掉领头的 1 补齐 23 位；阶码忘 +127、尾数多写个 1 是最高频坑。

【图像】

-0.4375 的 IEEE 754 单精度拼装

符号位	阶码 8 位	尾数 23 位
1（负）	01111101 = 125	1100…0（领头的 1 不存）

1 + 01111101 + 110…0 = BEE0 0000H

真指数 -2 + 127 = 125

IEEE 754 单精度位段图：符号位、阶码、尾数拼出 BEE0 0000H

【定位】 计算机组成原理 · 第 2 章数据的表示和运算 · IEEE 754 单精度浮点数编码。

【易混对比】

- 真指数和阶码：真指数是规格化后 2 的真正次方，本题是 -2；阶码是真指数加上偏置 127 之后存进机器的值，本题是 125。存的是阶码，算的是真指数，两个数不要直接互换。
- 尾数和规格化后的小数部分：规格化写成 1.11×2^{-2}，小数点后是 11；存进机器的尾数是把领头的 1 藏起来、只存 11 再补 0 到 23 位。领头的 1 是约定的、不用存的，存进去反而多了一位。
- 单精度和双精度：单精度是 32 位，1 位符号加 8 位阶码加 23 位尾数，偏置 127；双精度是 64 位，1 位符号加 11 位阶码加 52 位尾数，偏置 1023。本题是单精度，偏置和位宽都要按单精度的来。

【坑位】 本题最核心的陷阱，是规格化时小数点移位方向或位数错一位。0.4375 的二进制是 0.0111，小数点要向右移 2 位才变成 1.11，指数是 -2；只移 1 位就得到 0.111，那是 0.875，不是 0.4375。移位错一位，阶码和尾数跟着全错，答案就滑到另一选项上。

【套路】 浮点数拼装固定走三步：第一步化二进制并规格化，把小数写成 $1.xxx \times 2^{\text{指数}}$ 的样子，本题是 1.11×2^{-2}；第二步算阶码，阶码 = 真指数 $+127$，本题是 $-2+127=125$，换成 8 位二进制是 01111101；第三步拼尾数，去掉领头的 1，把剩下的 11 后面补 0 到 23 位，再把符号位 1、阶码、尾数按顺序拼起来，每 4 位一组换成十六进制。

【类比】 先想科学记数法写大数：4375 写成 4.375×10^3，领头的 4 不能丢，小数点移几位、指数就是几。IEEE 754 单精度就是二进制版的科学记数法：先把数挪成 $1.xxx$ 开头，挪的位数记成指数；为了让指数不用带负号，存的时候统一加 127；领头的 1 大家都知道它在那儿，就不用再存一遍，省下来的一位留给尾数多记一位精度。

【口诀】 规格化、加一二七、去领头补零拼。

第 15 题 · 虚存地址变换（正确答案：C，01 8840H）

题干与选项：

某分页虚拟存储系统中，页大小为 4 KB。访问虚拟地址 00082840H 时，页表给出：虚页号 82 对应页框号 024H（存在位 0），虚页号 129 对应页框号 180H（存在位 1），虚页号 130 对应页框号 018H（存在位 1）。该虚拟地址转换后得到的主存地址是（ ）。

- A. 02 4840H
- B. 18 0840H
- C. 01 8840H
- D. 产生缺页异常

入手点： 先按页大小切地址（高位页号、低位偏移），页号是十六进制先换算，再查表拼回。

题干专有名词从零解释：

- 分页虚存：把虚拟地址空间切成等大的页，主存里对应等大的块（页框）。
- 页大小：本题 4 KB = 2^{12} 字节，所以偏移占低 12 位。
- 虚拟地址：程序看到的地址；虚页号：虚拟地址的高位部分，决定是哪一页；页内偏移：低位部分，页内的位置。
- 页表：虚页号 → 页框号的对照表。
- 页框号：主存里那块的编号。
- 存在位：1 表示该页已在内存，0 表示不在，会触发缺页。
- 主存地址 / 物理地址：页框号接偏移拼成的真实地址。

考查知识点： 分页地址变换（切地址、查页表、拼地址）。

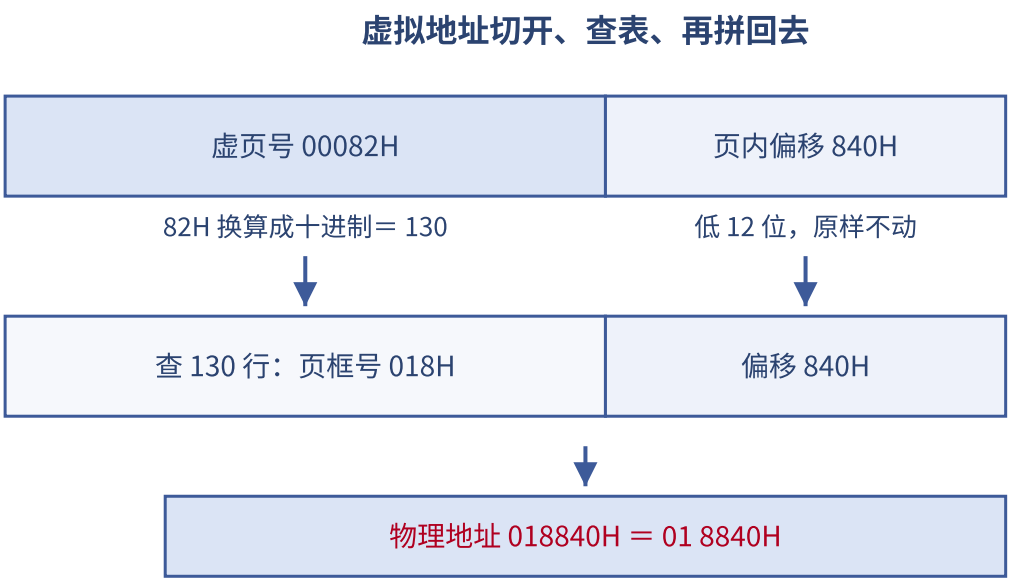
正确选项为什么对： 页大小 4 KB = 2^{12} → 低 12 位是偏移：00082840H 的低 12 位 = 840H；高位是虚页号 = 00082H。关键一步：82H 是十六进制 = 8 × 16 + 2 = 130（十进制），查表要查 130 那一行：页框 018H、存在位 1 → 页框号 018H（12 位）放在高位、偏移 840H（12 位）放在低位，拼成 24 位物理地址：018840H，即 01 8840H。

错误选项为什么错＋举例：

- A（02 4840H）：用了 82 行的页框 024H——查错行。举例：82H 如果当成十进制 82 去查，查到的是"虚页号 82"那行（页框 024H），但题干地址里的 82 是十六进制，真正的行号是 130；何况 82 那行存在位是 0，就算查它也该报缺页。
- B（18 0840H）：用了 129 行的页框 180H——看串了一行。举例：129 和 130 在表里是上下相邻的两行，180H 和 018H 又长得像，眼睛一滑就错行——这是本题的经典陷阱。
- D（缺页异常）：把虚页号当成十进制 82、查到存在位为 0 的那行才得出的结论。举例：正确行号是 130，存在位是 1，页就在内存里，根本不会缺页。

笔记本一句话： 第 15 题＋先按页大小切出页号和偏移，页号是十六进制先换算，查对行后页框号原样拼回偏移。

【图像】



虚拟地址切分与拼接图：虚页号换算后查表拼出物理地址

【定位】 计算机组成原理 · 第 3 章存储器层次结构 · 分页虚拟存储的地址变换。

【易混对比】

- 虚页号和页框号：虚页号是虚拟地址高位部分给出的“程序里的页编号”；页框号是查页表后得到的“主存里的块编号”。前者是查表的钥匙，后者是查表的结果，查表就是用前者换后者。
- 十六进制页号和十进制页号：地址里的 82 后面跟的是十六进制位，$82H$ 换算成十进制是 130；直接把 82 当成十进制去查表，查到的是另一行。进制是查表前必须先换算的一步，不是可省的细节。
- 页内偏移和页号：页内偏移是地址低位部分，在页里的位置，查表前后原样不动；页号是地址高位部分，查表后要换成页框号。拼地址时是“新页框号接旧偏移”，不要把偏移也换掉。

【坑位】 本题最核心的陷阱，是把地址里的 82 当成十进制 82 去查表。地址是十六进制写的，$82H = 8 \times 16 + 2 = 130$，正确行是虚页号 130 那行，页框号 $018H$，存在位是 1；按十进制 82 查，查到的是页框号 $024H$、存在位 0 的那行，既查错行，又会误判成缺页。进制换算这一步错，后面全错。

【套路】 分页地址变换固定走三步：第一步按页大小切地址，页大小 4 KB $= 2^{12}$ 字节，所以低 12 位是偏移，高位是虚页号，本题切出虚页号 $00082H$ 和偏移 $840H$；第二步换算进制再查表，$82H$ 换算成十进制 130，查 130 行得到页框号 $018H$，并确认存在位是 1；第三步拼地址，页框号放高位、偏移放低位，拼成 $018840H$，也就是 $018840H$。

【类比】 先想图书馆找书：虚拟地址像“第 82 排书架第 840 本书”这样的馆内编号，但排号是十六进制写的，$82H$ 实际是第 130 排。先按排和本切开编号，再拿排号去对照表查这排书实际放在仓库哪个架子（页框号），最后把仓库架子号和本内位置拼起来，才是真正能走到的位置。排号进制没换算，就会跑错排。

【口诀】 先切后换再查拼，十六进制别当十进制。

第 16 题 · Cache 比较器个数与位数（正确答案：A，8 个，20 位）

题干与选项：

某 Cache 的数据区容量为 32 KB，主存块大小为 64 B，采用 8 路组相联映射。访存时所需的比较器的个数和位数分别是（ ）。

- A. 8 个，20 位
- B. 8 个，23 位
- C. 64 个，20 位
- D. 64 个，23 位

入手点： 先数行数 → 组数 → 切地址三段 → 比较器个数 = 路数。

题干专有名词从零解释：

- Cache：CPU 和主存之间的高速小容量存储。
- 数据区：Cache 里真正存数据的部分，本题 32 KB。
- 主存块：主存按固定大小分块，本题 64 B，Cache 一行装一个块。
- 行：Cache 的一行，装一个主存块。
- 组相联：Cache 的行先分组，8 路 = 每组 8 行，主存块只能进指定组内的任意一行。
- 组号：地址里指出是哪一组；标记：地址里剩下的高位，用来在组内比对是哪一块；块内偏移：地址里指出块内的字节位置。
- 比较器：同时比对标记的硬件，几行就几个。

考查知识点： 组相联 Cache 的地址划分与比较器。

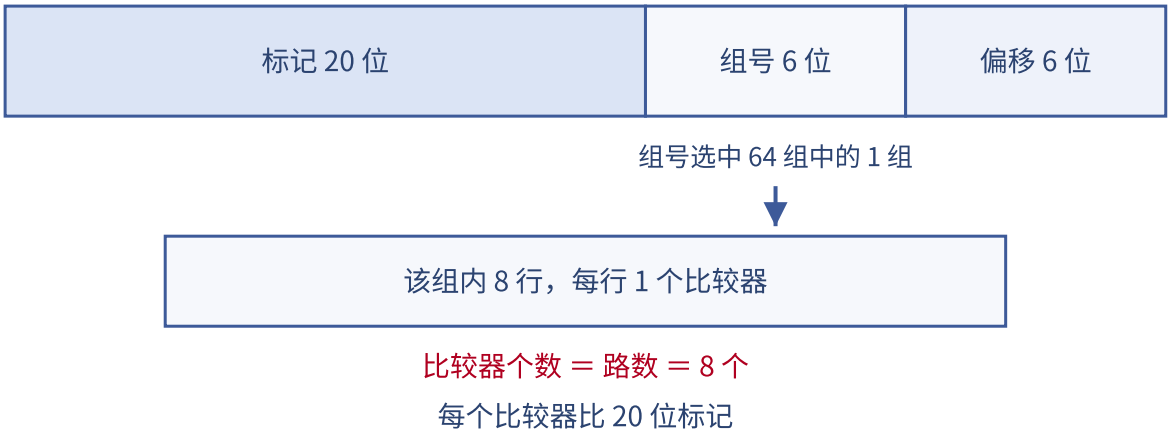
正确选项为什么对： 行数 = 32 KB ÷ 64 B = 512 行；组数 = 512 ÷ 8 = 64 组。地址切分（按 32 位地址）：块内偏移：64 B 的块内有 64 个字节位置，$2^6 = 64$，所以区分 64 种位置需要 6 位二进制，即 $\log_2 64 = 6$ 位；组号：64 组，$2^6 = 64$，同样需要 6 位，即 $\log_2 64 = 6$ 位；标记 = 32 − 6 − 6 = 20 位。访存时选中 1 组，把地址里的标记和组内 8 行的标记同时比对 → 比较器 8 个，每个比 20 位。

错误选项为什么错＋举例：

- B（8 个、23 位）：个数对，位数错。举例：23 位是把组号误算成 3 位（拿"8 路"直接开 $\log_2$：$\log_2 8 = 3$）凑出来的——组号位数由组数（64 组）决定，是 $\log_2 64 = 6$，不是路数。
- C（64 个、20 位）：位数对，个数错。举例：64 是组数——64 组是"选 1 组"，不是"同时比 64 个"；真正同时比的是组内的 8 行，所以是 8 个比较器。
- D（个数、位数都错）：两处都按错的口径算，等于把组数和路数、组号和标记全混了。

笔记本一句话： 第 16 题＋比较器个数 = 路数，位数 = 标记位数 = 总位数 − 组号 − 块内偏移。

【图像】



Cache 地址三段划分与比较器图：标记、组号、块内偏移

【定位】 计算机组成原理 · 第 3 章存储器层次结构 · 组相联 Cache 的地址划分与比较器。

【易混对比】

- 组数和路数：组数是 Cache 一共分了多少组，本题是 64 组；路数是每组里有几行，本题是 8 路。组数决定组号占几位，路数决定同时比对几行，两个数各管一件事。
- 标记和组号：组号是地址里用来“选中哪一组”的部分；标记是地址里剩下的高位，用来在组内“比对是哪一块”。组号先选组，标记再在组内比对，分工不要颠倒。
- 比较器个数和组数：比较器是同时工作的比对硬件，有几行同时比对就有几个比较器；组数是“轮流选中一组”，不是“同时比对的个数”。本题同时比对的是组内 8 行，所以是 8 个比较器，不是 64 个。

【坑位】 本题最核心的陷阱，是组号位数按路数算。有人看到 8 路，直接算 $\log_2 8 = 3$ 位当组号，剩下标记就成了 23 位。组号位数由组数决定，组数 64 才对应 $\log_2 64 = 6$ 位；路数 8 只决定比较器个数。把“每组几行”当成“有几组”，位数就错了 3 位。

【套路】 组相联 Cache 题目固定走三步：第一步算行数和组数，行数 = 数据区容量 ÷ 块大小 = 32 KB ÷64 B = 512 行，组数 = 行数 ÷ 路数 = 512 ÷ 8 = 64 组；第二步切地址，块内偏移 = $\log_2 64 = 6$ 位，组号 = $\log_2 64 = 6$ 位，标记 = $32 - 6 - 6 = 20$ 位；第三步定比较器，个数 = 路数 = 8 个，每个比标记的 20 位。

【类比】 先想一排带抽屉的柜子：柜子分了 64 组，每组有 8 个抽屉。找东西时，先按组号找到其中一组，再把要找的东西的名字（标记）同时和这组 8 个抽屉里的标签比对，需要 8 个人同时比对，不是 64 个人。组号是“去第几组”，路数是“这组里同时比几个抽屉”。

【口诀】 路数定个数，组数定组号，余下全是标记。

第 17 题 · DRAM 内存条（正确答案：C，不正确）

题干与选项：

某内存条由 8 个 8192×8192×8 位的 DRAM 芯片组成，总线宽度为 64 位，支持突发传送，每个芯片有一个行缓冲。下列关于该内存条的叙述中，不正确的是（）。

- A. 该内存条的总容量为 512 MB
- B. 该内存条采用多模块交叉编址
- C. 该内存条的地址引脚数为 26
- D. 该内存条每个芯片的行缓冲为 8192×8 位

入手点： 容量按"单元数 × 位宽"、行缓冲等于一整行、地址引脚行列复用只算一份；找"不正确"的先逐项验。

题干专有名词从零解释：

- DRAM：动态随机存储器，靠电容存数据，要定期刷新。
- 芯片规格 8192×8192×8 位：每片有 8192×8192 = 2^{26} 个地址单元，每个单元 8 位；8 片并排，每个地址一共读出 8 × 8 = 64 位。
- 总线 64 位：一次传 64 位数据，正好 8 片各出 8 位拼成。
- 突发传送：一次给地址、连续传一批数据的传送方式。
- 行缓冲：DRAM 里暂存一整行的缓冲，访问某一行时整行先被搬进缓冲。
- 行列地址复用：行地址和列地址分两次、共用同一组引脚送入，省引脚。
- 多模块交叉编址：多个模块轮流编址，配合突发传送出数据。

考查知识点： DRAM 组织：容量、行缓冲、地址引脚。

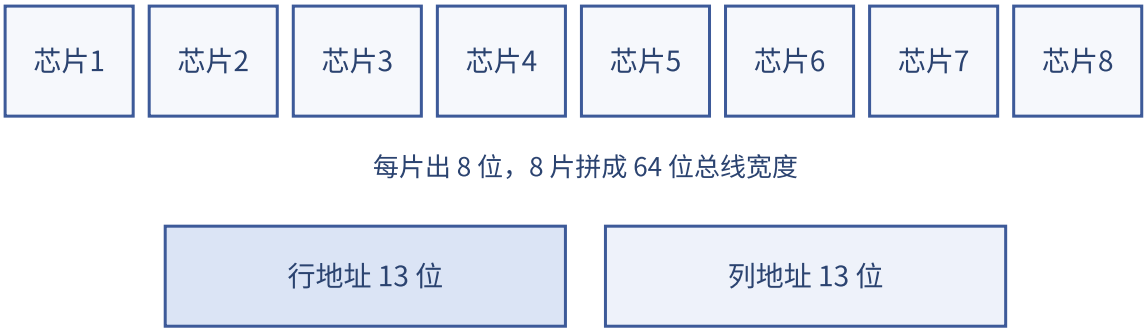
正确选项为什么对（本题找的是错述）： C 说地址引脚 26 位是错的。8192 = 2^{13}（2^{10} = 1024，2^{11} = 2048，2^{12} = 4096，2^{13} = 8192），行地址 13 位、列地址 13 位，但行列分两次共用同一组引脚送入 → 地址引脚只有 13 根，不是 13 + 13 = 26。错就错在"复用"二字被忽略了。

错误选项为什么错＋举例（其余三项都是对的）：

- A（总容量 512 MB）：8 片是分工拼宽度不是拼深度：每片 2^{26} 个单元，8 片并排每个地址读出 8 B → 2^{26} × 8 B = 536870912 B；536870912 ÷ 1024 ÷ 1024 = 512，所以是 512 MB。举例：8 片并排是拼宽度——每个地址同时从 8 片各取 8 位拼成 64 位；地址只有 2^{26} 个，不是 8 × 2^{26} 个。如果以为 8 片是 8 份独立地址空间、把地址数也乘 8，就会把容量算多——宽度和深度不能混着乘。
- B（多模块交叉编址）：多片并行交叉组织正是支撑 64 位总线宽和突发传送的落点。举例：连续地址轮流落在不同芯片上，突发传送时各芯片可以流水式出数据，带宽才上得去。
- D（行缓冲 8192×8 位）：行缓冲就是一整行：8192 个单元 × 每单元 8 位 = 8192×8 位。举例：访问某一行时，整行先被搬进缓冲，之后同一行的列访问都直接命中缓冲，不用再去阵列里取。

笔记本一句话： 第 17 题＋DRAM 三件事：容量按"单元数 × 位宽"、行缓冲等于一整行、地址引脚行列复用只算一份。

【图像】



DRAM 内存条图：8 片并排拼宽度与行列地址复用

【定位】 计算机组成原理 · 第 3 章存储器层次结构 · DRAM 芯片组织与内存条。

【易混对比】

- 拼宽度和拼深度：8 片并排是拼宽度，每个地址同时从 8 片各取 8 位，拼成 64 位；地址个数还是每片的 2^{26} 个，不乘 8。拼深度才是地址个数相加，那是另一回事。本题容量是地址个数乘拼出来的位宽，不是地址个数也乘 8。
- 行地址和列地址：行地址指出芯片阵列里的哪一行，列地址指出这一行里的哪一列；两个地址都是 13 位，但分两次、共用同一组引脚送入。位数相同不等于引脚数相加，复用是省引脚的关键。
- 行缓冲和整行数据：行缓冲是芯片里暂存一整行的缓冲，大小等于一行的数据量，本题是 8192 个单元乘每单元 8 位；它不是只存一个单元，也不是存整个芯片。

【坑位】 本题最核心的陷阱，是看到行地址 13 位、列地址 13 位，就把地址引脚数写成 $13+13=26$。行列地址复用的意思是行地址和列地址分两次走同一组引脚，引脚数只要 13 根。忽略“复用”两个字，就会把共用的一组引脚算成两组。

【套路】 DRAM 组织题目固定验三项：第一项验容量，容量 = 每片地址单元数 × 并排拼出的位宽，本题是 $2^{26}\times 8$ 字节 $=512$ MB；第二项验行缓冲，行缓冲 = 一行单元数 × 每单元位宽，本题是 8192×8 位；第三项验地址引脚，先算 $8192=2^{13}$ 得出行列地址各 13 位，再看有没有复用，有复用就只算一份 13 根，没有复用才相加。

【类比】 先想超市货架：8 片芯片像 8 个并排的货架格，每个格子一次各拿 8 件，拼起来一次拿 64 件，这叫拼宽度。行缓冲像把整排货架一次搬上柜台，之后同一排的东西直接在柜台上取，不用再回货架。行列地址复用像同一个门口分两次进货：先送行号，再送列号，门口只要一个，不用为了行和列各开一个门。

【口诀】 行列共用一根线，十三不是二十六。

第 18 题 · ISA 规定的内容（正确答案：B，仅 I、III）

题干与选项：

下列选项中，属于指令集体系结构（ISA）规定内容的是（ ）。

I. 指令字格式和指令类型

II. CPU 的时钟周期

III. 通用寄存器个数和位数

IV. 加法器的进位方式

- A. 仅 I、II
- B. 仅 I、III
- C. 仅 II、IV
- D. I、III、IV

入手点： 判断 ISA 只看一条线：软件看得见的是 ISA，看不见的是实现。

题干专有名词从零解释：

- ISA / 指令集体系结构：软件和硬件之间的合同——指令长什么样、有哪些指令、寄存器怎么编址。
- 指令字格式和指令类型：指令的二进制长相、操作数怎么安排。
- 时钟周期：CPU 一个节拍的时间，主频的倒数。
- 通用寄存器个数和位数：程序员能直接用的寄存器有几个、每个几位。
- 加法器的进位方式：加法电路内部是串行进位还是并行进位，属于电路实现细节。
- 微体系结构：ISA 之下的具体实现——流水线、电路、主频等，软件看不见。

考查知识点： ISA 与微体系结构的分界。

正确选项为什么对： I 是 ISA：写程序、写编译器必须知道指令长什么样。III 是 ISA：指令里要给寄存器编号、运算按位宽来，程序员必须知道。举例：同一套 ISA 可以有主频 3 GHz 和 5 GHz 的不同芯片，时钟周期不同但跑同一份程序——说明时钟周期不是 ISA 的一部分。

错误选项为什么错＋举例：

- A（含 II）：时钟周期是实现参数。举例：同一种 ISA 的两颗 CPU，主频可以一个 2 GHz、一个 4 GHz，指令集完全一样——时钟周期变了，ISA 没变。
- C（仅 II、IV，全是实现细节）：把分界线划反了。举例：按 C 的逻辑，程序员连指令格式都不知道，那编译器根本没法生成代码，C 的分界线划反了。
- D（I、III、IV，多含 IV）：加法器进位方式指令层面完全无感。举例：加法指令 ADD 算出 2 + 3 = 5，不管电路内部是串行进位还是并行进位，结果和程序员看到的指令一模一样。

笔记本一句话： 第 18 题＋判断 ISA 只看一条线：软件看得见的是 ISA，看不见的是实现。

【图像】



ISA 与微体系结构分界图

【定位】 计算机组成原理 · 第 4 章指令系统 · 指令集体系结构与微体系结构的分界。

【易混对比】

- ISA 和微体系结构：ISA 是软件和硬件之间的约定，规定指令长什么样、有哪些指令、寄存器有几个、每位多宽，写程序和写编译器的人必须知道；微体系结构是 ISA 之下的具体做法，比如流水线怎么排、电路怎么搭、主频跑多快，软件是看不见的。同一套 ISA 可以做出主频不同的芯片，ISA 本身不变。
- 指令字格式和电路实现：指令字格式是“指令的二进制长相”，属于 ISA；加法器内部是串行进位还是并行进位，是“电路怎么把加法算出来”，属于实现。长相是约定，做法是实现，约定不变、做法可以换。
- 寄存器个数和时钟周期：通用寄存器有几个、每个几位，指令里要给寄存器编号，程序员必须知道，属于 ISA；时钟周期是 CPU 一个节拍的时间长短，是芯片跑多快的参数，属于实现。个数和位宽进约定，速度进实现。

【坑位】 本题最核心的陷阱，是把“硬件参数”一股脑都当成 ISA 的内容。时钟周期和加法器进位方式听起来都是硬件的事，但 ISA 只管软件能看见、写程序必须依赖的部分；主频能换、进位电路能换，程序照样跑，说明这两项不是约定的一部分。判断时不要问“这是不是硬件”，要问“换掉它，程序员写出的程序要不要跟着改”。

【类比】 先想餐厅菜单和后厨做法：菜单上写了有哪些菜、每道菜叫什么、份量多大，这就是 ISA，点菜的人和写菜单的人都要按它来；后厨是用大火还是小火炒、是先放盐还是后放盐，这就是微体系结构，点菜的人吃到的菜名和份量不变，后厨做法换了也没关系。时钟周期像后厨炒菜的速度，加法器进位方式像后厨切菜的手法，都不在菜单上。

【口诀】 软件能见是约定，看不见的实现别算。

第 19 题 · 扩展操作码（正确答案：D，128）

题干与选项：

某计算机采用 16 位定长指令格式，地址码为 6 位。已有二地址指令 12 条、一地址指令 254 条，则零地址指令最多有几条（）。

- A. 0
- B. 2
- C. 64
- D. 128

入手点： 扩展操作码永远两步：剩余前缀数 × 2^地址码位数，少乘一步必错。

题干专有名词从零解释：

- 定长指令：本题 16 位，指令总长度固定。
- 地址码：指令里指出操作数地址的部分，本题 6 位。
- 二地址指令：指令里带 2 个地址码；一地址指令：带 1 个；零地址指令：不带地址码，操作数在栈顶等隐含位置。
- 操作码：指令里指出"做什么操作"的部分。
- 扩展操作码：上一级用剩的前缀展开成下一级的号段，像电话号段分配。
- 前缀：上一级操作码里留出来、准备展开的那几个编码。

考查知识点： 扩展操作码的容量计算。

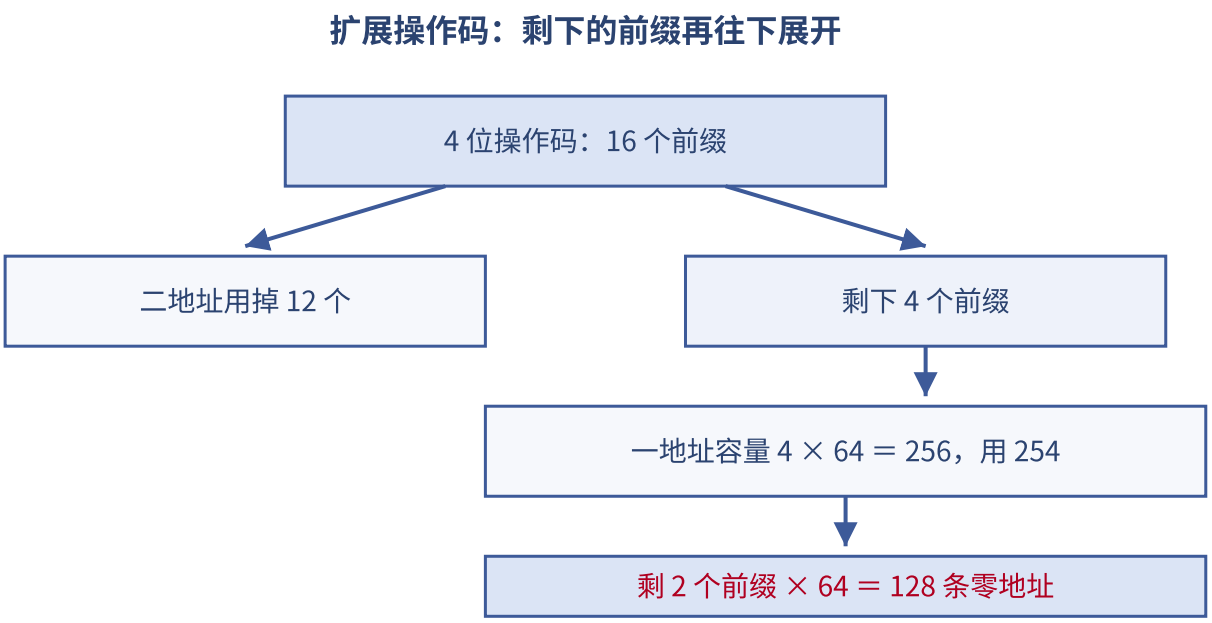
正确选项为什么对： 16 位指令、二地址占 2 × 6 = 12 位地址码 → 操作码剩 16 − 12 = 4 位，4 位二进制有 $2^4 = 16$ 种组合，即 16 个前缀；二地址用了 12 个，剩 4 个前缀。一地址指令：每个剩余前缀展开成 $2^6 = 64$ 条（比二地址少一个 6 位地址码，空出的 6 位有 $2^6 = 64$ 种组合），一地址容量 = 4 × 64 = 256 条；用 254 条，剩 2 个前缀。零地址：每个剩余前缀再展开 $2^6 = 64$ 条 → 2 × 64 = 128 条。

错误选项为什么错＋举例：

- A（0）：以为号段被一地址占满了。举例：256 条容量用了 254 条，还剩 2 个前缀没用，号段根本没占满。
- B（2）：就是"剩余前缀数"，不是指令条数。举例：剩 2 个前缀，每个前缀能展开 64 条零地址指令，2 只是前缀个数，忘了最后乘 2^6 这一步。
- C（64）：只相当于展开了 1 个前缀。举例：2 个剩余前缀 × 64 = 128，只算 1 个就少了一半。

笔记本一句话： 第 19 题＋扩展操作码永远两步：剩余前缀数 × 2^地址码位数，少乘一步必错。

【图像】



扩展操作码前缀展开树图：剩余前缀逐级展开成零地址指令

【定位】 计算机组成原理 · 第 4 章指令系统 · 扩展操作码的容量计算。

【易混对比】

- 前缀数和指令条数：前缀是上一级留出来、准备往下展开的编码个数；指令条数是前缀展开之后实际得到的指令个数。一个前缀能展开 $2^6 = 64$ 条指令，剩 2 个前缀不是 2 条指令，而是 $2 \times 64 = 128$ 条。
- 二地址、一地址、零地址指令的操作码长度：指令总长固定 16 位，地址码每条 6 位；二地址指令占 12 位地址码，操作码剩 4 位；一地址指令占 6 位地址码，操作码剩 10 位；零地址指令不占地址码，操作码占满 16 位。地址码越少，操作码能展开的空间越大。
- 容量和已用条数：容量是某一级最多能有的条数，已用条数是实际已经占掉的条数；剩余前缀要从“容量减去已用”里算，不能直接拿已用条数去乘。

【坑位】 本题最核心的陷阱，是算到“剩 2 个前缀”就停下来，直接选 2。2 只是前缀个数，不是零地址指令条数；每个剩余前缀还要再展开 $2^6 = 64$ 条，2×64 才是 128。少乘最后这一步，答案正好是正确结果的六十四分之一，是扩展操作码题目里最常见的断尾错误。

【套路】 扩展操作码题目固定走两步：第一步算剩余前缀数，本题是二地址操作码 4 位共 16 个前缀，用 12 个剩 4 个；4 个前缀各展开 64 条，一地址容量 256 条，用 254 条剩 2 个前缀；第二步把剩余前缀数乘以 $2^{\text{地址码位数}}$，本题是 $2 \times 2^6 = 128$ 条。记住口头禅：剩几段，就乘几段能展开的条数，不要把段数当成条数交卷。

【类比】 先想电话号段分配：4 位短号一共 16 个号段，分给二地址指令 12 个，剩下 4 个号段；每个号段后面再加 6 位号码，能分出 64 个一地址号码，一共 256 个，用掉 254 个还剩 2 个号段；每个号段再加 6 位，又能分出 64 个零地址号码，2 个号段就是 128 个号码。号段数不是电话数，号段后面加的位数才是放大倍数。

【口诀】 剩几段乘六十四，少乘一步就掉队。

第 20 题 · 源程序到可执行文件的过程（正确答案：A，预处理 → 编译 → 汇编 → 链接）

题干与选项：

将高级语言源程序转换为可执行目标文件的主要过程是（ ）。

- A. 预处理、编译、汇编、链接
- B. 预处理、汇编、编译、链接
- C. 预处理、编译、链接、汇编
- D. 预处理、汇编、链接、编译

入手点： 按产物倒推顺序：编译的产物是汇编程序，汇编的产物才是机器码。

题干专有名词从零解释：

- 预处理：展开头文件、宏替换、去掉注释，备料。
- 编译：把高级语言翻成汇编语言。
- 汇编：把汇编语言翻成机器码，生成目标文件。
- 链接：把多个目标文件和库拼成可执行文件。
- 源程序：人写的高级语言代码；目标文件：机器码零件，还不能直接运行；可执行文件：链接完成、能直接运行的整机。

考查知识点： 程序转换的四个阶段与顺序。

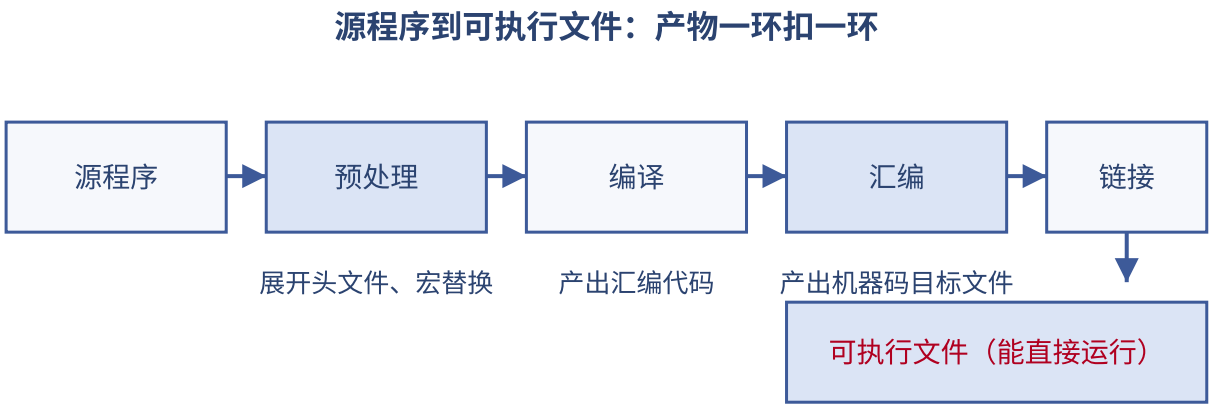
正确选项为什么对： 预处理备料（展开头文件、宏替换）→ 编译出汇编（人话菜谱翻成半成品）→ 汇编出机器码（半成品翻成零件）→ 链接拼整机（零件＋库拼成可执行文件）。按产物倒推：编译的产物是汇编程序，汇编的产物才是机器码，所以编译必在汇编之前。

错误选项为什么错＋举例：

- B（编译与汇编对调）：编译的产物就是汇编代码，对调后逻辑讲不通。举例：编译器输出的是 .s 汇编文件，汇编器读 .s 输出 .o 机器码——如果汇编在编译前面，汇编器连输入都没有。
- C（链接提到汇编之前）：还没有目标文件，无可链接。举例：链接器的输入是 .o 目标文件，汇编还没跑出来，哪来的 .o 给它链？
- D（后三步顺序全错）：预处理开头是对的，但汇编跑到了编译前面、链接也跑到了编译前面，产物链条从头断到尾。举例：编译的产物是汇编代码，汇编器读汇编出机器码——汇编在编译前面，汇编器连输入都没有；链接的输入是目标文件，汇编没跑出来，哪来的目标文件给它链？

笔记本一句话： 第 20 题＋预处理备料、编译出汇编、汇编出机器码、链接拼整机，顺序不能换。

【图像】



源程序到可执行文件的四阶段流程图

【定位】 计算机组成原理 · 第 1 章计算机系统概述 · 高级语言源程序到可执行文件的转换过程。

【易混对比】

- 编译和汇编：编译是把高级语言翻成汇编语言，产出的是汇编代码；汇编是把汇编语言翻成机器码，产出的是目标文件。两个步骤名字像、都是翻译，但输入和产出完全不同，顺序也不能换。
- 汇编和链接：汇编产出的是单个目标文件，还不能直接运行；链接是把多个目标文件和库拼成一个可执行文件，拼完才能直接运行。目标文件是零件，可执行文件是整机，零件没做出来就没东西可拼。
- 预处理和编译：预处理是备料，展开头文件、替换宏、去掉注释，产出的还是高级语言样子；编译是正式翻译，从高级语言翻成汇编语言。备料在前、翻译在后，备料没做完，翻译拿到的就是缺材料的原文。

【坑位】 本题最核心的陷阱，是觉得编译和汇编“都是翻译，顺序无所谓”。顺序由产出链条定死：编译的产出是汇编代码，正好是汇编步骤的输入；汇编的产出是机器码目标文件，正好是链接步骤的输入。把汇编放在编译前面，汇编器连输入都没有；把链接放在汇编前面，链接器连目标文件都没有。按产物倒推一步，顺序就不会乱。

【套路】 程序转换顺序题目固定用产物倒推：先问每一步产出什么，预处理产出处理过的高级语言代码，编译产出汇编代码，汇编产出机器码目标文件，链接产出可执行文件；再把产出和下一步的输入对上号，产出能接上下一步输入的顺序就是正确顺序。本题对下来就是预处理、编译、汇编、链接，任何一步对不上号的选项都可以直接排除。

【类比】 先想做一道菜上桌：预处理是备料，洗菜切菜、把调料摆好；编译是按菜谱把备好的料写成半成品做法，相当于把家常话翻成厨师能直接照做的步骤；汇编是把步骤变成机器能执行的动作，相当于把做法变成一道道已经做好的零件菜；链接是把零件菜和配菜拼成一盘能上桌的整菜。备料、写做法、做零件、拼整盘，顺序乱了，后一步就没东西可接。

【口诀】 预编汇链四步走，备料翻汇翻码拼整机。

第 21 题 · 中断 I/O 方式（正确答案：C，不正确）

题干与选项：

下列关于中断 I/O 方式的叙述中，不正确的是（ ）。

- A. 中断 I/O 方式适用于键盘、针式打印机等慢速字符设备
- B. 中断 I/O 方式下，数据传送是通过软件完成的
- C. 外设准备数据的时间应小于中断处理时间
- D. 在中断 I/O 方式下，外设准备数据的过程中 CPU 可以执行其他进程

入手点： 中断 I/O = 慢设备、软件搬运、并行工作；时间大小关系是"准备远大于处理"。

题干专有名词从零解释：

- 中断 I/O 方式：外设备好数据就发中断信号，CPU 响应后搬运。
- 中断：外设打断 CPU、请求服务的信号。
- 中断服务程序：CPU 响应中断后执行的、搬运数据的那段代码。
- DMA：直接存储器访问，硬件成批搬运、CPU 基本不插手（和中断方式的分界线）。
- 外设准备数据时间：外设把一个数据准备好要多久，慢，毫秒级。
- 中断处理时间：CPU 响应中断、搬走数据要多久，快，微秒级。
- 慢速字符设备：键盘、针式打印机这类一次传一个字符、速度慢的设备。

考查知识点： 中断 I/O 的特征与时间关系。

正确选项为什么对（本题找的是错述）： C 把大小关系说反了。外设准备数据（慢，毫秒级）必须远大于中断处理时间（快，微秒级），否则 CPU 会被门铃淹没——备好一个字就响一次门铃，处理必须远快于备下一份的时间。原说法"准备时间应小于处理时间"正好反了。

错误选项为什么错＋举例（其余三项都是对的）：

- A（键盘、针式打印机适用）：举例：键盘敲一个键才产生一个字符，速度极慢，正好用"备好就响门铃"的中断方式；换成高速磁盘用中断方式，门铃会响到 CPU 干不了别的活。
- B（数据传送通过软件完成）：举例：中断到来时，是 CPU 停下手头活、执行中断服务程序把数据从设备寄存器取走——搬运动作是软件（CPU 执行程序）完成的，这正是中断方式与 DMA（硬件搬运）的分界线。
- D（外设准备数据时 CPU 运行其他进程）：举例：打印机备数据的几毫秒里，CPU 不用干等，去跑别的进程；门铃一响再回来取——这就是中断方式的意义：CPU 与外设并行。

笔记本一句话： 第 21 题＋中断 I/O = 慢设备、软件搬运、并行工作；准备时间必须远大于处理时间。

【图像】

外设准备 vs 中断处理：准备远比处理慢



注：不是真实比例，1 毫秒 = 1000 微秒，短条已放大才看得见。
门铃响一次，要在下一个数据备好之前搬走，处理必须远快于准备。

外设准备时间与中断处理时间的时间轴对比

上图里上面那条长条，是一台慢设备把一个数据准备好要花的时间，单位是毫秒；下面那条短条，是处理器收到通知后把数据搬走要花的时间，单位是微秒。两条条的长度不是按真实比例画的，因为 1 毫秒等于 1000 微秒，按真实比例画短条就看不见了，图里是把短条放大了才画出来的，关键看的是量级关系：准备远比处理慢。

【定位】计算机组成原理 · 第 7 章 · 输入输出系统 · 中断 I/O 方式

【易混对比】

- 门铃式搬运：设备把一个数据准备好，就发一个信号请处理器来取，处理器停下手头的事、跑一段搬运程序把数据取走，这就是中断 I/O 方式，搬运靠软件完成。
- 硬件成批搬运：设备攒够一批数据后，由专门的硬件直接把整批数据搬进内存，处理器基本不插手，这就是直接存储器访问，搬运靠硬件完成。
- 区分点：一次只传一个字、速度很慢的设备，适合门铃式；一次传一大批、速度很快的设备，用门铃式会把处理器响到没空干别的活，适合硬件成批搬运。

【坑位】

最核心的陷阱，是把“准备”和“处理”两个时间的主角弄混。错误的说法听起来像在讲“处理要从容一点，所以处理时间应该更长”，可真实的顺序是反过来的：设备每准备好一个数据就按一次门铃，处理器必须在下一个数据准备好之前把上一个搬走，处理一定要远快于准备。反过来，如果处理比准备还慢，门铃就会越积越多，处理器会被门铃淹没，什么正事都干不了。记住量级就不会反：准备是毫秒级，处理是微秒级。

【套路】

碰到问两个时间大小关系的题，按三步走：

- 先给两个时间标量级：准备是毫秒级，处理是微秒级。
- 再比大小：准备远大于处理。
- 最后反问自己一句：如果反过来，门铃会不会越积越多、把处理器淹没？会，就说明反过来的说法是错的。

【类比】

先想一个厨房场景：厨师（外设）慢慢做一道菜要花好几分钟，做好了按一下门铃；主人（处理器）听到门铃去端菜，只要几秒钟就端走了。厨师做菜就是外设准备数据，慢；主人端菜就是中断处理，快。门铃响一次，主人必须在下一道菜做好之前把上一道端走，不然菜会在门口越堆越多。中断 I/O 方式就是这个门铃机制：设备慢，处理快，设备准备数据的过程里处理器还可以去干别的活。

【口诀】

准备毫秒长，处理微秒短，门铃慢响才不乱。

第 22 题 · 并行处理技术（正确答案：C，不正确）

题干与选项：

下列关于并行处理技术的叙述中，不正确的是（）。

- A. 多核处理器是 MIMD 的典型代表
- B. 向量处理器是 SIMD 的典型代表
- C. 硬件多线程只可用于多核处理器
- D. 对称多处理（SMP）中各处理器共享单一物理地址空间

入手点： Flynn 看指令流 × 数据流；概念题里出现"只、仅、必须"这类绝对说法，先打问号。

题干专有名词从零解释：

- Flynn 分类法：按指令流条数 × 数据流条数给机器分型。
- SISD：单指令流单数据流，传统单核；SIMD：单指令流多数据流，一条指令同时算一串数据；MISD：多指令流单数据流，罕见；MIMD：多指令流多数据流，多个核各跑各的。
- 多核处理器：一个芯片里多个核。
- 向量处理器：一条指令处理一串数据的处理器。
- 硬件多线程：靠多套寄存器等硬件上下文快速切换线程的技术。
- SMP / 对称多处理：多个处理器对等、共享同一个物理地址空间。

考查知识点： Flynn 分类与硬件多线程、SMP 概念。

正确选项为什么对（本题找的是错述）： C 说硬件多线程"只可用于多核"是错的。硬件多线程靠多套线程上下文（寄存器组等）实现快速切换，和有几个核无关，单核照样能做。举例：单核 CPU 配两套寄存器，切线程时不用把现场存到内存、直接换一套寄存器就跑，这就是单核上的硬件多线程。

错误选项为什么错＋举例（其余三项都是对的）：

- A（多核处理器是 MIMD 的典型）：举例：4 核 CPU，4 个核各跑各的程序、各处理各的数据——4 条指令流 × 4 条数据流，正是 MIMD。
- B（向量处理器是 SIMD 的典型）：举例：一条向量加法指令同时算 8 个数的加法——1 条指令流 × 8 条数据流，正是 SIMD；"多"的是数据，不是核。
- D（SMP 共享单一物理地址空间）：举例：两颗对等的 CPU 看到的是同一份内存地址空间，A 核写 0x1000，B 核读 0x1000 读到的是同一个位置——这就是 SMP 的定义特征。

笔记本一句话： 第 22 题＋Flynn 看指令流 × 数据流；硬件多线程不挑核数，单核也能玩。

【图像】

Flynn 分类：指令流 x 数据流



硬件多线程：单核也能用，不看核数，不属于上面任何一格。

指令流条数乘数据流条数的 2×2 分类格

横着看是数据流有几条，竖着看是指令流有几条，四个格子就是四种机器类型。格子里的例子要和格子的位置对上：一条指令同时算一串数，落在单指令流、多数据流那格；几个核各跑各的程序、各算各的数据，落在多指令流、多数据流那格。图下面还单独写了一句：硬件多线程不看核数，单核也能用，它不属于这张格子里的任何一格，别把它往多核那格里硬塞。

【定位】计算机组成原理 · 第 6 章 · 中央处理器 · 并行处理技术与 Flynn 分类

【易混对比】

- 一条口令、一群人齐步做同一个动作：指挥员只喊一个口令，所有人同时做同一件事，只是每人手里的数据不同，这就是单指令流多数据流，典型例子是向量处理器，一条加法指令同时算好几个数的加法。
- 每人各领各的活、各做各的事：几个房间里的人互不干扰，有人唱歌有人算账，这就是多指令流多数据流，典型例子是多核处理器，几个核各跑各的程序。
- 多核与硬件多线程的区别：多核是人多了，一个芯片里有好几个能独立干活的核；硬件多线程是一个干活的人备了好几套工具，干到一半换一套工具就能接着干另一件活，换活不用把台面收拾干净重摆。工具套数和人数是两回事，所以单核也能配多套工具。

【坑位】

最核心的陷阱，是“只可用于多核”里的“只”字。看到“多线程”就联想到“要好几个核才转得起来”，这个联想听起来顺，可切换线程靠的是多套现场（寄存器组这类硬件），不是靠核的数量。单核配两套现场，切线程时直接换一套现场就能跑，根本不用等第二个核。所以凡是给一个技术配“只、仅、必须”这类绝对说法，先打一个问号，再去看它真正依赖的是什么。

【套路】

碰到分类题，按两步走：

- 先数两条线：指令流有几条？数据流有几条？数完直接对表入格，不用背整张表的定义。
- 再查绝对词：选项里出现“只、仅、必须”时，先问它依赖的到底是什么。硬件多线程依赖的是多套现场，不是核数，核数一变，结论就不成立。

【类比】

先想一个厨房场景：一个厨师备了两套刀具，切菜切到一半，换一套刀具就能接着切肉，不用把案板收拾干净、把菜刀洗好放回去再拿肉刀，这就是硬件多线程，换活快靠的是备了多套工具，不是一个厨房里有几个厨师。几个厨师各做各的菜，那是多核；一个口令让一排帮厨同时切同一批菜，那是单指令流多数据流。人数、工具套数、口令条数，是三个不同的东西。

【口诀】

指令数据数两条，单核多线也照跑。

第 23 题 · 多道程序系统与"抖动"（正确答案：D，不正确）

题干与选项：

下列关于多道程序系统的叙述中，不正确的是（）。

- A. 多道程序系统中的程序并发执行
- B. 多道程序系统不必支持虚拟存储管理
- C. 多道程序系统共享资源时需要进行管理
- D. 多道程序系统中进程数越多，CPU 利用率越高

入手点： 多道靠错峰提利用率，但进程过多会抖动，利用率过犹不及。

题干专有名词从零解释：

- 多道程序设计：内存里同时放几道程序，一道等 I/O 时 CPU 去跑另一道。
- 分区：把内存切成若干块分给不同程序；覆盖：程序分段，轮流装入同一块内存；交换：把暂时不跑的程序整体换到磁盘上，腾出内存给别人。
- 并发：宏观上多道程序同时推进。
- 资源共享：CPU、内存、设备大家轮流用，需要管理。
- 虚拟存储：用磁盘作后盾、让程序看到比内存大的地址空间，后来才加的配置。
- 抖动 / 颠簸：进程太多、每人分到的页框太少，刚调进来的页马上又被换出去，CPU 大部分时间在换页而不是干活。

考查知识点： 多道程序的基本概念与抖动。

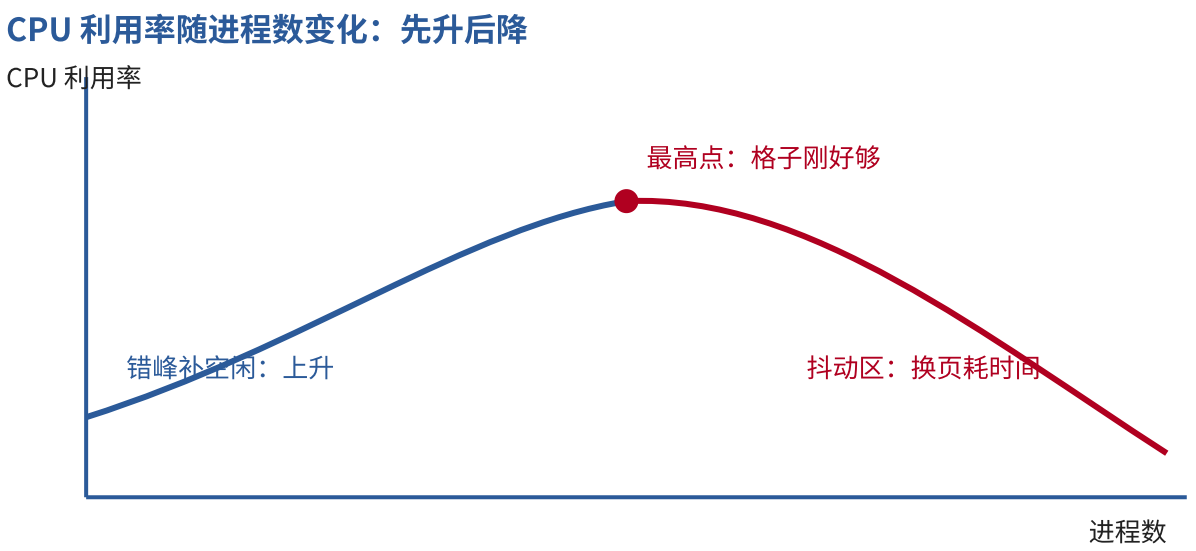
正确选项为什么对（本题找的是错述）： D 说"进程数越多 CPU 利用率越高"，说成了单调上升。事实是：进程数增加到一定程度后，每人分到的页框太少，抖动发生，CPU 全忙着换页，利用率反而下降。举例：内存 100 页框，10 道程序每道分 10 页框，刚好够用；硬塞 50 道，每道只剩 2 页框，每道程序跑两步就缺页换页，CPU 利用率从 90% 跌到 30% 以下。

错误选项为什么错＋举例（其余三项都是对的）：

- A（多道程序并发执行）：举例：A 程序等磁盘时 CPU 切去跑 B，B 等打印机时再切回 A——宏观上 A、B 同时在推进，这就是多道的定义。
- B（多道不必支持虚拟存储）：举例：早期系统靠分区、覆盖、交换就能把几道程序同时装进内存、多道运行，虚拟存储是后来才加的豪华配置——"现代系统都有虚存"不能倒推成"多道必须有虚存"。
- C（共享资源需要管理）：举例：两道程序同时往同一台打印机输出，没有管理者分配，输出会混成一团——共享必然需要管理，否则资源会打架。

笔记本一句话： 第 23 题＋多道靠错峰提利用率，但进程过多会抖动，利用率过犹不及。

【图像】



CPU 利用率随进程数先升后降的曲线，拐点后进入抖动区

横轴是内存里的进程数，纵轴是处理器利用率。曲线先往上走：进程少的时候，一道程序等设备，处理器就空着，多放几道程序轮流补空闲，利用率就上去了。走到最高点后曲线往下掉：进程太多，每道程序分到的内存格子不够，刚调进来的页马上又被换出去，处理器的时间花在换页上，真正干活的时间反而少了，往下掉的那段就是抖动区。

【定位】操作系统 · 第 1 章 · 操作系统概述 · 多道程序设计与抖动

【易混对比】

- 宏观上同时推进：几道程序在一段时间内轮流上处理器，你跑一会儿他跑一会儿，从外面看大家都在往前走，这就是并发，不要求同一时刻真的同时在跑。
- 每道程序分到的内存格子太少：程序最近常翻的那叠页叫工作集，内存里放一页的格子叫页框，格子装不下那叠页，程序跑两步就缺页换页，换页的时间比干活还多，这就是抖动。
- 区分点：并发是“轮流推进、互相补空闲”，抖动是“格子不够、时间耗在换页上”，一个是多道的好处，一个是多道过头后的毛病。

【坑位】

最核心的陷阱，是把“越多越好”想成一条直线单调上升。进程多起来、错峰补空闲，这段话前半是对的，可它忘了拐点：进程数超过一定程度后，每道程序分到的页框太少，抖动一来，利用率反而往下掉。看到选项里出现“越多越高、越多越好”这类说法，先问一句：有没有拐点？有没有过头后反而变差的可能？有，就不能选单调上升的说法。

【套路】

碰到问“进程数和利用率关系”的题，脑子里画一条先升后降的曲线，分三段想：

- 先升段：进程少，互相错峰补空闲，利用率往上走。
- 顶点：每道程序分到的页框刚好够装它的工作集。
- 后降段：进程继续加，人均页框不够，抖动出现，利用率往下掉。

【类比】

先想一张自习室的大桌子：两个人轮流用桌子，一个人去接水时另一个人坐下来看书，桌子很少空着，这就是多道错峰的好处。可要是硬塞进来十个人，每人只分到桌子的一角，连一本书都摊不开，每个人都要不停地把书从书包里拿出来又放回去，真正看书的时间反而少了。这就是抖动：人人分不到整张桌子，时间都耗在搬书上，桌子的真正使用效率不升反降。

【口诀】

错峰利用往上走，塞满抖动往下掉。

第 24 题 · 操作系统初始化时创建什么（正确答案：A，中断向量表）

题干与选项：

在操作系统初始化（启动）过程中，需要创建的是（ ）。

- A. 中断向量表
- B. 文件系统根目录
- C. 硬盘分区表
- D. 文件系统索引节点表

入手点： 判断"是否在 OS 初始化时创建"，就问一句：它是住在内存里还是住在磁盘上？

题干专有名词从零解释：

- 操作系统初始化 / 启动：开机时 OS 建立运行环境的过程。
- 中断向量表："中断号 → 处理程序入口地址"的电话簿，放在内存固定位置。
- 文件系统根目录：文件系统的顶层目录。
- 硬盘分区表：写在磁盘最前面扇区、记录分区划分的表。
- 文件系统索引节点表 / inode 表：记录每个文件元数据的区域。
- 格式化：创建文件系统、在磁盘上划好各区域的操作。

考查知识点： OS 引导 / 初始化时创建的对象。

正确选项为什么对： 中断向量表只存在于内存里，断电就消失，必须每次开机由 OS 初始化代码亲手建好、填好。没有它，中断根本没法响应：CPU 一遇到中断（键盘敲击、时钟到点），就按中断号查这张表、跳过去处理。

错误选项为什么错＋举例：

- B（文件系统根目录）：根目录是文件系统的一部分，格式化建文件系统时就写在磁盘上了。举例：新硬盘格式化一次，根目录就写进去了，之后每次开机 OS 只是读它、不是重建它。
- C（硬盘分区表）：分区表是给磁盘分区时写进磁盘最前面扇区的。举例：分区分好后，分区表就躺在磁盘上，开机时 OS 只是读取它来认分区，不是创建它。
- D（索引节点表）：inode 表同样是创建文件系统时在磁盘上划好的区域。举例：格式化时划出多大一块放 inode，之后每个文件的 inode 就住在那块里，开机不重建。

笔记本一句话： 第 24 题＋开机现建的只有内存里的管理表；分区表、根目录、inode 表都是装机时写在磁盘上的。

【图像】



开机现建与开机只读的对照图，内存里的中断向量表开机现建，磁盘上的分区表、根目录、索引节点表开机只读

图左边是内存，断电就消失，所以里面的中断向量表必须每次开机现建；图右边是磁盘，断电后还在，所以分区表、根目录、索引节点表开机时只是被读出来认一认，不是重新创建。看图先分清住在哪里，再判断是不是开机创建。

【定位】操作系统 · 第 1 章 · 操作系统概述 · 系统引导与初始化

【易混对比】

- 内存里的电话簿：中断号对应处理程序入口地址的一张对照表，放在内存固定位置，断电就消失，每次开机必须由初始化代码亲手建好、填好，这就是中断向量表。
- 磁盘上的旧档案：硬盘分区表、文件系统根目录、索引节点表，都是在给磁盘分区、创建文件系统时就写在磁盘上的，断电后还在，开机时只是被读出来认一认，不是重新创建。
- 创建与读取的区别：创建是第一次把一张表写出来；读取是开机把已经写好的表拿来用。题目问的是“初始化时创建什么”，问的是创建，不是“开机时会用到什么”。

【坑位】

最核心的陷阱，是把“开机都要用到”当成“开机都要创建”。四个选项里的东西开机时确实都可能被用到、被读到，可只有内存里的那张电话簿断电会丢，必须现建；写在磁盘上的三样东西断电还在，开机不用重建。判断时别问“开机用不用它”，要问“断电后它还在不在”：还在，就是磁盘旧物；不在，就是开机现建。

【套路】

碰到问“初始化时创建什么”的题，按两问走：

- 第一问：断电后还在吗？还在，就是写在磁盘上的旧物，开机只读不建；不在，就是只住在内存里的表，开机必须现建。
- 第二问：它住在哪里？住在内存固定位置的，断电消失；住在磁盘区域的，长期保存。

【类比】

先想两种纸：一种是每天上班现贴在墙上的值班电话表，下班撕掉，第二天上班要重新贴一张，这就是中断向量表，内存里的东西，关机就没了；另一种是办好后长期放在抽屉里的房产证，户型图写进去一次就一直在，每天只是翻出来看一眼，这就是分区表、根目录、索引节点表这类磁盘上的东西，装机时写好，开机只是读。

【口诀】

内存表开机现建，磁盘表装机先写。

第 25 题 · 优先级抢占调度：调度次数（正确答案：C，6 次）

题干与选项：

有四个进程 P0、P1、P2、P3，其到达时间、优先级和需要时间如下表所示（优先级数值越小越优先，采用抢占式调度）：

| 进程 | 到达时间 | 优先级 | 需要时间 |

| P0 | 0 ms | 15 | 100 ms |

| P1 | 10 ms | 20 | 60 ms |

| P2 | 10 ms | 10 | 20 ms |

| P3 | 15 ms | 6 | 10 ms |

从 0 ms 到全部进程结束，共发生进程调度多少次（ ）。

- A. 4 次
- B. 5 次
- C. 6 次
- D. 7 次

入手点： 抢占调度画时间线，调度次数 = 进程上台的总段数，含开局、含回锅。

题干专有名词从零解释：

- 抢占式调度：高优先级进程到达时立刻踹下正在跑的进程。
- 优先级：本题数值越小越优先（P3 的 6 全场最高）。
- 到达时间：进程进入就绪队列的时刻。
- 服务时间 / 需要时间：进程总共要跑多久。
- 就绪队列：等着上 CPU 的进程排队。
- 调度次数："选一个进程上 CPU"的次数，包括开局第一次和被抢占后的重新上台。

考查知识点： 抢占式优先级调度的过程推演与调度次数统计。

正确选项为什么对： 画时间线：0 ms 只有 P0 → P0 上台（第 1 次）；10 ms P1（20）、P2（10）到达，P2 的 10 高于 P0 的 15 → 抢占，P0 已跑 10 ms、剩 90 ms 回队列，P2 上台（第 2 次）；15 ms P3（6）到达，全场最高 → 抢占 P2，P2 跑了 5 ms、剩 15 ms，P3 上台（第 3 次）；25 ms P3 跑完（10 ms），队列里 P2（10）最高 → P2 第二次上台（第 4 次），跑完剩下 15 ms 到 40 ms；40 ms P2 结束，队列剩 P0（15）、P1（20），P0 上台（第 5 次），跑完剩下 90 ms 到 130 ms；130 ms P0 结束，P1 上台（第 6 次），跑 60 ms 到 190 ms 全部结束。合计上台 6 次。

错误选项为什么错＋举例：

- A（4 次）：只数了"4 个进程各上台一次"。举例：P2 被 P3 抢占后 25 ms 又上台一次、P0 被 P2 抢占后 40 ms 又上台一次——这两段回锅漏数了。
- B（5 次）：数的是切换次数（6 段之间 5 个接缝），或漏了开局那次。举例：6 段 = 开局 1 次＋切换 5 次，只数切换就少 1。
- D（7 次）：把进程到达也误算成了一次调度。举例：10 ms 时 P1、P2 到达，15 ms 时 P3 到达——到达只是进队列排队，只有"上台"才算调度，到达本身不算。

笔记本一句话： 第 25 题＋抢占调度画时间线，调度次数 = 进程上台的总段数，含开局、含回锅。

【图像】

抢占调度时间线：0-190 ms，共 6 段



1. P0 0-10 ms
2. P2 10-15 ms
3. P3 15-25 ms
4. P2 25-40 ms（回锅）
5. P0 40-130 ms（回锅）
6. P1 130-190 ms

数段数 = 6，含开局、含回锅；切换接缝只有 5 个，别数缝。

优先级抢占调度的甘特图，0–190 ms 共六段上台

图里一条横向时间线从 0 ms 画到 190 ms，上面六段分别是在台上跑的进程，段与段的交界处就是一次抢占或者一次跑完交班。图下面用编号列出了每一段是谁、从几毫秒跑到几毫秒、跑了多久：第一段和第五段是同一个进程被抢占下去后又回锅上台，第二段和第四段也是同一个进程被抢占后又回锅。数调度次数就是数这六段，不是数有几个进程，也不是数有几个交界处。

【定位】操作系统 · 第 2 章 · 进程管理 · 处理机调度 · 抢占式优先级调度

【易混对比】

- 上台段数：一个进程被选中、上处理器跑的那一段，算一次调度，开局第一次上台也算，这就是调度次数。
- 换人接缝：两段之间的交界处，算一次切换。六段之间只有五个接缝，所以切换次数总比调度次数少 1。
- 到场排队：一个进程到达、进入等着上处理器的队列，只是排队，不算上台。到达几次不等于调度几次。
- 优先级数值大小：本题里数值越小越优先，数值 6 比数值 15 更优先，别按数值越大越优先反着排。

【坑位】

最核心的陷阱，是漏数回锅，或者只数接缝。四个进程不等于上台四次：被高优先级进程抢占下去的进程，后来还会再上台，那段回锅也要算；六段之间只有五个接缝，只数接缝会漏掉开局那一次。记住一句：数段不数缝，数上台不数到达。

【套路】

碰到抢占调度的计数题，按四步画时间线：

- 按到达时间把进程列出来，每个到达点都看一眼队列里谁的优先级最高。
- 每个到达点比一次优先级：新到的优先级更高，就把台上那个换下来，记下它已经跑了多久、还剩多久。
- 画出每一段谁在台上、从几毫秒到几毫秒、跑了多久。
- 数段数：含开局第一段，含被抢占后的回锅段，段数就是调度次数。

【类比】

先想一场晚会抢话筒：主持人手里的话筒一共递出去几次，就是调度次数。嘉宾到场只是进了后台排队，不算递话筒；两个人换一次话筒，那次换人比递话筒少算一次，因为第一次递出去不算换人；被大牌嘉宾打断、先下台等着的人，后来轮到他还会再上台接话筒，那次回锅也要算一次。话筒递出去的总次数，就是时间线上的总段数。

【口诀】

画线数段含开局，抢占回锅别漏计。

第 26 题 · 银行家算法：数安全序列（正确答案：B，2 个）

题干与选项：

系统中有三个进程 P0、P1、P2 和三类资源 A、B、C。当前已分配矩阵、尚需矩阵和可用资源向量如下：

已分配 Allocation：P0 (2, 0, 1)，P1 (0, 2, 0)，P2 (1, 0, 1)

尚需 Need：P0 (0, 2, 1)，P1 (1, 2, 3)，P2 (0, 1, 3)

可用 Available：(1, 3, 2)

此时系统的安全序列有几个（）。

- A. 1 个
- B. 2 个
- C. 3 个
- D. 4 个

入手点： Work 从可用资源起步，Need 不超 Work 才能跑、跑完加回它的已分配，每个分叉都要枚举到底。

题干专有名词从零解释：

- 银行家算法：分配资源前先预判"会不会死锁"的算法。
- 安全序列：一种进程排队顺序，按它走每个进程都能拿到所需资源、跑完并释放。
- 安全状态：至少存在一个安全序列的状态。
- 已分配 Allocation：进程已经拿在手里的资源。
- 尚需 Need：进程跑完还差的资源。
- 可用 Available：系统当前空闲的资源。
- 工作向量 Work：推演过程中"当前可用＋已释放"的合计。

考查知识点： 银行家算法安全序列的枚举。

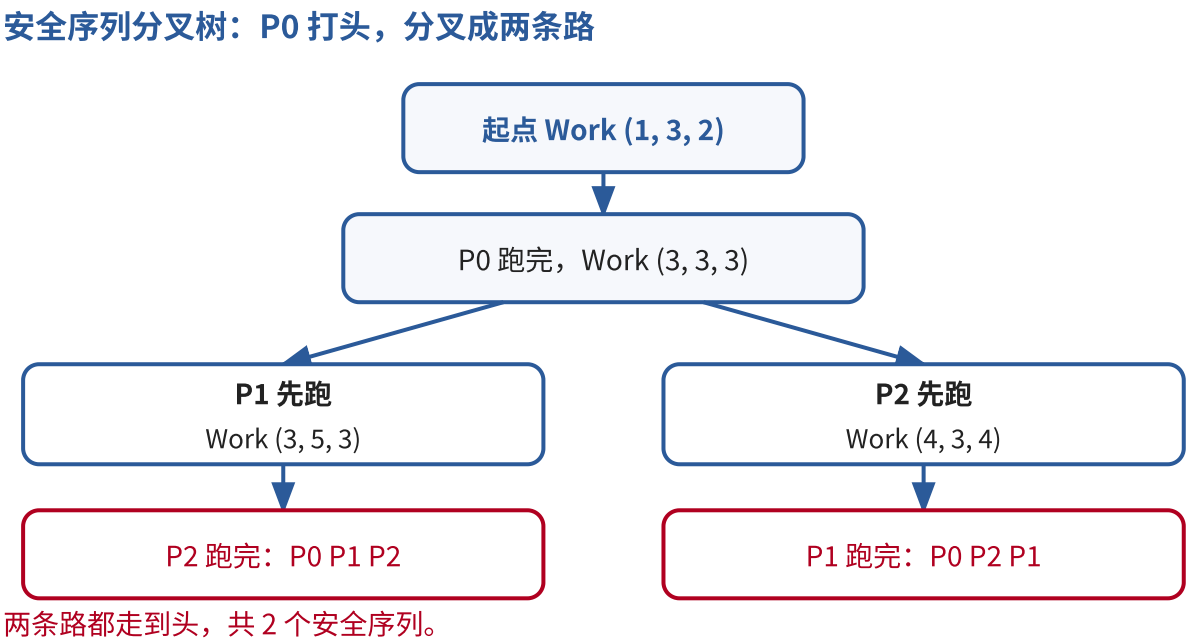
正确选项为什么对： Work 初始 = Available = (1, 3, 2)。逐个验 Need：P0(0, 2, 1) 全 ≤ Work ✓；P1(1, 2, 3) 里 C 要 3、可只有 2 ✗；P2(0, 1, 3) 同样卡在 C ✗ ——所以第一个跑完的只能是 P0，没有分叉。P0 释放 Allocation(2, 0, 1) 后 Work = (1, 3, 2) + (2, 0, 1)：逐分量相加，1+2 = 3、3+0 = 3、2+1 = 3，得 (3, 3, 3)。此时 P1(1, 2, 3) ✓、P2(0, 1, 3) ✓ 都能跑，分叉：分支一 P1 先：P1 释放 (0, 2, 0)，Work = (3, 3, 3) + (0, 2, 0) = (3, 5, 3)，P2 ✓ 跑完 → ⟨P0, P1, P2⟩；分支二 P2 先：P2 释放 (1, 0, 1)，Work = (3, 3, 3) + (1, 0, 1) = (4, 3, 4)，P1 ✓ 跑完 → ⟨P0, P2, P1⟩。两条路都通到底，共 2 个。

错误选项为什么错＋举例：

- A（1 个）：找到一条序列就收工。举例：只推了 ⟨P0, P1, P2⟩ 就停，没回头试 P2 先跑的分支——题目问的是"几个"，每个分叉都要枚举到底。
- C（3 个）：多半把 P1 或 P2 也算成了可打头。举例：P1 第一步就要 C 资源 3 个，可用只有 2 个，第一步就被卡死，不可能打头。
- D（4 个）：把排不通的顺序也算了进去。举例：⟨P1, …⟩ 打头的顺序第一步就走不通，⟨P2, …⟩ 也一样，第一步就走不通——排不通的不能算安全序列。

笔记本一句话： 第 26 题＋数安全序列：Work 从可用资源起步，Need 不超 Work 才能跑、跑完加回它的已分配，每个分叉都要枚举到底。

【图像】



安全序列枚举分叉树，P0 打头后分叉为 P1 先和 P2 先两条路

树从最上面一个起点开始：起点是系统当前空闲的资源。第一步只有一条路能走，就是先让第一个进程跑完，跑完后它手里的资源还回来，空闲资源变多。到了第二层，树分叉成两条：一条是先让第二个进程跑、再让第三个跑，另一条是反过来。两条路都走到头、三个进程都跑完，才算两条安全序列，数的时候两条都要数，少数一条就错了。

【定位】操作系统 · 第 2 章 · 进程管理 · 死锁避免 · 银行家算法

【易混对比】

- 还差多少：一个进程跑完还差的资源，叫尚需，推演时要拿它和当前空闲比。
- 手里已有多少：一个进程已经拿在手里的资源，叫已分配，进程跑完后这部分会还回来。
- 系统当前空闲多少：系统手里还没分出去的资源，叫可用，推演就是从它起步。
- 推演中的合计：推演过程中“当前空闲加上已经跑完的进程还回来的资源”，叫工作向量，它会越跑越大，不是固定不变的可用数。
- 安全状态与安全序列：安全状态是“至少有一条路走得通”这个状态本身；安全序列是那条走得通的排队顺序本身。题目问“有几个序列”，就是问有几条不同的路走得通。

【坑位】

最核心的陷阱，是找到一条路就收工。第一条路走通了，不代表只有这一条路；分叉点后面每一条叉都要走到头、数进去。反过来，第一步就卡住的进程不能打头：它还差的资源超过了当前空闲，它自己跑不起来，就不会有“它跑完还资源”这回事，不能靠后面还没发生的释放来硬算它能打头。先找唯一能打头的，再数后面的分叉。

【套路】

碰到数安全序列的题，按四步走：

- 工作向量从可用资源起步。

- 逐个比尚需：尚需每一项都不超过工作向量，这个进程才能跑；先找出第一步唯一能跑的进程。
- 跑完加回已分配：跑完一个，就把它的已分配按项加回工作向量，工作向量变大后再比下一个。
- 遇到都能跑就分叉：每条叉都独立走到头，最后把走通到底的叉数加起来。

【类比】

先想银行放贷的场景：银行手头的现金就是工作向量，客户做完生意还差的钱就是尚需，客户已经贷走的钱就是已分配。银行先找一个还差得少、手头现金够付的客户，让他做完生意把贷走的钱还回来，钱袋变鼓了，再看接下来谁够格。分叉就像两条都能走的还款顺序：先让甲还再让乙还也行，先让乙还再让甲还也行，两条顺序都要试一遍，才知道一共有几条活路。

【口诀】

Work 起步比尚需，跑完加回再分叉。

第 27 题 · CPU 模式：用户态与内核态（正确答案：C，用户态只能执行非特权指令）

题干与选项：

下列关于 CPU 模式的叙述中，正确的是（）。

- A. CPU 处于用户态时只能执行特权指令
- B. CPU 处于内核态时只能执行特权指令
- C. CPU 处于用户态时只能执行非特权指令
- D. CPU 处于内核态时只能执行非特权指令

入手点： 用户态是"只能"，内核态是"都能"；见到给内核态配"只能"直接排除。

题干专有名词从零解释：

- 用户态 / 目态：应用程序跑的访客模式。
- 内核态 / 管态：操作系统内核跑的管理员模式。
- 特权指令：一旦用错会危害整个系统的指令——开关中断、置时钟、修改页表基址、I/O 指令等。
- 非特权指令：加减乘除、读写自己内存、跳转这类人畜无害的指令。
- 系统调用：用户程序请求内核服务的入口，陷入内核态的正规通道。

考查知识点： 用户态与内核态的指令执行权限。

正确选项为什么对： 用户态只能执行非特权指令。用户程序跑在用户态，私自执行特权指令时硬件会当场拦下并触发异常——这就是"只能"二字的含义。

错误选项为什么错＋举例：

- A（把用户态的权限说反了）：用户态恰恰碰不了特权指令。举例：用户程序想直接关中断、改页表基址，硬件不答应——想干这些事只能走系统调用请内核代劳。
- B（内核态只能执行特权指令）：错在"只能"。举例：内核也要跑普通计算（加减、查表、搬数据），它是什么都能执行，只是"特权指令只有它能执行"——"只能执行特权指令"等于说内核不能做普通算术，荒谬。
- D（给内核态配了"只能"）：错在"只能"。举例：内核态是什么都能执行——特权指令只有它能执行，但普通算术照样跑；D 说内核态"只能"执行非特权指令，等于说内核碰不了特权指令，正好说反了。内核处理系统调用时，加减、比较、跳转这些非特权指令天天在跑，特权指令也一样在跑，两样都能，才叫内核态。

笔记本一句话： 第 27 题＋用户态"只能"非特权，内核态"都能"执行；见到给内核态配"只能"直接排除。

【图像】

用户态 vs 内核态：只能 vs 都能

用户态（访客模式）	内核态（管理员模式）
非特权指令：能	非特权指令：能
特权指令：不能	特权指令：能
只能做普通的事	普通和高级都能做

关键：高级指令只有内核态能，不等于内核态只能干高级的。

用户态与内核态权限对照图，用户态只能执行非特权指令，内核态两种指令都能执行

左边是访客模式，右边是管理员模式。左边那栏里，普通指令那行标着“能”，高级指令那行标着“不能”；右边那栏里，普通指令和高级指令两行都标着“能”。看图记住一句：访客是“只能”，管理员是“都能”，高级指令只是“只有管理员能”，不是“管理员只能干高级的”。

【定位】操作系统 · 第 1 章 · 操作系统概述 · 处理器运行模式 · 用户态与内核态

【易混对比】

- 访客模式：应用程序跑的模式，叫用户态，只能做普通的事。
- 管理员模式：操作系统内核跑的模式，叫内核态，普通的事和高级的事都能做。
- 高级指令：一旦用错会危害整个系统的操作，比如开关中断、修改页表基址、直接碰设备，这就是特权指令，只有管理员模式能执行。
- 普通指令：加减乘除、读写自己内存、跳转这类不会危害整个系统的操作，这就是非特权指令，两种模式都能执行。
- “只能”与“都能”：只能是范围的上限，超出就不行；都能是全覆盖，两样都行。给管理员模式配“只能”，就是把全覆盖说成了上限，一定是错的。

【坑位】

最核心的陷阱，是给内核态配“只能”。看到“内核态只能执行……”就觉得管理员就该只干高级活，这个感觉听起来顺，可管理员也要做普通算术、查表、搬数据，处理一次服务请求的过程中普通指令天天在跑。内核态的正确说法是“都能”：高级指令只有它能执行，不等于它只能执行高级指令。判断时先看主语，再看“只能”还是“都能”。

【套路】

碰到问两种模式权限的题，按两步走：

- 先看主语是访客还是管理员：访客对应用户态，管理员对应内核态。
- 再看“只能”还是“都能”：访客只能做普通的事；管理员普通的事和高级的事都能做。凡是给管理员配“只能”的说法，先排除。

【类比】

先想小区的两种钥匙：普通住户（用户态）只能开自己家门、用自家水电，开不了配电房；物业管理员（内核态）既能开自己办公室的普通门、算账，也能开配电房的总闸。总闸只有管理员能开，不等于管理员只能开总闸，他日常也要开普通门、做普通的事。用户态和内核态的区别就在这里：一个是只能做普通的事，一个是两样都能做。

【口诀】

用户只能非特权，内核两样都能干。

第 28 题 · 什么事件让进程从执行态变阻塞态（正确答案：D，仅 I、III、IV）

题干与选项：

下列事件中，可能使进程 P 由执行态变为阻塞态的是（ ）。

I. 进程 P 读文件

II. 时间片用完

III. 进程 P 申请外设

IV. 进程 P 执行信号量的 wait() 操作

- A. 仅 I、IV
- B. 仅 II、III
- C. 仅 III、IV
- D. 仅 I、III、IV

入手点： 变阻塞 = 主动等事件；变就绪 = 被抢走 CPU。

题干专有名词从零解释：

- 执行态：进程正在 CPU 上跑。
- 就绪态：进程只差一个 CPU，随时能跑。
- 阻塞态：进程在等某件事，等的时候占着 CPU 也没用，睡过去。
- I/O 请求：向磁盘等设备要数据。
- 时间片：分给进程的一段 CPU 使用时间，用完就轮换。
- 信号量：一个计数器，表示某类可用资源还剩多少；wait() / P 操作把这个计数器减 1，减到负数时进程主动睡到信号量队列里等唤醒。
- 外设：打印机、磁盘等设备。

考查知识点： 进程三状态转换的判别。

正确选项为什么对： I 读文件 → 发起磁盘 I/O，数据没回来之前只能睡等 → 阻塞。III 申请外设 → 设备正被别人用（或需要等待分配）→ 阻塞。IV wait() → 信号量值不够（资源已被占完），进程主动睡到信号量队列里 → 阻塞。II 时间片用完 → 被时钟中断赶下台、回就绪队列，是变就绪不是阻塞。

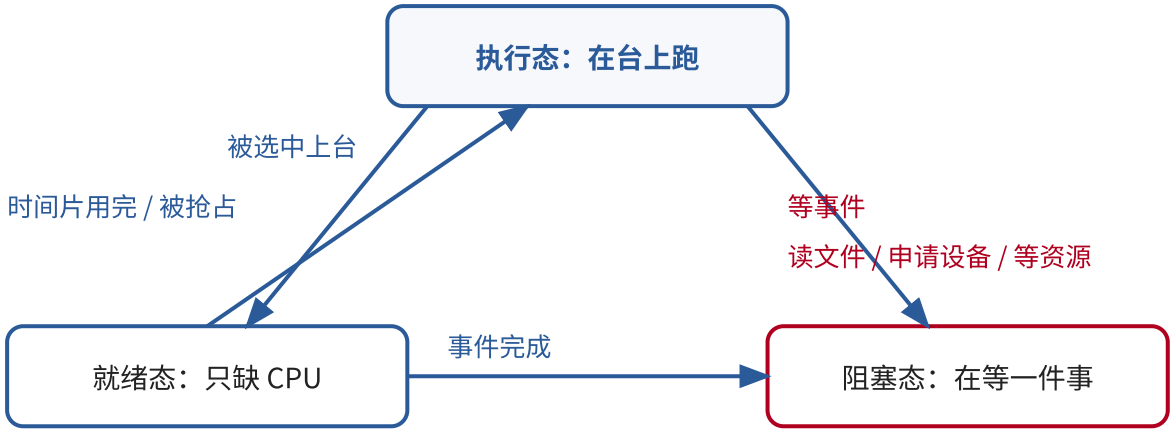
错误选项为什么错＋举例：

- A（仅 I、IV）：漏了 III。举例：进程申请打印机，打印机正被别的进程用着，申请者只能睡等——和读文件等数据一个道理，都是主动等事件。
- B（仅 II、III）：II 是就绪不是阻塞，正好选反了主角。举例：时间片用完的进程什么都不缺、只缺 CPU，轮到它立刻就能接着跑——这叫就绪，不叫阻塞。
- C（仅 III、IV）：漏了 I。举例：读文件发起磁盘 I/O，磁盘比 CPU 慢几个数量级，数据没回来之前进程只能睡等——读文件就是阻塞的典型。

笔记本一句话： 第 28 题＋变阻塞 = 主动等事件（I/O、设备、资源）；变就绪 = 被抢走 CPU（时间片、抢占）。

【图像】

进程三状态：变阻塞 vs 变就绪是两条线



进程三状态转换图，变阻塞与变就绪是两条不同的线

图里有三个框：在台上跑、只差处理器就能跑、在等一件事。从“在台上跑”出发有两条线：一条指向“在等一件事”，线上写着等事件，比如读文件、申请设备、资源不够时做的等待操作；另一条指向“只差处理器就能跑”，线上写着时间片用完、被抢走处理器。两条线的终点不一样，别把“下台”都画成同一条线。另外还有两条回路：等的那件事完成了，就回到“只差处理器就能跑”；轮到它了，就回到“在台上跑”。

【定位】操作系统 · 第 2 章 · 进程管理 · 进程状态与状态转换

【易混对比】

- 在等一件事：进程在等数据回来、等设备空出来、等资源够用，等的时候给它处理器也没用，它只能睡过去，这就是阻塞态。
- 只差处理器：进程什么都不缺，只缺处理器，轮到它立刻就能接着跑，这就是就绪态。
- 自己主动等与被动下台：自己发起等待，比如发起读文件、申请设备，是主动等，变阻塞；被时钟赶下台、被更高优先级的进程抢走处理器，是被动下台，变就绪。
- 区分点：判断时问一句“它在等什么具体的事？”能答出具体事件的，是阻塞；答不上来、只缺处理器的，是就绪。

【坑位】

最核心的陷阱，是把“下台”都当成阻塞。时间片用完也是下台，可那个进程什么都不缺、只缺处理器，轮到它立刻能跑，所以它回的是就绪队，不是阻塞队。阻塞的关键不是“不在台上”，而是“在等一件事”。看到一个事件先别急着选阻塞，先问：它是自己在等事件，还是被抢走了处理器？

【套路】

碰到判断状态转换的题，按两问走：

- 第一问：是自己在等事件吗？读文件等数据、申请设备等分配、资源不够时做的等待操作，都是自己在等事件，变阻塞。
- 第二问：是被抢走处理器吗？时间片用完、被更高优先级的进程抢占，都是被抢走处理器，变就绪。
- 两问的答案不会同时成立，事件属于哪一问，状态就往哪边变。

【类比】

先想饭馆里的两桌客人：一桌点了菜等厨房上菜，菜没好，给他一张空桌子也没用，他只能坐着等，这就是阻塞；另一桌是用餐时间到了被请到等位区，菜和座位都不缺，只等一张空桌子，空桌一出立刻能回去吃，这就是就绪。时间片用完像用餐时间到被请出，不是等菜，所以它变的是就绪，不是阻塞。

【口诀】

等事件变阻塞，丢处理器变就绪。

第 29 题 · 缺页处理：哪一步"不一定"发生（正确答案：A，淘汰内存中的页）

题干与选项：

某进程访问的页 b 不在内存中，引起缺页异常。在缺页处理过程中，不一定包含的操作是（）。

- A. 淘汰内存中的页
- B. 建立页号与页框号的对应关系
- C. 将页 b 从外存读入内存
- D. 修改页表中页 b 的存在位

入手点： 缺页必做三件：调入、登记映射、置存在位；淘汰只在没空框时才发生。

题干专有名词从零解释：

- 缺页异常：访问的页不在内存时 CPU 触发的异常，转入 OS 处理。
- 页框：主存里放一页的格子。
- 外存：磁盘，页的老家。
- 页表：虚页号 → 页框号的对照表。
- 存在位：页表里的标记，1 表示页已在内存。
- 淘汰 / 置换：页框全满时按算法挑一页牺牲、腾出格子。
- 脏页：被改过、淘汰时要先写回外存的页。
- 空闲页框：还没装页的格子。

考查知识点： 缺页处理流程中哪些步骤是必做的。

正确选项为什么对（本题找的是"不一定"项）： 淘汰是"不一定"发生的。读页进来需要一个空页框：如果内存里正好有空闲页框，直接用，根本不用淘汰任何页；只有页框全满时，才需要挑一页牺牲品淘汰（若是脏页还要先写回外存）。举例：书不在阅览架上，管理员一定会去库房把书取来、登记好位置；但"先扔掉一本别的书腾位子"只在架子满了时才需要。

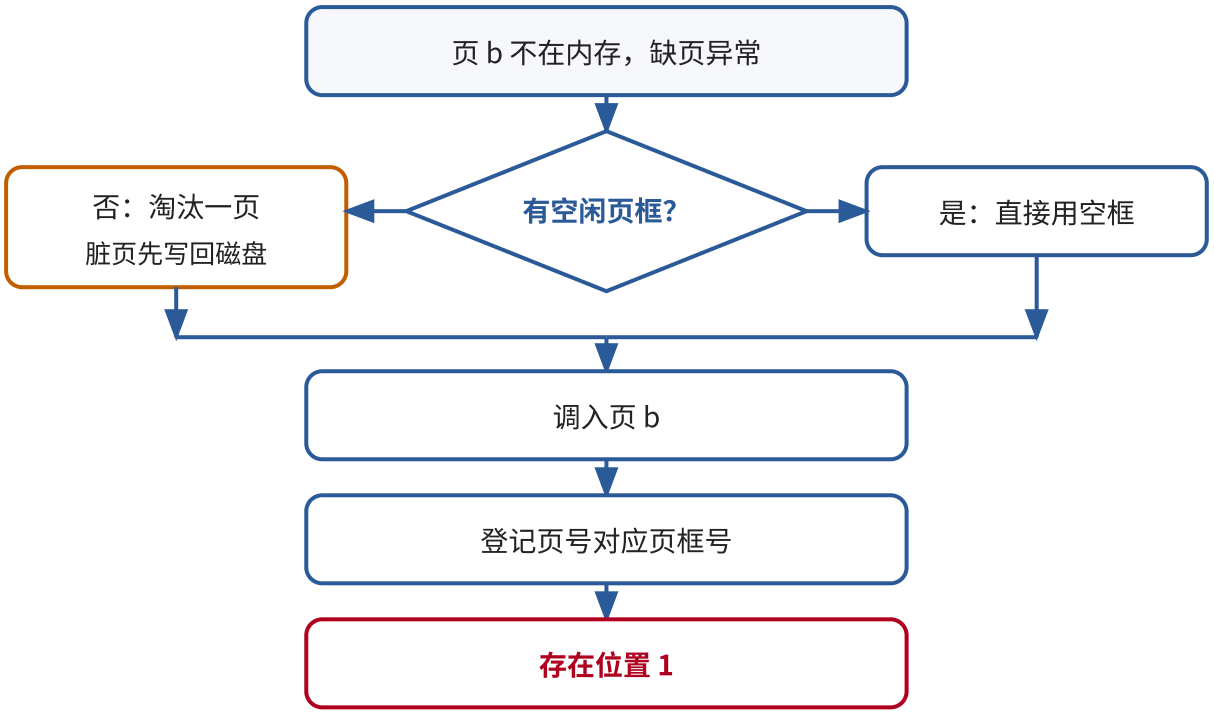
错误选项为什么错＋举例（其余三项都是必做的）：

- B（建立页号与页框号的对应关系）：必做。举例：页 b 调进 5 号页框却不在页表里登记，下次访问页 b 还是找不到，等于白调——不登记，这次调入等于白干。
- C（将页 b 从外存读入内存）：必做。举例：页 b 的老家在磁盘上，不把它读进来，CPU 访问的还是空气——页就是靠这一步进内存的。
- D（修改页表中页 b 的存在位）：必做。举例：存在位不置 1，下次访问页 b 时硬件一看存在位还是 0，会再报一次缺页，死循环。

笔记本一句话： 第 29 题＋缺页必做三件：调入、登记映射、置存在位；淘汰只在没空框时才发生。

【图像】

缺页处理：先问有没有空闲页框



缺页处理流程图，先问有没有空闲页框，再决定要不要淘汰

流程从最上面“要访问的页不在内存”开始，先走到一个菱形判断：内存里有没有空闲的格子？有，就直接用空格子；没有，才走淘汰那条分支，挑一页请出去，如果那页被改过还要先写回磁盘。两条分支汇合后都要走同样的三步：把要的页从磁盘读进来、在对照表里登记页号对应哪个格子、把在内存的标记改成“在”。图里淘汰只出现在“没有空格子”那条分支上，不在主流程里。

【定位】操作系统 · 第 3 章 · 内存管理 · 虚拟内存 · 缺页处理

【易混对比】

- 把要的页请进来：要访问的页老家在磁盘上，处理缺页一定要把它读进内存，这就是调入，每次缺页都做。
- 把一页请出去腾格子：内存格子全满时，按规则挑一页牺牲、腾出格子，这就是淘汰，只在没有空格子时才做。
- 登记与改标记：对照表里登记“这个页号住在哪个格子”，叫建立映射；把对照表里的在内存标记改成“在”，叫修改存在位。这两步每次缺页都做，漏了任何一步，下次访问还是找不到页，或者还会再报一次缺页。
- 区分点：调入、登记、改标记是每次都走的主流程；淘汰是“格子满了才走”的条件分支。

【坑位】

最核心的陷阱，是把缺页处理想成一套固定流程、每一步每次都走。题目里的“不一定”三个字，就是在问流程里有没有条件分支：有空闲格子就不用淘汰，淘汰只是“满了才做”的分支。看到“不一定”先找分支，别把分支上的步骤当成每次必做的步骤。

【套路】

碰到问缺页处理“必做还是不一定”的题，按两步走：

- 先背必做三件：把要的页读进来、在对照表里登记页号对应哪个格子、把在内存的标记改成“在”。
- 再单问一句：内存里有空闲格子吗？有，就不用淘汰；没有，才加淘汰，被改过的页淘汰前先写回磁盘。

【类比】

先想图书馆的阅览架：要看的书不在阅览架上，管理员一定会去库房把书取来、登记好它放在哪个架位、把架位牌改成“在架”，这三件每次都要做。只有架子满了，才需要先撤下一本旧书腾位子，架子有空位时根本不用撤书。淘汰就是那个“架子满了才撤书”的动作，不是每次取书都要做的动作。

【口诀】

调入登记置存在，空框在手不淘汰。

第 30 题 · 什么不影响缺页率（正确答案：D，页缓冲队列的长度）

题干与选项：

下列选项中，不会影响系统缺页率的是（ ）。

- A. 页面置换算法
- B. 工作集的大小
- C. 进程的数量
- D. 页缓冲队列的长度

入手点： 缺页率看"页在不在"：算法、工作集、人均页框都影响；页缓冲只省找回时间，不减缺页次数。

题干专有名词从零解释：

- 缺页率：缺页次数 ÷ 访问总次数。
- 页面置换算法：页框满时挑谁淘汰的规则，如 LRU、FIFO。
- 工作集：进程最近频繁访问的那批页。
- 页缓冲 / 缓冲队列：被淘汰的页先不真丢、挂进队列暂存的"后悔药"区。
- 缺页处理代价：一次缺页处理要花的时间 / 磁盘 I/O。

考查知识点： 影响缺页率的因素辨析（率 vs 代价）。

正确选项为什么对（本题找的是"不影响"项）： 页缓冲只降低单次缺页的找回代价，不改变缺页发生的次数。被淘汰的页挂进缓冲队列，若进程马上又访问它，可以直接从队列里捡回来、免去一次磁盘 I/O——但这次访问在统计上仍然算一次缺页。题目问的是率，D 只影响代价不影响次数。

错误选项为什么错＋举例（其余三项都会影响）：

- A（页面置换算法）：举例：LRU 赶走最久没用的页，FIFO 赶走最早进来的页——算法差，好页被误赶走，回头访问又缺，缺页率直接上升。
- B（工作集的大小）：举例：进程工作集是 50 页，只分给它 30 页框，工作集装不进，每跑几步就缺一页；分够 50 页框，缺页率立刻下来。
- C（进程的数量）：举例：内存 100 页框，2 个进程每人分 50 页；硬塞 10 个进程，每人只剩 10 页框，人均页框越少，缺页率整体上升。

笔记本一句话： 第 30 题＋缺页率看"页在不在"：算法、工作集、人均页框都影响；页缓冲只省找回时间，不减缺页次数。

【图像】



缺页率与缺页代价的对照图，改页在不在的因素影响缺页率，页缓冲只改找回速度

图中间是缺页率的算法：缺页次数除以访问总次数。左边三个因素改的是页在不在，所以影响缺页率；右边页缓冲队列改的是找回多快，只省单次找回的时间，不改变缺页次数，所以不影响缺页率。看图先分清每个因素改的是次数还是速度。

【定位】操作系统 · 第 3 章 · 内存管理 · 虚拟内存 · 缺页率的影响因素

【易混对比】

- 缺了几次：缺页次数除以访问总次数，只看缺了几次，这就是缺页率。
- 一次缺页多贵：处理一次缺页要花的时间和磁盘读写，看一次多贵，这就是缺页代价。
- 后悔药柜台：被淘汰的页先不真丢、挂进一个队列里暂存，马上又要用到它时可以直接从队列里捡回来、不用跑磁盘，这就是页缓冲队列。它省的是单次找回的时间，不是缺页的次数。
- 常翻的那叠页：进程最近频繁访问的那批页，叫工作集，工作集的大小决定了要分多少格子才够装；格子不够，缺页次数就上去。工作集改的是“页在不在”，页缓冲改的是“找回多快”，一个影响率，一个只影响代价。

【坑位】

最核心的陷阱，是把“捡回来就等于没缺过”。从缓冲队列里捡回来的页确实不用跑磁盘，看起来像没缺过，可统计上那次访问发生时页本来就不在内存，缺页已经记了一次。题目问的是率，也就是次数，不是代价，也就是时间。把“省时间”当成“省次数”，就会把只影响代价的选项误当成影响率的选项。

【套路】

碰到问“什么影响缺页率”的题，按两步走：

- 先看题目问的是率还是代价：问率只数次数，问代价才算时间。题目问率，就别拿省时间的选项来凑数。
- 再看每个选项改的是什么：改“页在不在”的会影响率，比如换页规则、常翻页叠的大小、人均分到的格子数；只改“找回多快”的不影响率，比如页缓冲队列的长度。

【类比】

先想上课点名：人不在教室就是缺席一次，哪怕他后来从走廊三分钟跑回来上课，缺席记录已经记下了。走廊离教室近，只让他回来得快，不会把缺席次数抹掉。页缓冲队列就是那条走廊：被淘汰的页暂存在走廊里，捡回来

省的是跑磁盘的时间，缺页那一次已经记在账上了，次数不会因此减少。

【口诀】

缓冲省时不省次，率看页在不在。

第 31 题 · 系统调用中哪些由操作系统完成（正确答案：B，仅 II、III）

题干与选项：

执行系统调用时，涉及下列操作，其中由操作系统完成的是（ ）。

I. 保存断点和程序状态字

II. 保存通用寄存器的内容

III. 执行系统调用服务程序

IV. 将 CPU 模式改为内核态

- A. 仅 I、III
- B. 仅 II、III
- C. 仅 II、IV
- D. 仅 II、III、IV

入手点： 陷入瞬间自动发生的归硬件；内核代码一步步执行的归 OS。

题干专有名词从零解释：

- 系统调用：用户程序请求内核服务的入口，如 read、write。
- 陷阱指令：用户程序执行系统调用时用的特殊指令，一执行就陷入内核。
- 断点：被中断时下一条要执行的指令地址，即 PC。
- 程序状态字 PSW：记录 CPU 当前状态的寄存器——模式位、条件码等。
- 通用寄存器：程序员能直接用的数据寄存器。
- 内核栈 / 进程控制块：内核保存现场的地方。
- 系统调用服务程序：内核里真正干活的函数，如读文件的实现。

考查知识点： 系统调用过程中硬件与 OS 的分工。

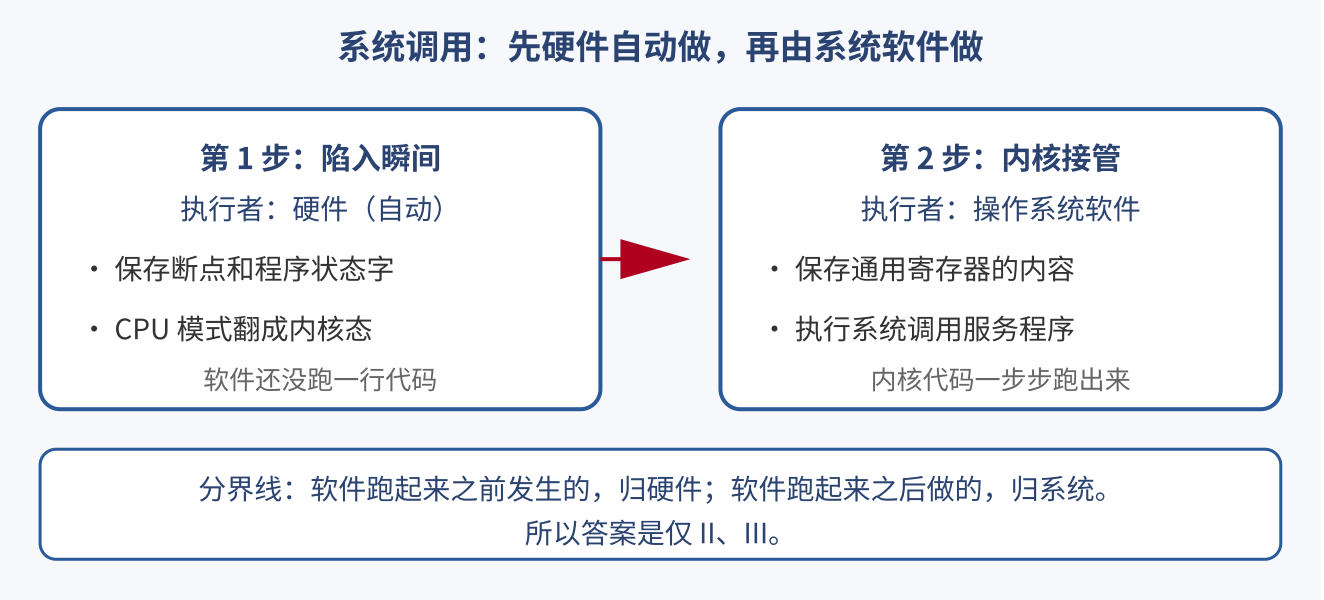
正确选项为什么对： 用户程序执行陷阱指令的那一瞬间，硬件自动完成两件事：把断点（PC）和 PSW 压栈保存（I），并把 CPU 状态位翻成内核态（IV）——这两步不需要软件插手。等内核接管之后，OS 软件才登场：由中断 / 系统调用处理程序把通用寄存器等现场保存到内核栈或进程控制块（II），再按系统调用号去执行对应的服务程序（III）。

错误选项为什么错＋举例：

- A（仅 I、III）：I 是硬件做的。举例：陷阱指令一执行，硬件在软件还没跑一行代码之前就已经把 PC 和 PSW 存好了——I 发生在"陷入瞬间"，执行者是硬件。
- C（仅 II、IV）：IV 也是硬件做的。举例：模式位是 CPU 状态寄存器里的一位，陷阱指令执行时硬件自动把它翻成内核态，不需要 OS 写代码去翻。
- D（仅 II、III、IV）：多带了硬件做的 IV。举例：同 C——切模式是硬件在陷入瞬间的自动动作，OS 接管时模式已经是内核态了。

笔记本一句话： 第 31 题＋陷入瞬间：存断点、切内核态是硬件自动的；存寄存器、跑服务程序才是 OS 干的。

【图像】



陷入瞬间硬件做、内核接管后系统做，两段流程图

【定位】操作系统 · 第 1 章 · 系统调用中硬件与操作系统的分工

【易混对比】

- 陷入瞬间自动做的事 vs 内核接管后一步步做的事：前者是硬件在陷阱指令执行的那一刻自动完成的，不用等任何软件跑起来；后者是操作系统软件接管之后用代码一步步执行的。
- 保存断点和程序状态字 vs 保存通用寄存器：前者发生在陷入瞬间、由硬件做；后者发生在内核接管之后、由操作系统做。两处保存的时机和执行者都不同，不能混成一件事。

- 改 CPU 模式 vs 执行服务程序：改模式是陷入瞬间的硬件自动动作，模式改好之后操作系统才登场；执行服务程序是操作系统登场之后才干的活，顺序不能颠倒。

【坑位】

最容易栽的地方，是把“改 CPU 模式”算到操作系统头上。改模式听起来像一句软件命令，让人觉得是操作系统写代码改的。但真实顺序是：用户程序一执行陷阱指令，硬件立刻把模式位翻成内核态，操作系统接管的时候模式已经是内核态了。记住分界线就能避开这个坑：软件还没跑一行代码之前发生的事，执行者是硬件；软件跑起来之后做的事，执行者才是操作系统。

【套路】

判断“这一步是谁做的”，固定走两步。第一步，问时机：是不是陷阱指令执行的那一瞬间自动发生的？是，就归硬件。第二步，问执行方式：是不是内核代码一步步跑出来的？是，就归操作系统。拿本题套一遍：保存断点和程序状态字、改 CPU 模式，都是陷入瞬间自动发生的，归硬件；保存通用寄存器、执行系统调用服务程序，都是内核代码跑出来的，归操作系统。

【类比】

想象医院急诊交接。病人被推进抢救室大门的那一刻，门禁自动弹开、灯光自动亮起，这就像陷入瞬间硬件自动做的事，不用医生下命令。医生进场之后，一项一项查体征、开处方、动手治疗，这就像内核接管后操作系统一步步做的事。自动弹开的门不能算成医生做的，医生进场后的治疗也不能算成门禁做的，分工就按“进场前自动发生”还是“进场后动手做”来分。

【口诀】瞬间硬件做，接管系统做。

第 32 题 · 驱动程序与 I/O 控制方式（正确答案：A，不正确）

题干与选项：

下列关于驱动程序的叙述中，不正确的是（）。

- A. 驱动程序与 I/O 控制方式无关
- B. 设备的初始化由驱动程序控制完成
- C. 进程在驱动程序中等待设备完成操作时，可能会进入阻塞态
- D. 读 / 写设备的具体操作由驱动程序操纵设备控制器完成

入手点： 驱动是最贴硬件的一层：初始化、发命令、等完成都归它，而且控制方式变、驱动就变。

题干专有名词从零解释：

- 驱动程序：OS 里最贴近硬件的一层软件，把上层命令翻译成对设备控制器的寄存器操作。
- 设备控制器：设备上的硬件接口，驱动通过读写它的寄存器操纵设备。
- I/O 控制方式：程序查询、中断、DMA、通道四种。
- 程序查询：驱动反复轮询状态位等设备就绪。
- 中断方式：驱动设置好并等待设备中断。
- DMA 方式：驱动把缓冲区地址、字节数写进 DMA 控制器、由 DMA 成批搬运。
- 设备独立性软件：追求"与设备无关"的上层接口，不是驱动。

考查知识点： 驱动程序的职责及其与 I/O 控制方式的关系。

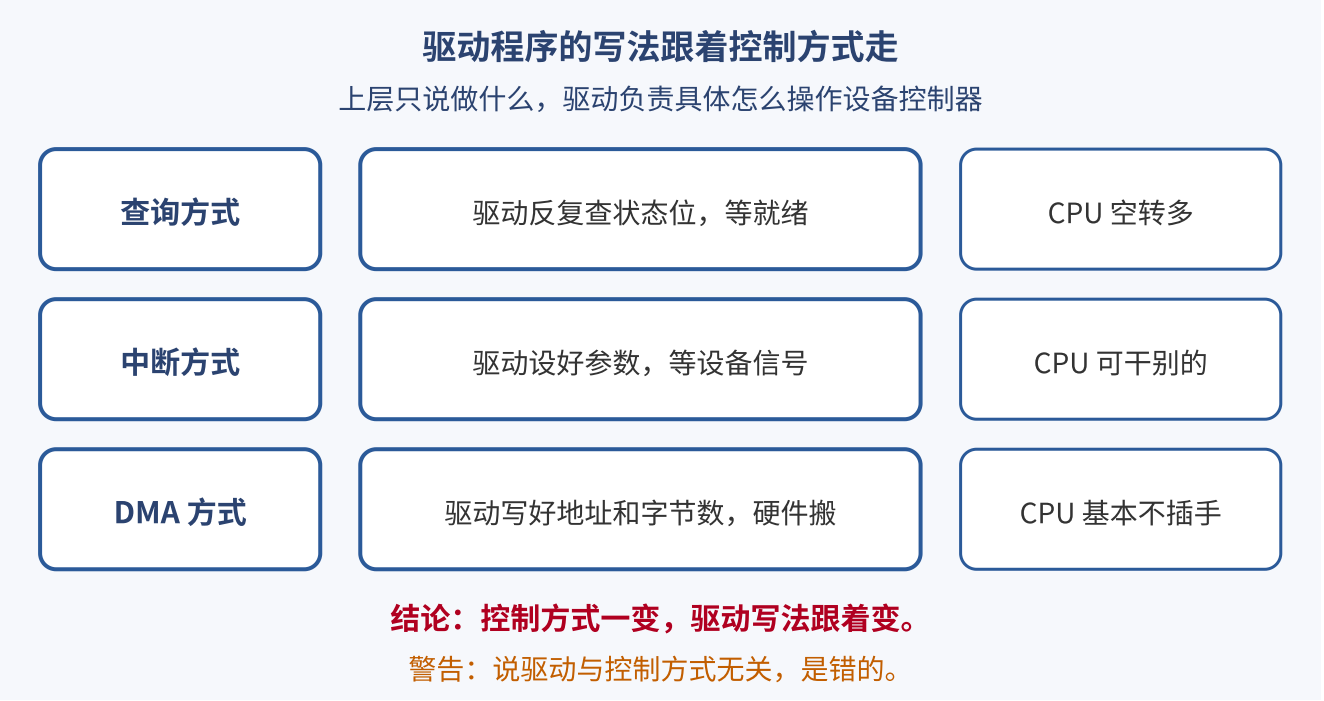
正确选项为什么对（本题找的是错述）： A 说"驱动程序与 I/O 控制方式无关"，是彻头彻尾的错话。控制方式变了，驱动要写的代码就完全不同：查询方式下驱动要反复轮询状态位；中断方式下驱动要设置好并等待中断；DMA 方式下驱动要先把缓冲区地址、字节数写进 DMA 控制器。举例：司机怎么开车，取决于开的是手动挡、自动挡还是电动车——说"驾驶方式与车型无关"是错的。

错误选项为什么错＋举例（其余三项都是对的）：

- B（设备的初始化由驱动控制）：举例：设备上电后清状态、设寄存器、备缓冲——这些初始化动作正是驱动发命令完成的。
- C（进程执行到驱动里等设备时可能阻塞）：举例：进程读磁盘，驱动发完命令后设备要几毫秒才备好数据，进程只能睡过去等完成中断——在驱动里等设备就是阻塞。
- D（读 / 写设备的具体操作由驱动操纵控制器完成）：举例：上层只说"读这块盘"，真正往控制器寄存器里写命令、启动传输的，就是驱动——这就是驱动"实际动手操作设备"的那一层。

笔记本一句话： 第 32 题＋驱动是最贴硬件的一层：初始化、发命令、等完成都归它，而且控制方式变、驱动就变。

【图像】



三种输入输出控制方式下驱动程序的不同写法对照图

【定位】操作系统 · 第 5 章 · 驱动程序的职责及其与输入输出控制方式的关系

【易混对比】

- 驱动程序 vs 设备独立性软件：驱动程序是最贴近硬件的一层软件，负责把上层命令翻译成对设备控制器的具体寄存器操作；设备独立性软件是更靠上的一层，追求的是让上层不用管具体是哪种设备。说“驱动与控制方式无关”，等于把最贴硬件的一层说成了与硬件无关的一层，方向正好反了。
- 四种输入输出控制方式下的驱动写法：用反复轮询等设备就绪的办法时，驱动要一遍遍查状态位；用设备发信号通知的办法时，驱动要先设置好、再等信号；用硬件成批搬运的办法时，驱动要先把缓冲区地址和字节数写进搬运控制器。控制方式一换，驱动要写的代码跟着全换。
- 驱动里等待 vs 上层等待：进程在驱动程序里等设备备好数据时可能进入阻塞态，也就是先睡过去、等设备完成信号到了再醒来继续。这不是驱动卡住了系统，而是等慢设备的正常办法。

【坑位】

最核心的陷阱在“无关”两个字上。题目问的是“不正确”的说法，选项说驱动程序与输入输出控制方式无关。乍一看，驱动程序好像只是翻译命令的中间层，跟用哪种控制方式没关系。但控制方式恰恰决定了驱动怎么写代码：轮询、中断、硬件搬运，每种方式下驱动的写法完全不同。所以“无关”这个说法错得彻底，看到“无关”“一定”“都一样”这类绝对化的词，要先打个问号，再去核对它和硬件细节有没有关系。

【类比】

想象一个翻译兼司机。老板只说“去机场”，真正怎么开，取决于开的是手动挡、自动挡还是自动驾驶的车：手动挡要自己换挡踩离合，自动挡不用换挡，自动驾驶要先设好目的地再交给系统。司机就是驱动程序，车型就是输入输出控制方式。说“司机怎么开车与车型无关”，一听就知道是错话，因为开法本来就是跟着车型走的。

【口诀】驱动贴硬件，方式一变它就变。

第 33 题 · OSI 模型中哪一层管相邻结点的流量控制（正确答案：B，数据链路层）

题干与选项：

在 ISO/OSI 参考模型中，实现两个相邻结点之间流量控制功能的是哪一层（）。

- A. 物理层
- B. 数据链路层
- C. 网络层
- D. 传输层

入手点： 流量控制在两层都有，先看题干的范围词：说"相邻结点"就选链路层，说"端到端"才选传输层。

题干专有名词从零解释：

- ISO/OSI 参考模型：把网络功能从下到上切成 7 层的标准框架，从下到上依次是：物理层、数据链路层、网络层、传输层、会话层、表示层、应用层。本题只用到下面四层，各管一摊事：物理层只管传原始比特（电压高低、接口形状），不管快慢协调；数据链路层管相邻两个结点之间这一段链路上的事（成帧、差错、流量控制）；网络层管选路和分组转发（数据走哪条路到目的网络）；传输层管端到端的进程间通信（从源主机到目的主机的全程）。
- 相邻结点：用一段链路直接连着的两个设备，比如电脑和它插着的交换机、交换机和路由器——中间没有第三个设备。
- 流量控制：给发送速度装刹车，让发方别发得太快、收方别被撑死。实现办法之一是滑动窗口：收方告诉发方"我还能收多少"，发方按这个数发。
- 端到端：从源头主机到目的主机的全程，中间隔着多少设备都不管，只看两头。

考查知识点： OSI 各层功能归属，流量控制在链路层与传输层的分工。

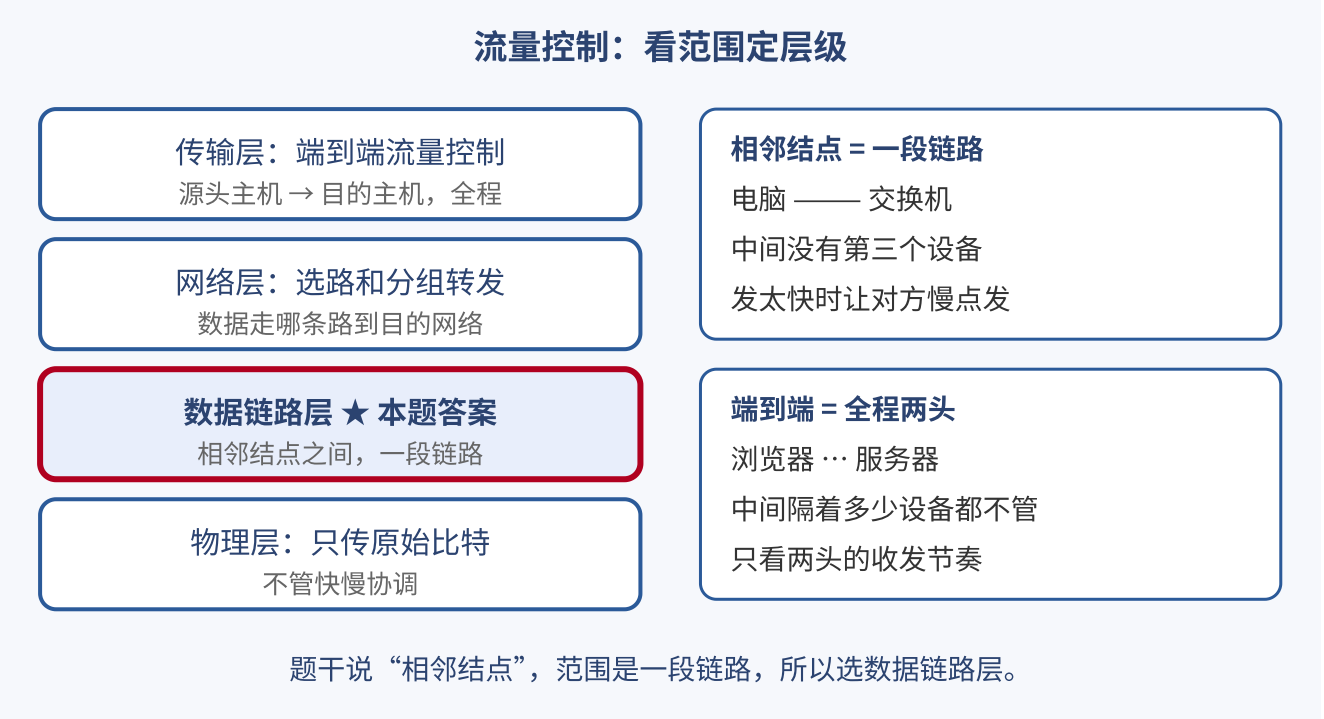
正确选项为什么对： 题干问的是"相邻结点之间"的流量控制，范围是点到点的一段链路，这正是数据链路层的地盘。举例：电脑往交换机发一串帧，交换机处理不过来时用滑动窗口让电脑慢点发——管的就是这一段链路，不涉及全程。

错误选项为什么错＋举例：

- A（物理层）：物理层只管把比特变成信号传出去。举例：网线传 0 和 1 时电压是高是低归物理层管，但"发太快了让对方慢点"这种协调它不管——它连"帧"的概念都没有。
- C（网络层）：网络层管选路和转发，它操心的是全网层面的拥塞控制（哪条路堵了换条路），不是某一段链路上两个结点的快慢协调。举例：路由器决定把分组往左还是往右转发，这是选路；至于它和邻居之间发多快，那是链路层的事。
- D（传输层）：传输层确实也做流量控制，但那是端到端的。举例：浏览器和服务器之间隔着十几个路由器，传输层管的是"服务器别发得太快、浏览器收不过来"，范围是全程；题干说的是"相邻结点"，范围对不上。

笔记本一句话： 第 33 题＋流量控制看范围：相邻结点选链路层，端到端选传输层。

【图像】



分层图中标出相邻结点一段链路与端到端全程的范围差别

【定位】计算机网络 · 第 3 章 · 流量控制在数据链路层与传输层的分工

【易混对比】

- 相邻结点之间 vs 端到端：相邻结点之间指的是一段链路直接连着的两个设备，中间没有第三个设备；端到端指的是从源头主机到目的主机的全程，中间隔着多少设备都不管，只看两头。范围词一变，负责的层就跟着变。
- 数据链路层的流量控制 vs 传输层的流量控制：两层都做“别发太快、别把对方撑死”这件事，差别只在范围。管一段链路的，归数据链路层；管全程两头的，归传输层。
- 流量控制 vs 拥塞控制：流量控制是收方怕被发方撑死，给发方装刹车；拥塞控制是整条路怕被堵死，调整全网的发送节奏。本题问的是相邻结点之间的快慢协调，属于流量控制，不是全网选路层面的拥塞控制。

【坑位】

最核心的陷阱，是看到“流量控制”四个字就直奔传输层。传输层确实也做流量控制，而且名气更大，所以很多人不看范围词就选它。但题干特意写了“相邻结点之间”，这个范围词就是定层级的钥匙：一段链路的事归数据链路层，全程两头的事才归传输层。做题时先圈出范围词再选层，就不会被“传输层也做流量控制”这个正确但不对题的知识带偏。

【套路】

判断流量控制归哪一层，固定看范围词。第一步，圈出题干里的范围词：写着“相邻结点”“点到点”“一段链路”，选数据链路层；写着“端到端”“源主机到目的主机”“进程之间”，选传输层。第二步，再核对一次：范围是“一小段”还是“全程”？一小段对应数据链路层，全程对应传输层。范围词和层级对上号，答案就定了。

【类比】

想象一条接力跑道。相邻两个接力点之间，上一棒跑太快、下一棒接不住，两个人之间要互相喊“慢点、稳点”，这就是相邻结点之间的流量控制，管的是这一小段。而从起点到终点的全程配速，是教练站在全局看的，管的是

整场节奏，这就是端到端的流量控制。一小段的配合不能交给看全场的教练，全场的配速也不能只靠两个相邻接力点商量，范围不同，负责人就不同。

【口诀】相邻找链路，全程找传输。

第 34 题 · 奈奎斯特公式求最大数据传输速率（正确答案：C，800 kb/s）

题干与选项：

带宽为 200 kHz 的无噪声信道，采用 4 个幅值的 ASK 调制，该信道的最大数据传输速率是（ ）。

- A. 200 kb/s
- B. 400 kb/s
- C. 800 kb/s
- D. 1600 kb/s

入手点： 先看题干有没有噪声：说"无噪声"就用奈奎斯特公式，再算"每个码元装几比特"，最后乘起来。

题干专有名词从零解释：

- 带宽 W：信道能通过的频率范围，本题 200 kHz = 200000 Hz（Hz 是每秒变化的次数）。
- 无噪声：信道里没有干扰，这是奈奎斯特公式的前提条件。
- ASK 调制：用信号的幅度大小表示信息，"4 个幅值"就是信号有 4 种不同的幅度状态。
- 码元：信号的一次取值，一个码元就是"信号在某一时刻的样子"；本题 4 个幅值 = 每个码元有 4 种可能。
- 码元速率：每秒传几个码元，单位是波特（Baud）。
- 数据传输速率：每秒传几个比特，单位是 b/s；1 kb/s = 1000 b/s。
- 奈奎斯特公式：无噪声信道的最大数据传输速率 = $2W \times \log_2 L$，其中 L 是信号的状态数（本题 L = 4），$\log_2 L$ 就是"一个码元能装几比特"。
- 香农公式：有噪声信道的最大速率 = $W \times \log_2(1 + S/N)$，要用到信噪比；本题没给信噪比，用不上。

考查知识点： 奈奎斯特公式与香农公式的分工，码元速率与比特率的换算。

正确选项为什么对： 分三步算。第一步：一个码元装几比特。4 种状态，$2^2 = 4$，所以 $\log_2 4 = 2$，即每个码元装 2 比特。第二步：码元速率上限。奈奎斯特说每秒最多变 2W 次信号，$2 \times 200000 = 400000$ 码元/秒。第三步：比特率 = 码元速率 × 每码元比特数 = $400000 \times 2 = 800000$ b/s = 800 kb/s。

错误选项为什么错＋举例：

- A（200 kb/s）：把带宽数值直接当速率。举例：200 kHz 是"每秒能通过的频率范围"，不是"每秒传的比特数"；按 A 的算法等于默认每秒变 1 次、每次 1 比特，两步都漏了。
- B（400 kb/s）：只算了码元速率 2W = 400000 码元/秒，忘了乘每码元的 2 比特。举例：码元速率是"每秒发几个信号"，比特率是"每秒发几个比特"，一个信号装 2 比特时，比特率是码元速率的 2 倍。
- D（1600 kb/s）：把"4 个幅值"当成"每码元 4 比特"。举例：4 种状态只需要 2 位二进制就能区分（00、01、10、11），所以是 2 比特不是 4 比特；按 4 比特算是把状态数 L 直接当成了比特数。

笔记本一句话： 第 34 题＋无噪声用奈奎斯特：速率 = $2W \times \log_2 L$，先算"每码元几比特"，别把电平数当比特数。

【图像】



奈奎斯特公式三步计算流程图

【定位】计算机网络 · 第 2 章 · 奈奎斯特公式与香农公式的分工

【易混对比】

- 没有噪声的信道 vs 有噪声的信道：没有噪声时，用只看带宽和信号状态数的公式；有噪声时，要用带上信噪比的公式。题干有没有给信噪比、说没说“无噪声”，就是选公式的第一信号。
- 奈奎斯特公式 vs 香农公式：前者算的是没有噪声时速率的天花板，等于两倍带宽乘以一个码元能装的比特数；后者算的是有噪声时速率的天花板，等于带宽乘以信噪比换算出来的对数。两个公式的前提不同，不能混着用。
- 码元速率 vs 数据传输速率：码元速率数的是每秒发几个信号，单位是波特；数据传输速率数的是每秒发几个比特，单位是比特每秒。一个信号能装几个比特，比特率就是码元速率的几倍。
- 信号状态数 vs 每码元比特数：信号有 4 种状态，意思是 4 种样子只需要 2 位二进制就能区分开，所以每码元装 2 比特，不是 4 比特。状态数和比特数之间隔着一个取对数的换算。

【坑位】

最核心的陷阱，是把“4 个幅值”直接当成“每码元 4 比特”。4 这个数字在题干里很显眼，顺手一乘就得出 1600 kb/s，看起来算得很快，实际上是把状态数当成了比特数。正确换算是先问一句：几种状态需要几位二进制才能区分？4 种状态对应 00、01、10、11，正好 2 位，所以每码元是 2 比特。凡是题干给“几种状态”“几个幅值”“几个电平”，都要先取对数换算成比特数，再往速率里乘。

【套路】

没有噪声的信道求最大速率，固定走三步。第一步，算每码元装几比特：信号状态数是多少，就求以 2 为底的对数，4 种状态得 2 比特。第二步，算码元速率上限：两倍带宽，本题是两倍 200 kHz，得 400000 码元每秒。第三步，两数相乘得比特率：400000 乘以 2，得 800000 比特每秒，换算成 800 kb/s。三步缺一步都会掉进错误选项。

【类比】

想象一个红绿灯路口。带宽好比路口每秒最多能变几次灯，信号状态数好比每次灯亮有几种颜色组合。先算每次灯亮能传达几种信息：4 种颜色组合用 2 位暗号就能区分。再算每秒变几次灯。最后把“每秒变几次”乘以“每次传几位暗号”，才是每秒真正传出去的信息量。只数变灯次数，漏了每次的信息量；把颜色种数当成暗号位数，又会把信息量算多一倍。

【口诀】无噪声，二倍带宽乘每码元比特。

第 35 题 · 由 IP 和掩码求网络地址（正确答案：B，183.80.64.0）

题干与选项：

某主机 IP 地址为 183.80.72.48，子网掩码为 255.255.192.0，该主机所在网络的网络地址是（）。

- A. 183.80.0.0
- B. 183.80.64.0
- C. 183.80.72.0
- D. 183.80.192.0

入手点： 网络地址 = IP 与掩码按位与；逐字节看掩码：255 的字节原样抄，0 的字节写 0，只有中间字节才真算。

题干专有名词从零解释：

- IP 地址：给主机编的 32 位地址，写成 4 段十进制、用点隔开，每段 0~255，本题是 183.80.72.48。
- 子网掩码：32 位、同样写成 4 段，用来区分"哪几位是网络号、哪几位是主机号"：掩码为 1 的位是网络号，为 0 的位是主机号。本题掩码 255.255.192.0。
- 网络号：地址里表示"属于哪个网络"的部分；主机号：地址里表示"网内哪台主机"的部分。
- 网络地址：主机号全清零后的地址，代表整个网络本身。
- 按位与（AND）：逐位比较两个二进制数，两位都是 1 结果才是 1，其余情况都是 0。
- 块大小：掩码某个字节是 M 时，这个字节被切成大小为 256 − M 的"块"，网络地址该字节必是块大小的整数倍。

考查知识点： 子网划分：IP 与掩码求网络地址。

正确选项为什么对： 逐字节算。第一字节：183 与 255 → 255 全是 1，原样抄，得 183。第二字节：80 与 255 → 原样抄，得 80。第四字节：48 与 0 → 0 全是 0，写 0。第三字节是唯一要真算的：72 = 01001000，192 = 11000000，逐位与：0∧1 = 0、1∧1 = 1、0∧0 = 0、0∧0 = 0、1∧0 = 0、0∧0 = 0、0∧0 = 0、0∧0 = 0 → 01000000 = 64。所以网络地址 = 183.80.64.0。块大小法验算：第三字节掩码 192 → 块大小 = 256 − 192 = 64，边界是 0、64、128、192；72 落在 64 开头的那一块（64~127），取块首 64，结果一致。

错误选项为什么错＋举例：

- A（183.80.0.0）：把第三字节整个抹成 0。举例：这相当于把掩码当成 255.255.0.0（/16）来算，过度清零——掩码第三字节是 192 不是 0，不能整字节抹掉。
- C（183.80.72.0）：只把最后一字节清零，第三字节没和掩码做与运算。举例：72 看着像网络的一部分就直接抄了，但掩码不是 255 的字节永远要过一遍与运算；72 ∧ 192 = 64，不是 72。
- D（183.80.192.0）：把掩码的字节 192 抄成了网络地址。举例：192 是掩码里的数，不是 IP 里的数；网络地址的每一字节都来自"IP 与掩码的与运算"，不是从掩码里抄一个数过来。

笔记本一句话： 第 35 题＋求网络地址就是 IP ∧ 掩码：255 抄、0 写 0、中间字节算与；速算用块大小法，边界取块首。

【图像】



逐字节按位与示意图，第三字节是唯一需要真算的一字节

【定位】计算机网络 · 第 4 章 · 子网划分中由地址和掩码求网络地址

【易混对比】

- 网络地址 vs 主机地址：网络地址是把主机号部分全部清零后得到的地址，代表整个网络本身；主机地址是网里某一台具体设备的地址，主机号部分不是全零。求网络地址就是做一次清零动作，不是把原地址照抄。
- 掩码字节是 255 vs 是 0 vs 是中间值：掩码字节是 255，意思是这一字节全是网络号，原地址字节照抄；是 0，意思是这一字节全是主机号，直接写 0；是中间值，意思是这一字节里一部分是网络号、一部分是主机号，必须逐位做与运算，不能照抄也不能整字节抹掉。
- 按位与运算 vs 直接抄数：网络地址的每一字节都来自地址字节和掩码字节逐位比较的结果，两位都是 1 才得 1，其余情况都得 0。把掩码里的数直接抄到答案里，等于跳过了这一步比较。

【坑位】

最核心的陷阱，是第三字节的 72 照抄不误。72 看着像网络的一部分，而且前两字节确实是照抄的，于是很多人顺手把 72 也照抄下来，得到 183.80.72.0。但掩码第三字节是 192 不是 255，说明这一字节不能照抄，必须逐位做与运算，72 和 192 做与运算得 64。记住一条：掩码不是 255 的字节，永远要过一遍与运算，眼睛看到的原数不能直接进答案。

【套路】

求网络地址固定走三步。第一步，逐字节看掩码：是 255 的字节，原地址字节照抄；是 0 的字节，写 0。第二步，只有中间值的字节才真算：把地址字节和掩码字节都写成 8 位二进制，逐位比较，两位都是 1 才得 1，再换回十进制。第三步，验算：用块大小法复核一次，块大小等于 256 减去掩码字节，网络地址该字节一定是块大小的整数倍。本题第三字节掩码 192，块大小是 64，72 落在 64 开头的一块，取块首 64，和与运算结果一致。

【类比】

想象一张带镂空花纹的盖板。掩码就是这张盖板：镂空的地方透出原地址的数字，盖住的地方只露出 0。255 的字节是整字节镂空，原数全透出来；0 的字节是整字节盖住，只剩 0；192 这种中间值的字节是半镂空，只透出原数的一部分，透出来的部分拼起来才是网络地址。求网络地址就是把盖板盖上去、抄下透出来的样子，而不是把盖板上的花纹本身抄下来。

【口诀】二五五照抄，零全清零，中间按位与。

第 36 题 · 看拓扑图定子网掩码与默认网关（正确答案：D，255.255.255.224，192.168.1.62）

题干与选项：

按图中网络拓扑，主机 H 的子网掩码与默认网关分别是多少（）。

（拓扑说明：已与原卷拓扑图在线预览核对：H 的 IP 为 192.168.1.60，与它直连的路由器接口是 192.168.1.62/27；192.168.1.1/30 在两路由器之间的链路上，不在 H 所在网段接口一侧。）

- A. 255.255.255.192，192.168.1.1
- B. 255.255.255.192，192.168.1.62
- C. 255.255.255.224，192.168.1.1
- D. 255.255.255.224，192.168.1.62

入手点： 掩码认"和 H 直连的那个路由器接口"的前缀，网关认"和 H 同子网的路由器接口"；.1 再顺眼，不在同子网就不是网关。

题干专有名词从零解释：

- 拓扑图：画出设备怎么连的图。本题拓扑细节已与原卷拓扑图在线预览核对：H 的 IP 为 192.168.1.60，与它直连的路由器接口是 192.168.1.62/27；192.168.1.1/30 在两路由器之间的链路上，不在 H 所在网段接口一侧。
- 子网掩码与前缀：/27 的意思是"前 27 位是网络号"，换算成掩码：前三个字节全是 1（255.255.255），第四字节前 3 位是 1、后 5 位是 0，即 11100000 = 128 + 64 + 32 = 224，所以 /27 对应 255.255.255.224。
- 默认网关：主机出本子网的"门"，必须是和主机同一个子网、且在路由器上的那个接口地址——主机要能直接把帧递到它手里。
- 同子网：两个地址落在同一个网络地址块里。判断办法用块大小法：掩码某字节是 M，块大小 = 256 − M，网络地址该字节必是块大小的整数倍。
- 路由器接口：路由器上接某段网的那个口，每个接口有自己的 IP 地址。

考查知识点： 子网划分的应用：由拓扑定掩码、定网关。

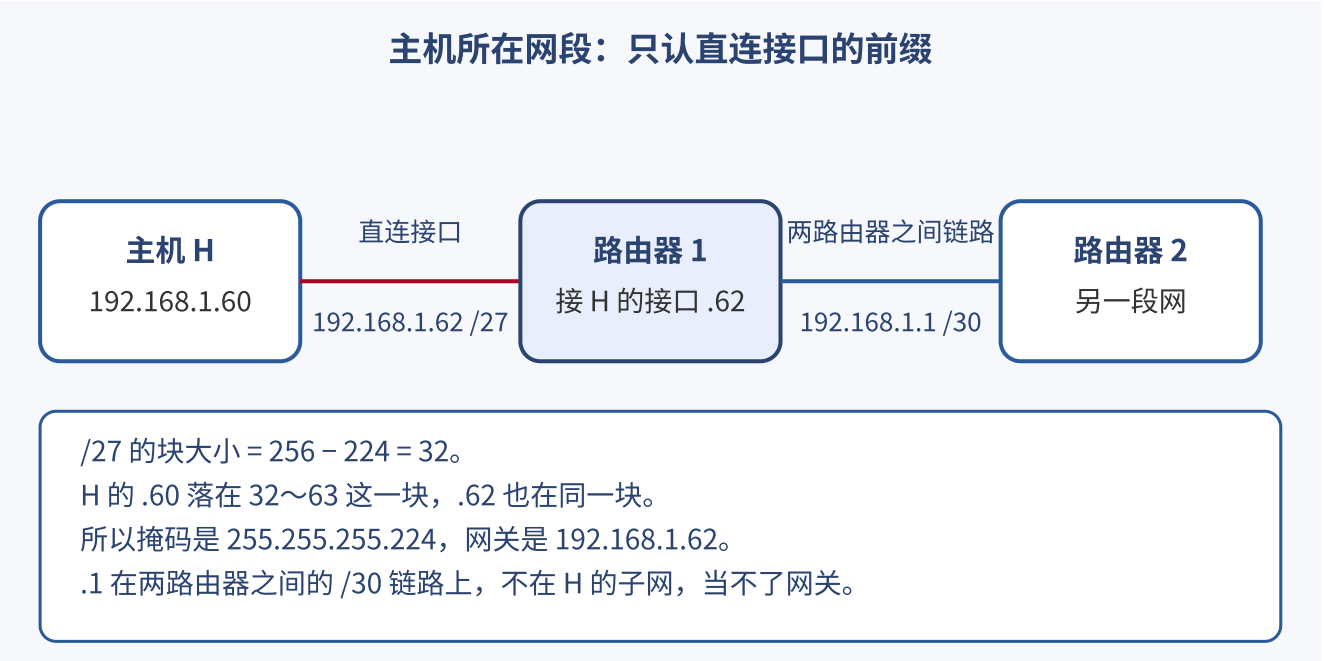
正确选项为什么对： 分两步。第一步定掩码：H 挂在路由器接口 192.168.1.62/27 下面，这段网的规矩就是 /27，所以 H 的掩码 = 255.255.255.224。第二步定网关：候选地址逐个验"是否与 H 同子网"。/27 的块大小 = 256 − 224 = 32，边界是 0、32、64、96……；H 的 .60 落在 32~63 这一块（网络地址 192.168.1.32），接口 .62 也落在同一块，同子网，可以当网关；.1 在两路由器之间的 /30 链路网段里，根本不在 H 的子网，H 连 ARP 广播都发不到它，当不了网关。所以是 D。

错误选项为什么错＋举例：

- A（255.255.255.192 + .1）：掩码抄了别的接口的 /26（255.255.255.192），网关又选了外网的 .1，双错。举例：/26 是另一段网的规矩（块大小 64），套到 H 这段网上，掩码和网关全错位。
- B（255.255.255.192 + .62）：网关 .62 对了，但掩码仍是 /26。举例：网关选对了只能说明"门找对了"，掩码错了说明"这段网的规矩认错了"，一题两问，错一问就不对。
- C（255.255.255.224 + .1）：掩码对了，网关 .1 不在 H 的子网内。举例：.1 看着像"默认网关的经典长相"，但网关不看顺眼不顺眼，只看在不在同一子网；.1 在两路由器之间的 /30 链路上，H 在 32~63 块，递不到。

笔记本一句话： 第 36 题＋掩码认直连接口的前缀，网关认同子网的路由器接口；.1 再顺眼，不在同子网就不是网关。

【图像】



按题干文字重画的布局：主机直连接口与两路由器之间链路分属不同网段

【定位】计算机网络 · 第 4 章 · 由拓扑定子网掩码与默认网关

【易混对比】

- 直连接口的前缀 vs 别的链路的前缀：主机所在网段的规矩，只认和主机直连的那个路由器接口的前缀；两路由器之间的链路有自己的前缀，管的是另一段网，不能套到主机头上。
- 默认网关 vs 看着像网关的地址：默认网关是主机出本子网要走的那扇“门”，必须是和主机在同一个子网、且在路由器上的接口地址。地址末尾是.1 只是常见习惯，不是当网关的资格，资格只看在不在同一子网。
- 掩码对了 vs 两问都对了：本题一题问两件事，掩码和网关各算一分。网关找对了，不代表掩码也对了；掩码对了，也不代表网关也对了。两问要分开验，错一问整题就不对。

【坑位】

最核心的陷阱，是.1 这个地址太顺眼了。很多人一看到 192.168.1.1，就默认它是网关，连子网都不验。但.1 在两路由器之间的链路上，属于另一段网，主机连直接递帧都递不到它。网关不看顺眼不顺眼，只看在不在同一子网：用块大小法一验，主机.60 落在 32 到 63 这一块，直连接口.62 也在同一块，.1 不在，.1 就当不了网关。凡是网关题，先验同子网，再谈顺眼。

【套路】

由拓扑定掩码和网关，固定走两步。第一步定掩码：找到和主机直连的路由器接口，认它的前缀，换算成掩码。本题直连接口是 /27，前三个字节全是 255，第四字节前 3 位是 1、后 5 位是 0，得 255.255.255.224。第二步定网关：把候选地址逐个验同子网，块大小等于 256 减去掩码字节，主机和候选地址落在同一块里的，才有资格当网关。本题块大小是 32，.60 和.62 都在 32 到 63 这一块，.1 不在，所以网关是.62。

【类比】

想象一栋楼里的住户要出门。住户所在楼层的出口，就是和他同楼层的那个楼梯口，这就是默认网关；隔壁楼的出口再气派、再像正门，也不是他这层楼的出口，他走不过去。子网掩码就是“楼层”的划分规矩：先看自己在第几层，再找本层的出口。拿着隔壁楼的出口当自家出口，门都递不到。

【口诀】掩码看直连，网关看同网。

第 37 题 · SDN 的南向接口（正确答案：B，南向接口）

题干与选项：

在 SDN 网络体系结构中，控制器向数据平面的交换机下发流表时，使用的是什么接口（）。

- A. 东向接口
- B. 南向接口
- C. 西向接口
- D. 北向接口

入手点： SDN 接口先定位"谁和谁在说话"，再对方向：控制器往下对交换机说话就是南向。

题干专有名词从零解释：

- SDN（软件定义网络）：把网络的"大脑"从每台交换机里抽出来、集中成一个控制器的做法。传统网络里每台交换机自己决定怎么转发；SDN 里交换机只剩"手脚"，按控制器下发的规则转发。
- 控制平面：SDN 的中间层，就是控制器，负责算路、下发规则。
- 数据平面：SDN 的下面一层，就是交换机，只按规则转发数据，不管怎么算路。
- 应用平面：SDN 的上面一层，跑各种网络应用（比如流量统计、访问控制），通过控制器间接指挥网络。
- 流表：控制器下发给交换机的规则表，每条规则写着"符合什么特征的数据包，往哪个口转发"；交换机收到数据包就查这张表。
- 南向接口：控制器 ↔ 下面的交换机之间的接口，下发流表就走它，最典型的协议是 OpenFlow。
- 北向接口：控制器 ↔ 上面的应用程序之间的接口，给应用层调用控制器用，本质是一组 API。
- 东向接口 / 西向接口：控制器 ↔ 其他控制器之间的接口，多个控制器之间互通信息；东和西不分家，一起记。

考查知识点： SDN 三层结构与四个接口的方向归属。

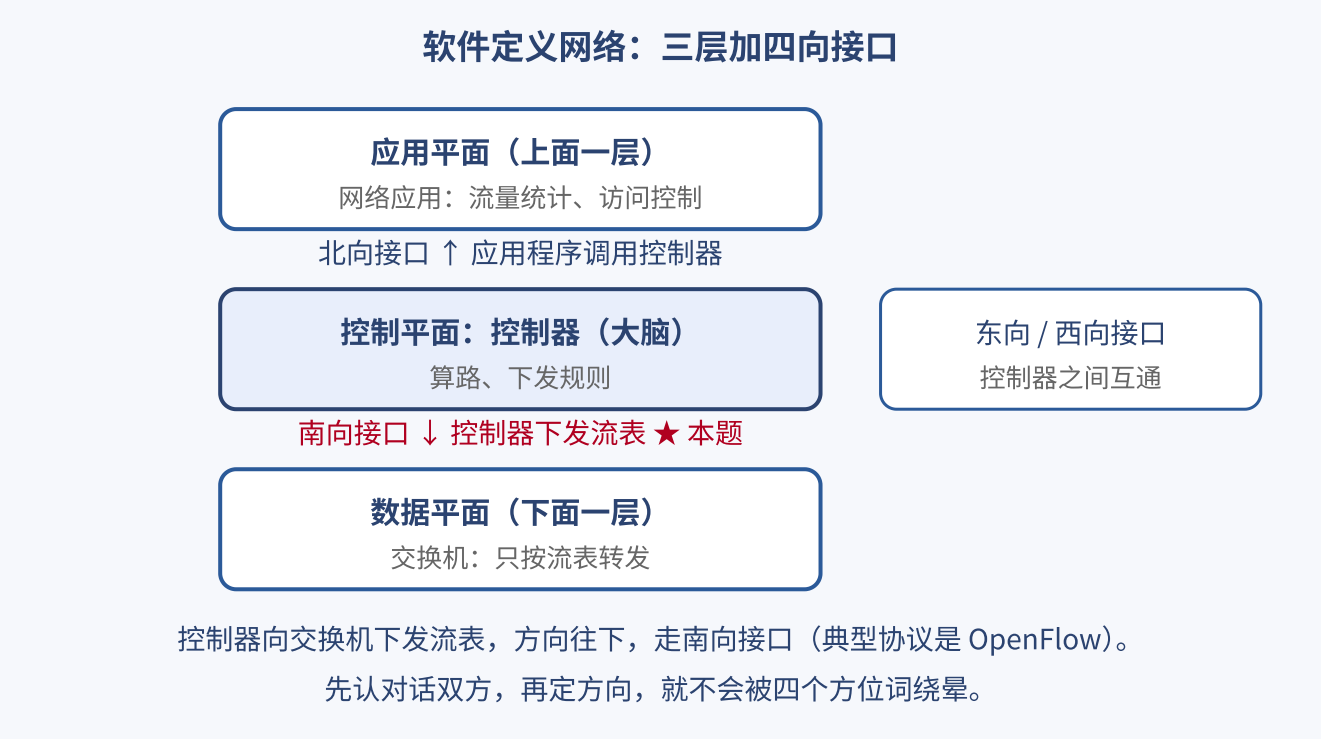
正确选项为什么对： 题干的动作是"控制器向数据平面的交换机下发流表"，说话双方是控制器和交换机，方向是控制器往下——按方位定义，这正是南向接口。举例：控制器算好一条"去往 192.168.1.0 网段的数据包走 3 号口"的规则，用 OpenFlow 协议从南向接口下发给交换机，交换机写进流表，之后的数据包就按表转发。

错误选项为什么错＋举例：

- A（东向接口）：东向是控制器与控制器之间。举例：网络里有两台控制器，A 控制器把自己管的那片网络的信息告诉 B 控制器，走东向接口——对话里没有交换机。
- C（西向接口）：同东向，也是控制器之间互联。举例：东西向是一对方向，记"控制器之间"就行；下发流表给交换机不走这里。
- D（北向接口）：北向朝上，给应用层调用。举例：一个流量统计应用想问控制器"现在各链路多忙"，它通过北向接口调控制器的 API——方向正好反了，是应用找控制器，不是控制器指挥交换机。

笔记本一句话： 第 37 题＋SDN 接口看方位：下发流表往下走是南向，给应用调用往上是北向，控制器之间是东西向。

【图像】



软件定义网络三层加四向接口方位图，下发流表走南向

【定位】计算机网络 · 第 4 章 · 软件定义网络的三层结构与接口方向

【易混对比】

- 南向接口 vs 北向接口：南向是控制器往下对交换机说话的接口，下发规则表走它；北向是控制器往上对应用程序说话的接口，应用程序调用控制器走它。方向一上一下，刚好相反，题干说“控制器向交换机下发”，方向往下，只能是南向。
- 东向接口和西向接口 vs 南向接口：东向和西向是一对方向，管的是控制器和控制器之间互通信息，对话里没有交换机；南向管的是控制器和交换机之间，对话里没有第二个控制器。先认清对话双方，就不会把“控制器之间”当成“控制器对交换机”。
- 控制平面 vs 数据平面：控制平面是中间层的大脑，也就是控制器，负责算路和下发规则；数据平面是下面一层的手脚，也就是交换机，只按规则表转发数据，不管怎么算路。下发流表就是大脑给手脚派活，方向往下。

【坑位】

最核心的陷阱，是把“接口”四个字当成按名字猜方向。东、南、西、北四个词摆在一起，很容易凭感觉蒙一个，或者把北向记成“控制器发命令”因为控制器在上面。但接口方向不是猜出来的，是按对话双方和动作方向定的：动作是控制器向交换机下发规则，交换机在控制器下面，方向往下，所以是南向。做题时先写出“谁对谁说话、往哪个方向”，再对方向，就不会被四个方位词绕晕。

【套路】

判断软件定义网络接口方向，固定走两步。第一步，认对话双方：控制器对交换机、控制器对应用程序、控制器对控制器，分别对应不同的接口。第二步，定方向：控制器往下对交换机是南向，控制器往上对应用程序是北向，控制器之间互通是东向和西向。本题对话双方是控制器和交换机，动作是往下发规则，两步一对，就是南向。

【类比】

想象一个餐厅。楼上的老板想出新菜谱，通过电话告诉后厨，后厨按菜谱做菜，这就是控制器往下给交换机下发规则，走南向。顾客想点菜、想问今天有什么推荐，是通过服务员往上找老板，这就是应用程序往上调用控制器，走北向。而两个分店的老板之间通电话商量进货，这是老板之间的沟通，走东向和西向。同样是打电话，找谁、往哪个方向，接口就不同。

【口诀】往下发流表走南向，往上调应用走北向。

第 38 题 · TCP 超时后拥塞窗口重新涨到 16KB 的时间（正确答案：C，11 RTT）

题干与选项：

甲、乙已建立 TCP 连接，MSS = 1 KB，甲一直有数据要发送。当拥塞窗口 cwnd = 16 KB 时发生了超时，cwnd 再次增长到 16 KB 至少需要多长时间（ ）。

- A. 4 RTT
- B. 5 RTT
- C. 11 RTT
- D. 16 RTT

入手点： 超时先做两件事（门限减半、窗口归 1），再先翻倍冲到门限、后 +1 爬到目标，两段 RTT 分开数再相加。

题干专有名词从零解释：

- TCP 连接：甲、乙之间已建立的可靠传输通道，数据按字节流发。
- MSS（最大报文段长度）：TCP 一次能发的最大数据块，本题 1 KB。
- cwnd（拥塞窗口）：发送方一次能发出去、还没等到确认的数据量上限，单位常写成 KB；窗口越大发得越猛。
- ssthresh（慢启动门限）：一个分界值，cwnd 在它下面用"慢启动"涨法，到了它就换成"拥塞避免"涨法。
- RTT（往返时间）：发一个包、收到它的确认所花的时间。
- 超时：发出去的数据在规定时间内没等到确认，发送方判定路上堵出事了，立刻收紧窗口。注意：超时和"收到 3 个冗余 ACK"是两回事，后者只把门限减半、窗口减半，不归 1；本题是超时，按归 1 算。
- 慢启动：cwnd < ssthresh 时的涨法，每过 1 个 RTT 翻一倍（1→2→4→8……），是指数涨。
- 拥塞避免：cwnd ⩾ ssthresh 时的涨法，每过 1 个 RTT 只 +1（8→9→10……），是线性涨。

考查知识点： TCP 拥塞控制：超时处理与两阶段窗口增长。

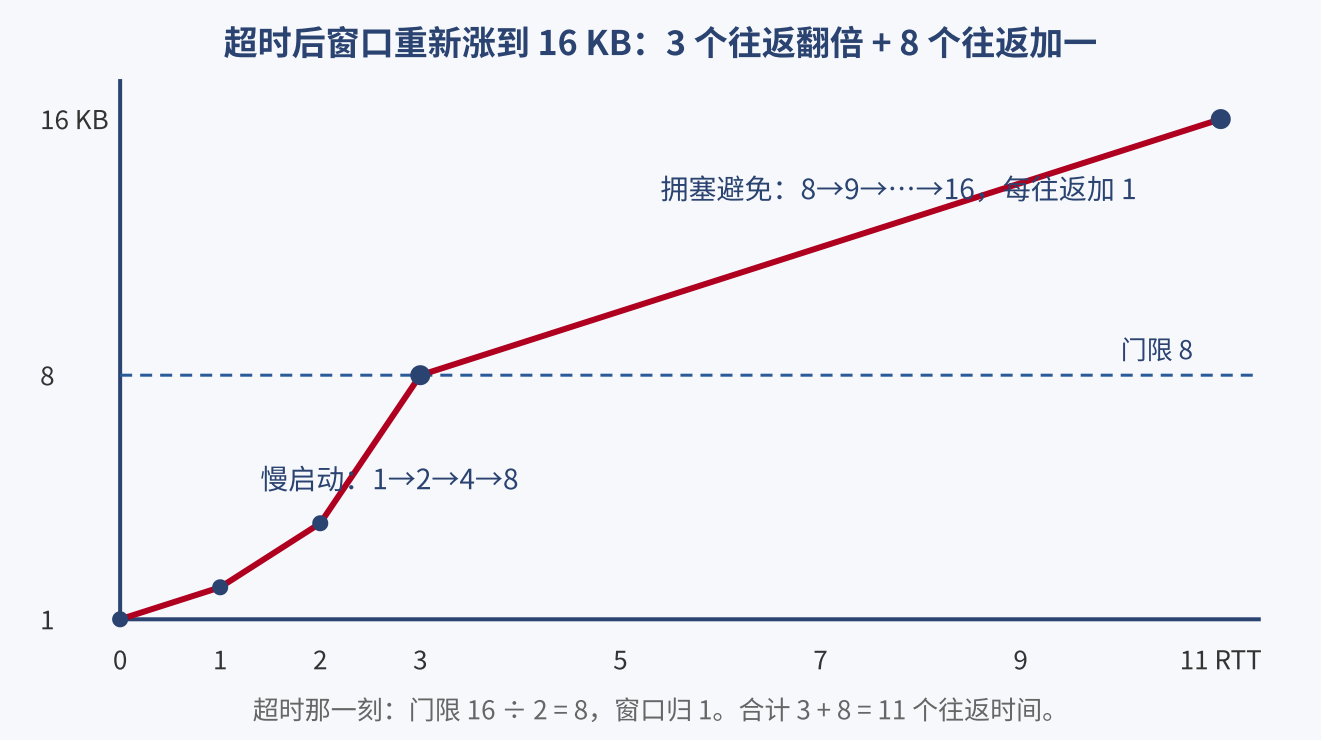
正确选项为什么对： 分三步推演。第一步，超时那一刻做两件事：门限 ssthresh 设为当前 cwnd 的一半，16 ÷ 2 = 8 KB；cwnd 直接重置为 1 个 MSS = 1 KB，从头再来。第二步，慢启动翻倍冲到门限：起点 cwnd = 1 KB，第 1 个 RTT 后 1→2，第 2 个 RTT 后 2→4，第 3 个 RTT 后 4→8，到达门限 8，慢启动结束，共用 3 个 RTT。第三步，拥塞避免 +1 爬到目标：8→9→10→11→12→13→14→15→16，每 RTT +1，共 8 个 RTT。合计 3 + 8 = 11 RTT。

错误选项为什么错＋举例：

- A（4 RTT）：当成慢启动一路翻倍到 16（1→2→4→8→16）。举例：翻倍只在 cwnd < 门限 8 时成立，到了 8 就必须换挡成 +1；一路翻倍等于把门限当不存在。
- B（5 RTT）：慢启动段数对了一半，但拥塞避免段严重少算。举例：8 涨到 16 靠 +1 需要 8 步（8→9→10→11→12→13→14→15→16），不是 2 步；5 RTT 等于只数了 3 + 2。
- D（16 RTT）：当成从 1 开始全程每 RTT +1。举例：全程 +1 是 1→2→……→16 要 15 步（或按它的口径 16 步），完全没用上慢启动的翻倍——慢启动阶段的 3 个 RTT 被白白拉长了。

笔记本一句话： 第 38 题＋超时先两件事：门限减半、窗口归 1；再先翻倍冲到门限、后 +1 爬到目标，两段 RTT 分开数再相加。

【图像】



拥塞窗口随往返时间变化曲线：先翻倍到门限再加一爬到目标

【定位】计算机网络 · 第 5 章 · 传输控制协议拥塞控制中的超时处理与两阶段增长

【易混对比】

- 超时 vs 收到 3 个冗余确认：超时是发出去的数据在规定时间内没等到确认，发送方判定路上出了问题，处理办法是门限减半、窗口直接归 1，从头再来；收到 3 个冗余确认是后面连续的确认都重复指向同一个缺口，处理办法是门限减半、窗口减半，不归 1。本题写的是超时，按归 1 算，不能按减半算。
- 慢启动涨法 vs 拥塞避免涨法：窗口在门限下面时，每过一个往返时间翻一倍，是指数涨；窗口到了门限之后，每过一个往返时间只加 1，是线性涨。两段涨法不同，数往返时间的办法也不同，不能一段的算法套到另一段。
- 门限 vs 窗口：门限是换挡的分界值，超时那一刻设为原来窗口的一半；窗口是当前实际能发的量，超时那一刻直接归 1。门限管“什么时候换挡”，窗口管“现在能发多少”，两个数各有各的算法。

【坑位】

最核心的陷阱，是慢启动一路翻倍翻到目标。1、2、4、8、16 这串翻倍数着很顺，4 个往返时间就到 16，看起来又快又漂亮。但翻倍只在窗口低于门限时成立，门限是 8，翻到 8 就必须换挡，改成每往返时间加 1，8 到 16 要再花 8 个往返时间。一路翻倍等于把门限当成不存在，少算了整整一段爬升。记住换挡点：到门限就换挡，换挡后的爬升要一步一步数。

【套路】

超时后窗口重新涨到目标，固定走三步。第一步，超时那一刻做两件事：门限设为原来窗口的一半，窗口归 1。第二步，翻倍段：从 1 开始每往返时间翻一倍，数到门限为止，本题是 1 到 2 到 4 到 8，用 3 个往返时间。第三步，加一段：从门限开始每往返时间加 1，数到目标为止，本题是 8 到 16，用 8 个往返时间。两段相加，3 加 8 得 11 个往返时间。分段数、再相加，是这类题唯一不会错的数法。

【类比】

想象开车上坡起步。超时好比路上突然堵死，司机先把油门松到底、回到起步挡，这就是窗口归 1，同时把“可以放心加速的上限”砍半，这就是门限减半。起步后先挂低挡猛加速，速度翻着倍往上冲，冲到上限就换挡，这就是翻倍段；换挡之后不敢再猛踩，只能一格一格往上加，这就是加一段。猛冲段和慢加段的节奏完全不同，算总时间必须两段分开数。

【口诀】超时门限半，窗口归一；翻倍到门限，加一爬到底。

第 39 题 · TCP 断开连接：双方进入 CLOSED 的时间（正确答案：D，1650 ms，75 ms）

题干与选项：

客户 C 和服务器 S 已建立 TCP 连接，RTT = 50 ms，MSL = 800 ms。数据传输完成后，C 主动请求断开连接。从 C 发出 FIN 的时刻算起，C 和 S 进入 CLOSED 状态所需的时间至少各是多少（）。

- A. 850 ms，50 ms
- B. 1650 ms，50 ms
- C. 850 ms，75 ms
- D. 1650 ms，75 ms

入手点： 先认谁主动（C 先发 FIN，2MSL 的等待记在 C 头上），再沿时间线数：单程时延按半个 RTT 算，C 的计时从"发出最后 ACK"那一刻起算。

题干专有名词从零解释：

- FIN：断开请求报文，"我没有数据要发了"的意思。
- ACK：确认报文，"我收到了"的意思。
- 四次挥手：断开连接的四步礼貌告别——一方发 FIN，对方回 ACK，对方再发 FIN，第一方再回 ACK。
- 主动关闭方：先发 FIN 的一方，本题是客户 C；被动关闭方：后发 FIN 的一方，本题是服务器 S。
- TIME_WAIT：主动方发出最后一个 ACK 后进入的等待状态，等 2MSL 再关闭。
- MSL（最长报文段寿命）：一个报文在网络里最多能存活的时间，本题 800 ms；2MSL = 1600 ms。
- CLOSED：连接彻底关闭的状态。
- RTT：往返时间，本题 50 ms；单程时延 = RTT ÷ 2 = 25 ms。
- FIN_WAIT / LAST_ACK：挥手过程中的中间状态名，时间线上提到时知道"还在等对方"就行，不必深究。

考查知识点： TCP 四次挥手的时间线推演，TIME_WAIT 的归属与计时起点。

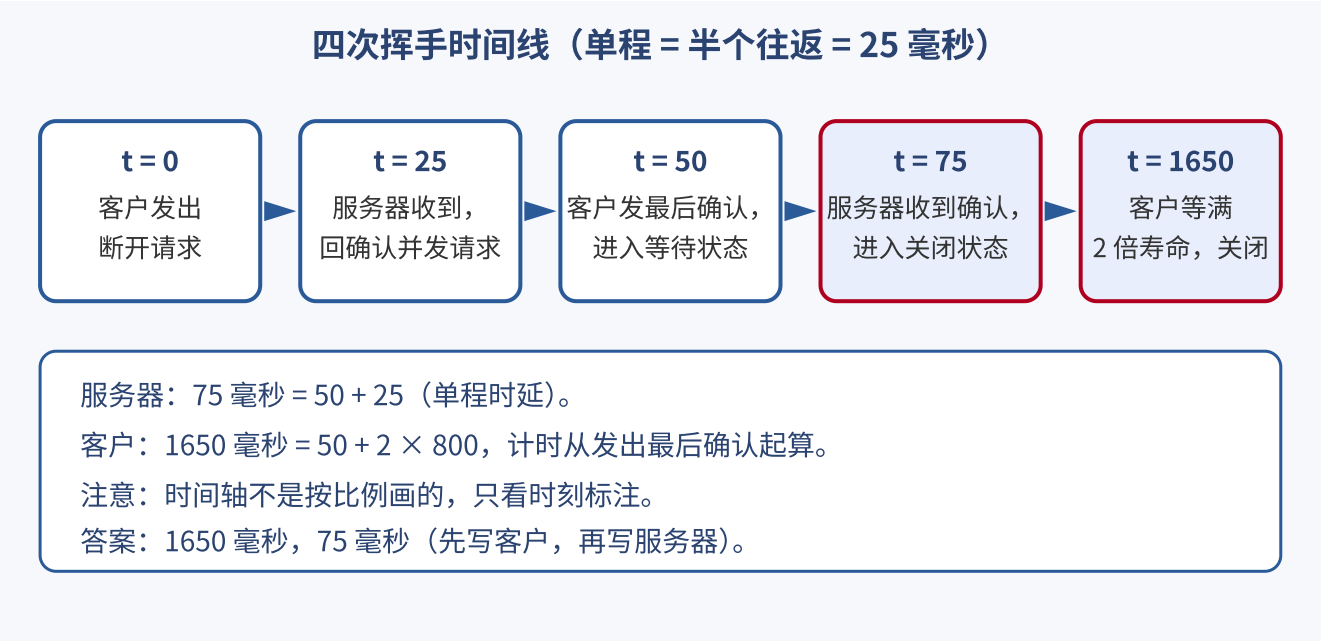
正确选项为什么对： 沿时间线推演（设 t = 0 为 C 发出 FIN 的时刻，按"最少时间"算：S 收到 FIN 后立刻回 ACK 并紧接着发自己的 FIN，两个动作可以合并在同一时刻发出）。t = 0：C 发出 FIN①，进入 FIN_WAIT。t = 25 ms（半个 RTT，单程）：S 收到 FIN①，回 ACK②、同时发 FIN③，进入 LAST_ACK。t = 50 ms（1 个 RTT）：C 收到 ACK② 和 FIN③，发出最后的 ACK④，进入 TIME_WAIT，开始等 2MSL。t = 75 ms（50 + 25）：S 收到 ACK④，进入 CLOSED，S 的总耗时是 75 ms。t = 50 + 2×800 = 1650 ms：C 的 TIME_WAIT 计时结束，进入 CLOSED，C 的总耗时是 1650 ms。所以是 D。

错误选项为什么错＋举例：

- A（850 ms，50 ms）：C 只等了 1 个 MSL（50 + 800 = 850），S 的时间也少算了最后一段 ACK 的单程。举例：TIME_WAIT 等的是 2MSL 不是 1MSL，少算一个 800 ms；S 也不是在 C 发出 ACK 的 t = 50 ms 就关闭，而是要等 ACK 传到 S 的 t = 75 ms。
- B（1650 ms，50 ms）：C 算对了，但 S 错了。举例：S 关闭的时刻是"收到最后 ACK"的 t = 75 ms，不是"C 发出 ACK"的 t = 50 ms——发出和收到之间隔着 25 ms 单程时延。
- C（850 ms，75 ms）：S 算对了，但 C 只等了 1 个 MSL。举例：50 + 800 = 850 是只等 1 个 MSL 的算法；TIME_WAIT 的规定是 2MSL，必须是 50 + 1600 = 1650 ms。

笔记本一句话： 第 39 题＋主动关的一方等 2MSL 且从"发出最后 ACK"起算，被动方收到最后 ACK 即关；所有单程时延按半个 RTT 数。

【图像】



四次挥手时间线：从发出断开请求到双方进入关闭状态的五个时刻

【定位】计算机网络 · 第 5 章 · 传输控制协议四次挥手与主动关闭方的等待

【易混对比】

- 主动关闭方 vs 被动关闭方：主动关闭方是先发出断开请求的一方，本题是客户；被动关闭方是后发出断开请求的一方，本题是服务器。等一段固定时间的等待状态记在主动关闭方头上，被动关闭方没有这段等待。
- 发出最后一个确认 vs 对方收到最后一个确认：主动关闭方的等待计时，从它发出最后一个确认的那一刻起算；被动关闭方进入关闭状态的时刻，是它收到最后一个确认的那一刻。发出和收到之间隔着半个往返时间的单程时延，两个时刻不能画等号。
- 等 2 倍最长报文段寿命 vs 等 1 倍：主动关闭方要等的时间是 2 倍，不是 1 倍。少算一倍，主动关闭方的时间就少了 800 毫秒，正好掉进错误选项。

【坑位】

最核心的陷阱，是被动关闭方的时间少算了最后一段单程。很多人算到“客户发出最后一个确认”就停了，给服务器记 50 毫秒。但服务器关闭的时刻不是客户发出确认的时刻，而是确认传到服务器的时刻，中间还有 25 毫秒单程时延，所以是 75 毫秒。记住一条：凡是“对方进入某个状态”的时刻，都要算到报文真正到达对方的那一刻，发出时刻不等于到达时刻。

【套路】

四次挥手计时固定走三步。第一步，认谁主动：主动关闭方背一段 2 倍最长报文段寿命的等待，被动关闭方不背。第二步，沿时间线数单程：每段单程时延按半个往返时间算，本题是 25 毫秒，发出、收到、再发出、再收到，一步一步往后推。第三步，分开收口：被动关闭方收到最后一个确认就关闭；主动关闭方从发出最后一个确认起算，等满 2 倍最长报文段寿命才关闭。本题被动方是 50 加 25 得 75 毫秒，主动方是 50 加 1600 得 1650 毫秒。

【类比】

想象两个人打电话告别。先挂电话的人说完“再见”不能立刻把电话线拔掉，要多等一会儿，防着对方还有话没说完、或者自己的“再见”对方没听清，这就是主动关闭方等 2 倍最长报文段寿命。后挂电话的人听到对方的

“再见”就可以挂了，不用再等，这就是被动关闭方收到最后一个确认就关闭。先挂的人多等，后挂的人听到就挂，等待的担子只落在先挂的人头上。

【口诀】主动等二倍，被动收到关。

第 40 题 · HTTP/1.1 取一个网页 + 一张图的总时间（正确答案：B，40 ms）

题干与选项：

主机 H 用 HTTP/1.1 浏览服务器 S 上的页面 news408.html（大小 1 MSS），页面中引用了同目录下的 1 幅图像（大小 3 MSS），RTT = 10 ms，忽略首部开销和段传输时延，H 已完成域名解析。从 H 请求与 S 建立 TCP 连接的时刻算起，到收齐全部内容，至少需要多长时间（ ）。

- A. 30 ms
- B. 40 ms
- C. 50 ms
- D. 60 ms

入手点： HTTP/1.1 只握手一次、请求串行排队；总时间分三段数：建连 1 RTT + 小文件 1 RTT + 大文件按慢启动分批数 RTT。

题干专有名词从零解释：

- HTTP/1.1：超文本传输协议 1.1 版，默认持久连接——建一次 TCP 连接，多个文件排队在同一条连接上取，不用反复握手。对比 HTTP/1.0：每取一个文件就要新建一次 TCP 连接。
- 持久连接：一条 TCP 连接复用到底，本题 html 和图像共用一次建连。
- 串行：请求只能一个接一个发，不能并行。原因：浏览器要先把 html 取回来、解析出"里面引用了一张图"，才知道要再发第二个请求。
- 三次握手：建立 TCP 连接的三步——客户端发 SYN，服务器回 SYN+ACK，客户端再回 ACK；从发出 SYN 到能带上请求，计 1 个 RTT（第三次的 ACK 包里可以直接带上 HTTP 请求，这是"至少"的算法）。
- MSS：TCP 一次能发的最大数据块，本题 html 是 1 MSS，图像是 3 MSS。
- RTT：往返时间，本题 10 ms。
- 慢启动：连接刚建好时 cwnd = 1 MSS，每过 1 个 RTT 翻一倍。慢启动约束的是服务器发数据的节奏：窗口只有 1 MSS 时，一次只能发 1 MSS，收到确认、窗口涨到 2 MSS 后才能一次发 2 MSS。
- 域名解析：把域名翻成 IP 地址的过程，题干说已完成，所以不计时间。

考查知识点： HTTP/1.1 持久连接与串行请求，TCP 慢启动约束下的分段计时。

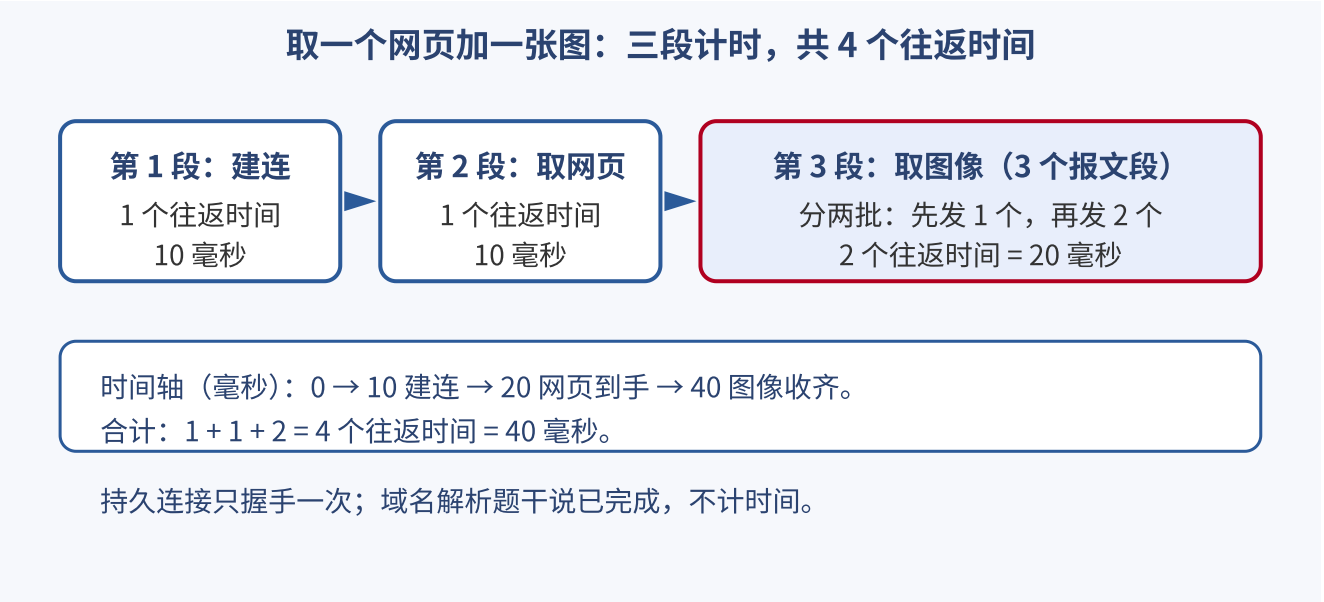
正确选项为什么对： 分三段推演。第一段，建连接：三次握手，计 1 个 RTT（累计 1 RTT）。第二段，取 html：发请求，1 MSS 的页面传回，计 1 个 RTT（累计 2 RTT），此时浏览器才知道还有张图。第三段，取图像（3 MSS）：发请求后，服务器受慢启动限制不能一次全发——先发 1 MSS（cwnd = 1），过 1 个 RTT 后 cwnd 涨到 2，再发剩下的 2 MSS，又过 1 个 RTT 收齐，本段 2 个 RTT（累计 4 RTT）。合计 4 RTT = 4 × 10 ms = 40 ms。

错误选项为什么错＋举例：

- A（30 ms）：图像按 1 个 RTT 一次传完，漏了慢启动。举例：3 MSS 一次发完要求 cwnd ⩾ 3，但连接刚建好时 cwnd 只有 1，必须分两批（1 MSS + 2 MSS）花 2 个 RTT；按 1 个 RTT 算等于把慢启动当不存在。
- C（50 ms）：多算了一个 RTT，典型是给图像请求又算了一次建连接。举例：HTTP/1.1 是持久连接，握手只花一次；给图像再算一次建连等于把 1.1 当成了 1.0。
- D（60 ms）：在 C 的基础上再多算一段，通常是把域名解析也计了时间。举例：题干明说"已完成域名解析"，解析时间不计；多算一段就是 60 ms 的来历。

笔记本一句话： 第 40 题＋HTTP/1.1 只握手一次、请求串行排队；总时间 = 建连 1 RTT + 每个小文件 1 RTT + 大文件按慢启动分批数 RTT。

【图像】



三段计时时间线：建连一段、取网页一段、取图像分两批一段

【定位】计算机网络 · 第 6 章 · 超文本传输协议持久连接与慢启动约束下的分段计时

【易混对比】

- 持久连接 vs 非持久连接：持久连接是建一次连接、多个文件排队在同一条连接上取，握手只花一次；非持久连接是每取一个文件就要新建一次连接，握手要花多次。本题用的是 1.1 版的持久连接，建连只算一次。
- 串行请求 vs 并行请求：串行是一个接一个发，浏览器要先把网页取回来、解析出里面引用了一张图，才知道要再发第二个请求；并行是多个请求同时发。本题的两个请求有先后依赖，只能串行，不能把两个文件的传输时间重叠着算。
- 小文件一段 vs 大文件分批：网页只有 1 个最大报文段，一次就能传完，算 1 个往返时间；图像有 3 个最大报文段，连接刚建好时窗口只有 1 个最大报文段，不能一次全发，必须分两批，先发 1 个、窗口涨到 2 再发剩下 2 个，算 2 个往返时间。文件大小不同，计时办法也不同。

【坑位】

最核心的陷阱，是图像按 1 个往返时间一次传完。3 个最大报文段看着不大，顺手就按一段算了，总时间变成 30 毫秒。但连接刚建好时发送窗口只有 1 个最大报文段，服务器第一批只能发 1 个，等确认回来、窗口涨到 2，才能发剩下 2 个，图像这一段必须花 2 个往返时间。记住一条：凡是问“至少需要多长时间”又给了文件大小和最大报文段，都要先问窗口够不够一次发完，不够就分批数。

【套路】

持久连接下取多个文件计时，固定分三段数。第一段建连：三次握手算 1 个往返时间。第二段取小文件：每个能一次传完的文件各算 1 个往返时间，本题网页是 1 个最大报文段，算 1 个往返时间。第三段取大文件：按慢启动分批数，窗口从 1 开始翻倍，第一批发 1 个、第二批能发 2 个，3 个最大报文段分两批，算 2 个往返时间。三段相加，1 加 1 加 2 得 4 个往返时间，乘以 10 毫秒得 40 毫秒。

【类比】

想象去快递站取两个包裹。先办一次取件登记，这就是建连，只办一次。第一个包裹很小，一次就能抱走，这就是取网页。第二个包裹很大，但刚开始只能用一只小推车，一次只能装 1 件，跑一趟回来换了大推车，才能一次装走剩下 2 件，这就是取图像分两批。小推车换大推车的过程，就是窗口从 1 涨到 2 的过程，趟数不能少算。

【口诀】握手一次，串行取件，大件分批。

附注

- 答案核对：本稿 40 道题的答案逐题对照了任务给定的官方答案（数据结构 1B 2D 3B 4C 5D 6D 7B 8D 9D 10A 11D；计组 12A 13B 14A 15C 16A 17C 18B 19D 20A 21C 22C；操作系统 23D 24A 25C 26B 27C 28D 29A 30D 31B 32A；计网 33B 34C 35B 36D 37B 38C 39D 40B），底本 4 份拆解的答案与官方答案 40 题全部一致，无冲突。
- 第 36 题说明：本题拓扑细节已与原卷拓扑图在线预览核对（H 的 IP 为 192.168.1.60，与它直连的路由器接口是 192.168.1.62/27；192.168.1.1/30 在两路由器之间的链路上，不在 H 所在网段接口一侧）。

核对结果

核对说明（适用于全部 40 题）： 第 0 节"题干与选项"系根据 2026-10-21 四科拆解（本身源自真题解析与讲解）与终审版解析逐题还原（非原卷逐字照录）。2026-10-25 已用用户夸克网盘中的原卷（全部文件 / 王铎熹 / 考研真题 / 408真题 / 统考真题 / 2022年计算机408统考真题.pdf，8 页）在线预览逐项核对：原文件未能下载到本地，核对以在线预览为准。核对结果：40 题答案仍全部等于官方答案（数据结构 1B 2D 3B 4C 5D 6D 7B 8D 9D 10A 11D；计组 12A 13B 14A 15C 16A 17C 18B 19D 20A 21C 22C；操作系统 23D 24A 25C 26B 27C 28D 29A 30D 31B 32A；计网 33B 34C 35B 36D 37B 38C 39D 40B）；还原出入 5 处已按原卷修正（见下）；其余 35 题关键数据与还原一致。

逐题核对结果：

- 第 3 题：C 选项实际为"仅 II、III"（原还原为"I、II"），已修正；C 的错因分析同步改写；答案 B 不变。
- 第 7 题：c 权实际为 1（原还原为 3），f 权实际为 4（原还原为 3），其余 8 条边一致；ve / vl / 余量已按实际权值重算（工期仍 12，g 余量仍 6、全场最大，关键路径为 1→3→4→6）；答案 B 不变。
- 第 13 题：C 选项下限实际为 -2^{32}（原还原为 $-(2^{32}-1)$），已修正；C 的错因分析同步改写；答案 B 不变。
- 第 18 题：A 选项确为"仅 I、II"，与还原一致，无改动；答案 B 不变。
- 第 20 题：D 选项实际排序为"预处理、汇编、链接、编译"（原还原为"汇编、链接、编译、预处理"），已修正；D 的错因分析同步改写；答案 A 不变。
- 第 27 题：A 确与还原一致；D 实际为"内核态只能执行非特权指令"（原还原为"内核态不能执行非特权指令"），已修正；D 的错因分析同步改写；答案 C 不变。
- 第 36 题：拓扑确与还原一致（H 192.168.1.60，直连接口 192.168.1.62/27），并补充精确信息：192.168.1.1/30 在两路由器之间的链路上；答案 D 不变。
- 其余 33 题：关键数字、条件、表格与还原一致，无改动。